

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 РАЗРАБОТКА ОБЩЕЙ СТРУКТУРЫ МИКРО-ЭВМ.....	5
2.1 Функциональный состав микро-ЭВМ	5
2.2 Разработка системы команд	7
2.3 Описание взаимодействия всех блоков микро-ЭВМ при выполнении команд программы	9
2 РАЗРАБОТКА ОСНОВНЫХ УСТРОЙСТВ МИКРО-ЭВМ	12
2.1 Запоминающее устройство	12
2.2 Устройство управления	15
2.3 Регистры общего назначения.....	19
2.4 Блок общих операций.....	21
2.4.1 Блок нулевой итерации	23
2.4.2 Блоки арбитража к общим шинам	24
2.4.3 Блок операции NOP (нет операции)	26
2.4.4 Блок операции JMP (безусловный переход)	28
2.4.5 Блок операции M->R (чтение из RAM в РОН)	30
2.4.6 Блок операции R->M (запись из РОН в RAM)	32
2.5 Арифметико-логическое устройство	34
2.5.1 Блоки арбитража к общим шинам	36
2.5.2 Блок регистров арифметико-логического устройства.....	38
2.5.3 Блок операции ROR (циклический сдвиг вправо).....	39
2.5.4 Блок операции ADDC (сложение с переносом).....	41
2.5.5 Блок операции INCS (инкремент по флагу S).....	43
2.5.6 Блок операции NOT (побитовое инвертирование).....	45
2.5.7 Блок операции AND (побитовое «И»).....	47
2.6 Стековое запоминающее устройство.....	49
2.6.1 Блоки арбитража к общим шинам.....	50
2.6.2 Блок схемы стека.....	51
2.7 КЭШ-память.....	54
2.7.1 Блок данных КЭШа.....	56
2.7.2 Второстепенные блоки КЭШа.....	61
2.7.3 Блок управления КЭШа.....	63
2.8 Контроллер прямого доступа к памяти.....	79
2.9 Предсказатель переходов.....	82
2.10 Организация арбитража.....	85
3 ПОЛНАЯ ВРЕМЕННАЯ ДИАГРАММА ФУНКЦИОНАЛЬНОГО МОДЕЛИРОВАНИЯ РАБОТЫ МИКРО-ЭВМ И ЕЁ АНАЛИЗ	87
ЗАКЛЮЧЕНИЕ	90
СПИСОК ЛИТЕРАТУРЫ.....	91
ПРИЛОЖЕНИЕ А Система команд микро-ЭВМ.....	92

ПРИЛОЖЕНИЕ Б	Схема комбинированная структурная совмещённого блока ЗУ.....	94
ПРИЛОЖЕНИЕ В	Схема комбинированная структурная блока устройства управления.....	95
ПРИЛОЖЕНИЕ Г	Схема комбинированная структурная блока специальных регистров.....	96
ПРИЛОЖЕНИЕ Д	Схема комбинированная структурная блока регистров общего назначения.....	97
ПРИЛОЖЕНИЕ Е	Схема комбинированная структурная блока общих операций.....	98
ПРИЛОЖЕНИЕ Ж	Схема комбинированная структурная блока операции «непосредственный переход».....	99
ПРИЛОЖЕНИЕ И	Схема комбинированная структурная блока операции «M->R».....	100
ПРИЛОЖЕНИЕ К	Схема комбинированная структурная блока операции «R->M».....	101
ПРИЛОЖЕНИЕ Л	Схема комбинированная структурная блока арифметико-логического устройства.....	102
ПРИЛОЖЕНИЕ М	Схема комбинированная структурная блока операции «циклический сдвиг вправо».....	103
ПРИЛОЖЕНИЕ Н	Схема комбинированная структурная блока операции «сложение с переносом».....	104
ПРИЛОЖЕНИЕ П	Схема комбинированная структурная блока операции «инкремент по флагу s».....	105
ПРИЛОЖЕНИЕ Р	Схема комбинированная структурная блока операции «побитовое инвертирование».....	106
ПРИЛОЖЕНИЕ С	Схема комбинированная структурная блока операции «побитовое 'и'».....	107
ПРИЛОЖЕНИЕ Т	Схема комбинированная структурная блока стекового устройства.....	108
ПРИЛОЖЕНИЕ У	Схема комбинированная структурная исполнительного блока стекового запоминающего устройства.....	109
ПРИЛОЖЕНИЕ Ф	Схема комбинированная структурная блока КЭША.....	110
ПРИЛОЖЕНИЕ Х	Схема комбинированная структурная блока данных КЭША.....	111
ПРИЛОЖЕНИЕ Ц	Схема комбинированная структурная блока флагов КЭША.....	112
ПРИЛОЖЕНИЕ Ш	Схема комбинированная структурная блока микро-ЭВМ.....	113
ПРИЛОЖЕНИЕ Щ	Схема комбинированная структурная блока управления предсказателя переходов.....	114

ВВЕДЕНИЕ

Разрабатываемая микро-ЭВМ будет иметь шины:

- шина данных разрядностью 8 бит
- адресная шина разрядностью 12 бит
- управляющая шина (разрядность будет определена в процессе разработки)

Архитектура принстонская, следовательно, шина для ROM и RAM будет общая, что нужно учитывать при разработке операций. При непосредственной адресации операнд располагается в адресном поле команды (ROM). Пример: `mov ax, 40h`

Количество регистров общего назначения – 20шт.

Тип КЭШ-памяти – с множественно-ассоциативным отображением ($k = 2$)

Алгоритм замещения строк – без анализа, что упрощает разработку (но уменьшает эффективность КЭШа). Тип синхронизации КЭШа с памятью один из самых сложных (если не самый) – флаговая отложенная синхронизация.

Операции АЛУ тривиальны, их нужно будет реализовать только на логических элементах. Арбитраж шин (централизованный параллельный) так же как и предсказатель переходов (4 бита, шаблон – исключающее “ИЛИ” IP и GHR) будут реализованы отдельно, т.к. разрабатываемая микро-ЭВМ будет не конвейеризированная, а лишь многотактовая (подразумевается процессор).

Стек будет реализован отдельным блоком и с направлением роста вниз и объемом 12 байт, ничего сложного.

Контроллер DMA задан объемом 8 байт с начальным адресом 0xF в памяти RAM, с ним придется повозиться, т.к. в предыдущем семестре на лабораторных подобного блока не разрабатывалось и заготовки отсутствуют.

1 РАЗРАБОТКА ОБЩЕЙ СТРУКТУРЫ МИКРО-ЭВМ

1.1 Функциональный состав микро-ЭВМ

Корневыми блоками микро-ЭВМ являются:

- блок запоминающего устройства (ROM и RAM в одном блоке)
 - блок памяти ROM
 - блок памяти RAM
 - схема управления блоками памяти
- блок устройства управления
 - блок специальных регистров
 - схема декодирования команд
 - схема контроля итераций операций
 - схема контроля задержки при взаимодействии КЭШа с памятью
 - схема контроля переполнения регистра IP
- блок регистров общего назначения
 - схема регистров общего назначения и управления ими
- блок общих операций
 - блоки арбитража к общим шинам (управления, адресной и данных)
 - оптимизированный блок 0-ой итерации
 - блок операции NOP
 - блок операции JMP
 - блок операции M->R
 - блок операции R->M
- блок арифметико-логического устройства
 - блоки арбитража к общим шинам (управления, адресной и данных)
 - блок регистров арифметико-логического устройства
 - блок операции ROR
 - блок операции ADDC
 - блок операции INCS
 - блок операции NOT
 - блок операции AND
- блок стека
 - блоки арбитража к общим шинам (управления, адресной и данных)
 - блок схемы стека
- блок КЭШа
 - блок управления КЭШа
 - основной блок данных КЭШа
 - блок контроля флагов КЭШа

- блок фиксации выходных сигналов основного блока данных КЭШ на момент исполнения операции КЭШ
- блок контроллера прямого доступа к памяти
 - схема контроля прямого доступа к памяти

Так или иначе, по варианту заданы параметры для реализации арбитража и предсказателя переходов. В данной микро-ЭВМ они не применяются, но (т.к. заданы) реализованы отдельными независимыми блоками в отдельных проектах:

- организация арбитража шин
- предсказатель переходов

Микро-ЭВМ помимо регистров общего назначения (20шт.) располагает специальными регистрами:

- IP (instruction pointer) – регистр, обозначающий смещение следующей команды относительно кодового сегмента, его разрядность совпадает с разрядностью адресной шины и равна 12 бит
- IR (instruction register) – регистр, хранящий исполняемую в данный момент команду, его разрядность определена общим количеством операций микро-ЭВМ, для нашей микро-ЭВМ разрядность составляет 4 бита
- AR (address register) – адресный регистр, используется в операциях, имеющих итерацию адресации, его разрядность совпадает с разрядностью адресной шины и равна 12 бит
- DR (data register) – регистр данных, используется как регистр накопитель для промежуточных данных, его разрядность совпадает с разрядностью шины данных и равна 8 бит
- FR (flag register) – регистр флагов, содержит текущее состояние процессора, используется во многих операциях как арифметических и логических, так и в управляющих и прочих (в данной микро-ЭВМ только в логических операциях АЛУ), его разрядность в данной микро-ЭВМ взята 8 бит просто для совпадения с разрядностью шины данных, на деле количество используемых бит значительно меньше (т.к. и количество операций микро-ЭВМ невелико)

Так или иначе, специальные регистры используются в различных операциях для хранения промежуточных результатов, достоверность содержимого специальных регистров не гарантируется, специальные регистры не могут быть адресованы, их использование задано жестко схемой каждой операции. Для хранения данных используйте память (RAM), или регистры общего назначения, достоверность их содержимого гарантируется, содержимое определяется только пользователем.

1.2 Разработка системы команд

Количество команд по заданию – 12:

- NOP (нет операции)
- JMP (безусловный переход)
- M->R (чтение из памяти в регистр общего назначения)
- ROR (циклический сдвиг вправо)
- ADDC
- AND
- PUSH
- POP
- INCS
- NOT
- R->M
- HALT

Для кодирования кода операции необходимо как минимум 4 бита, т.к. количество команд – 12 (диапазон значений КОП будет от 0000 до 1011).

При таком раскладе систему команд можно будет дополнить ещё 4-мя новыми операциями (диапазон значений от 1100 до 1111).

Количество регистров общего назначения – 20шт. Для их адресации понадобится, как минимум 5 бит (диапазон значений от 00000 до 10011).

Разрядность шины данных составляет 12 бит, соответственно для адресации участка памяти при нахождении его непосредственно в коде (а не в регистре общего назначения, к примеру) в команде будет указан полный адрес, занимающий 12 бит (адресация регистра требует меньше – всего 5 бит).

Т.к. разрядность шины данных равна 8 бит и имеет место пакетная передача данных из блока контроля КЭШа, то будет иметь место некоторая избыточность данных при чтении команд из памяти ROM. К примеру: минимальное количество бит для той же простейшей команды NOP равно 4 бита (только КОП и ничего другого), но т.к. память позволяет минимально считать только 8 бит, то последующие вычитанные 4 бита просто не используются и никак не влияют на что-либо. Можно было бы кодировать из как КОП следующей команды, но это не даёт никаких преимуществ, даже более, это усложнит схему декодирования в блоке устройства управления.

Та же ситуация со всеми остальными командами, где в таблице присутствует символ 'X', трактующийся как "любое значение (не влияет на команду никоим образом, является избыточным и имеет место лишь по причине задания конкретных параметров варианта для специализированного процессора курсового проекта)".

Структура системы команд приведена в приложении А.

Положение КОП фиксировано – всегда первые 4 бита команды ([0..3]).

Иные расшифровки:

- OFFSET[11..0] – адрес, куда будет происходить безусловный переход
- OPERAND_ADDR[11..0] – адрес памяти (RAM), откуда будет вычитываться операнд
- REG_NUM[4..0] – номер регистра общего назначения (POH)
- OPERAND[7..0] – непосредственно операнд, находящийся в памяти (ROM)
- ADDR[11..0] – адрес для записи в память (RAM), куда будет записан операнд
- X – любое значение (не влияет на команду никоим образом, является избыточным и имеет место лишь по причине задания конкретных параметров варианта для специализированного процессора курсового проекта)
- RESERVED – зарезервировано (распознаётся устройством управления как NOP и может использоваться для расширения набора команд)

Любой иной КОП, не указанный выше будет определён устройством управления как «зарезервированный КОП» и управление будет передано блоку NOP (нет операции).

1.3 Описание взаимодействия всех блоков микро-ЭВМ при выполнении команд программы

Главным блоком в иерархии блоков является блок устройства управления (УУ). Именно УУ осуществляет декодирование команды, контроль над сохранением состояния микро-ЭВМ при ожидании окончания взаимодействия блока КЭШа и блока памяти. УУ решает, какому именно блоку будет передано управление после декодирования команды, обеспечивает отсутствие коллизий, когда несколько блоков захватывают одну шину и выставляют туда свои данные - фактически арбитраж, но в слишком грубом приближении, т.к. блоки контроля содержатся и в каждом исполнительном блоке, которому может быть передано управление от блока УУ.

Мы можем задать непосредственно лишь тактовый сигнал CLOCK для нашей микро-ЭВМ, весь код, который будет выполнен микро-ЭВМ, содержится в ROM и полностью определяет поведение микро-ЭВМ.

После передачи управления УУ соответствующему исполнительному блоку, этот блок захватывает все шины (шину управления, адресную шину и шину данных) и полностью контролирует их до момента окончания всех итераций операции. Такими исполнительными блоками являются:

- блок общих операций
 - оптимизированный блок 0-ой итерации
 - блок операции NOP
 - блок операции JMP
 - блок операции M->R
 - блок операции R->M
- блок арифметико-логического устройства
 - блок операции ROR
 - блок операции ADDC
 - блок операции INCS
 - блок операции NOT
 - блок операции AND
- блок стека
 - блок схемы стека

Но, тем не менее, все исполнительные блоки взаимодействуют со всей остальной частью микро-ЭВМ (памятью и регистрами) только через устройство управления, которое ограничивает контроль исполнительных блоков над остальной системой.

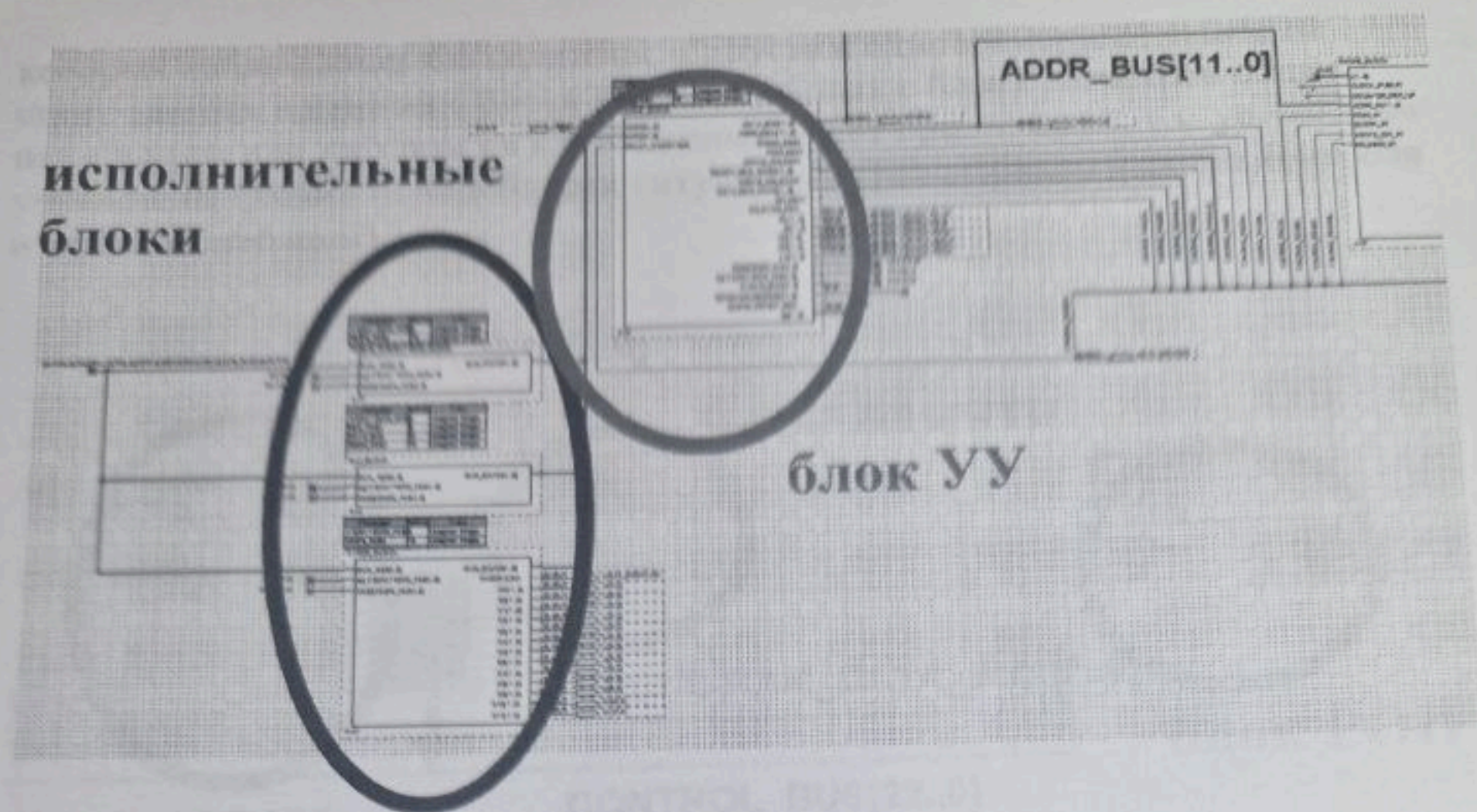


Рисунок 1.1 - Символическое представление исполнительных блоков и блока УУ

Взаимодействие исполнительных блоков и блока прямого доступа к памяти с блоком памяти (ROM-RAM) осуществляется только через блок КЭШа, который состоит из блока управления КЭШа, блока данных КЭШа, блока флагов КЭШа и общей схемы управления памятью. Блок КЭШа ограничивает взаимодействие с памятью, позволяя задать только атомарную команду (чтение-запись) и не беспокоясь, как она реализуется внутри блока КЭШа. Т.к. в микро-ЭВМ реализован прямой доступ к памяти, то одновременно к блоку памяти (КЭШа) могут обратиться и некий исполнительный блок через УУ и само периферийное устройство через контроллер прямого доступа к памяти, очевидно, если одновременно подавать сигналы на общие шины с обоих блоков, возникнет коллизия. Поэтому заранее с руководителем было оговорено, что любой исполнительный блок в паре с УУ имеет меньший фиксированный приоритет в доступе к памяти (КЭШу), а периферийное устройство получает доступ через контроллер прямого доступа к памяти тогда, когда хочет (хотя правильнее распределять приоритеты обратным образом). Приоритеты сделаны такими ввиду постоянной занятости общих шин – данных, адресной и управления соответствующим исполнительным блоком и блоком УУ, т.к. изначально архитектура задана принстонская и всегда идёт запрос либо на ROM либо на RAM, посему общие шины просто не могут быть свободны.

Блок памяти (КЭШа) и блок регистров общего назначения разделяют между собой общую шину данных, арбитражом которых занимается исполнительный блок, захвативший шину в текущий момент. Одновременно, невозможен случай, когда исполнительный блок задаёт сигналы, при

которых наблюдались бы коллизии. Допустим одновременный вывод на шину данных и данных с блока КЭШа и данных с блока регистров общего назначения, при котором на шине данных будет просто каша, т.к. устройство управления следит за подобными ситуациями (реализовано в схеме контроля итераций операций).

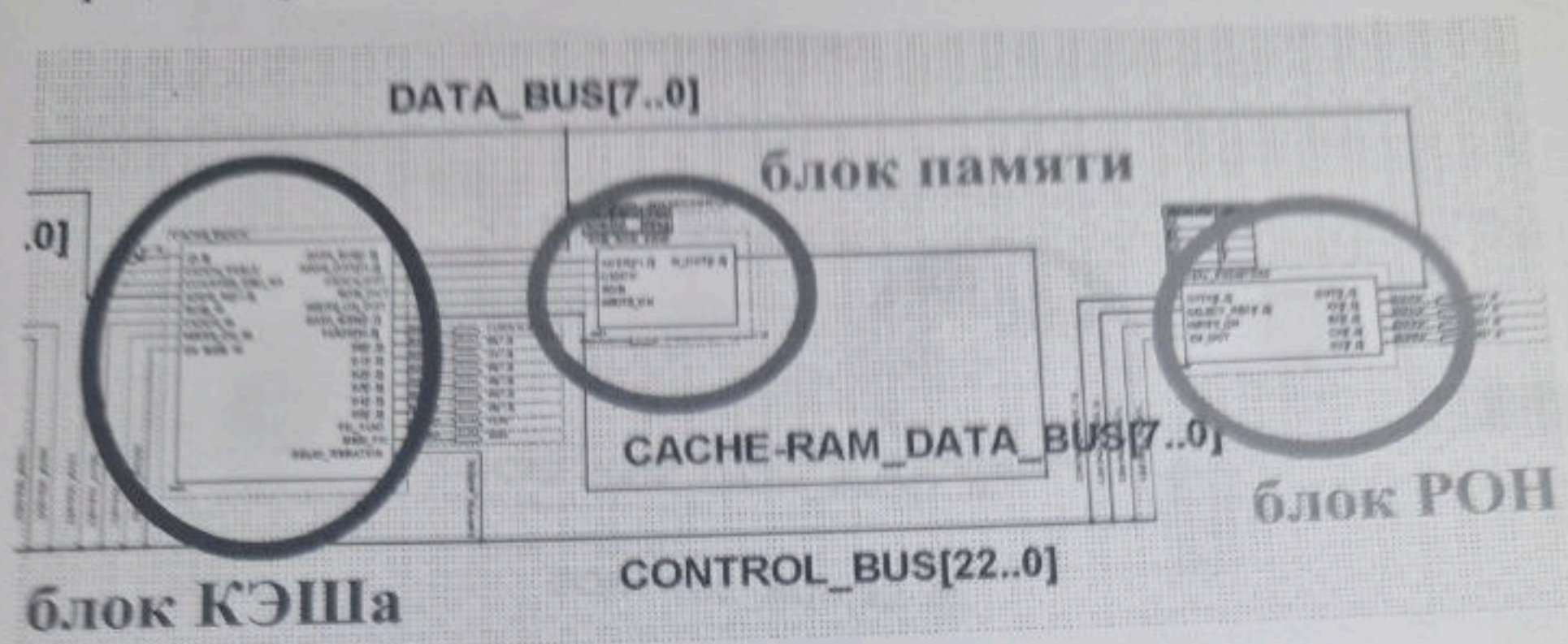


Рисунок 1.2 - Символическое представление блоков КЭШа, памяти и РОН

Взаимодействие исполнительных блоков с блоком регистров общего назначения осуществляется так же как и в описанном случае с памятью — через блок устройства управления, который ограничивает адресацию номера регистра общего назначения от 0 до 19 (т.к. общее количество регистров общего назначения — 20шт.).

2 РАЗРАБОТКА ОСНОВНЫХ УСТРОЙСТВ МИКРО-ЭВМ

2.1 Запоминающее устройство

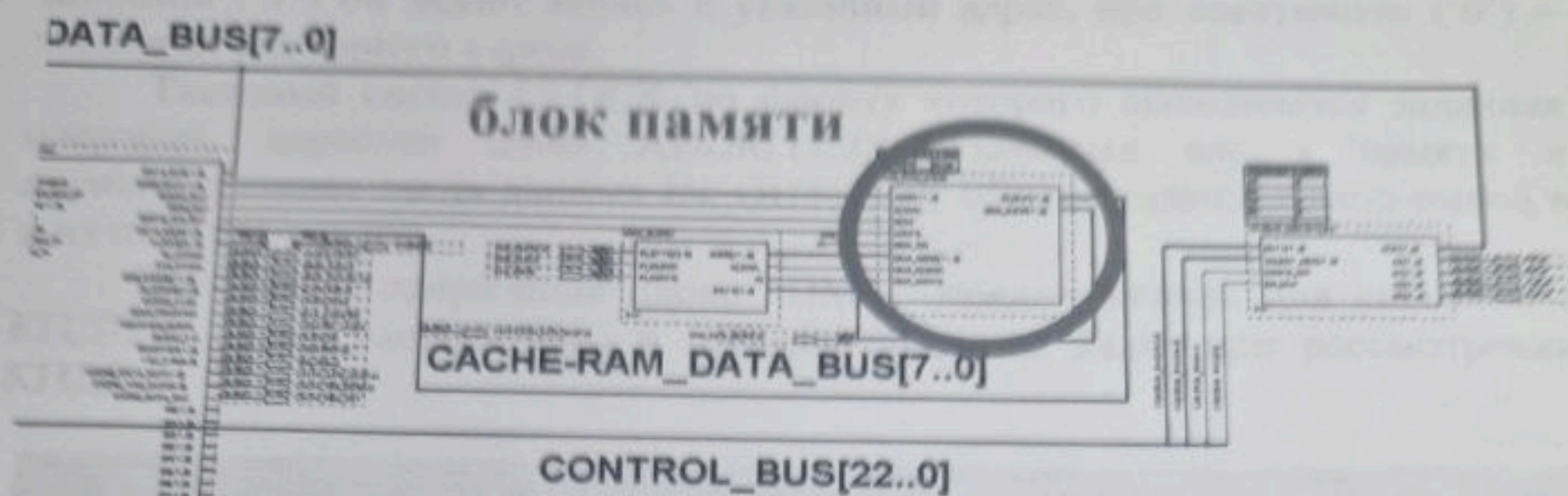


Рисунок 2.1 - Расположение блока памяти в корне проекта

Т.к. заданная архитектура по варианту – принстонская, то микро-ЭВМ имеет общую шину данных, адресную шину и шину управления для одного блока памяти (КЭШа). Блок памяти содержит в себе:

- блок памяти ROM
- блок памяти RAM
- схема управления блоками памяти

Схема блока памяти представлена в приложении Б.

На схеме видны 2 основных блока: LPM_ROM и LPM_RAM_IO. RAM изначально с двунаправленной шиной данных, «двунаправленная» шина данных ROM не имеет смысла, т.к. из ROM идет только чтение.

Файлы RAM.hex и ROM.hex задают начальные значения содержимого памяти RAM и ROM. Все важные параметры блоков отражены в таблице параметров на символическом изображении ниже.

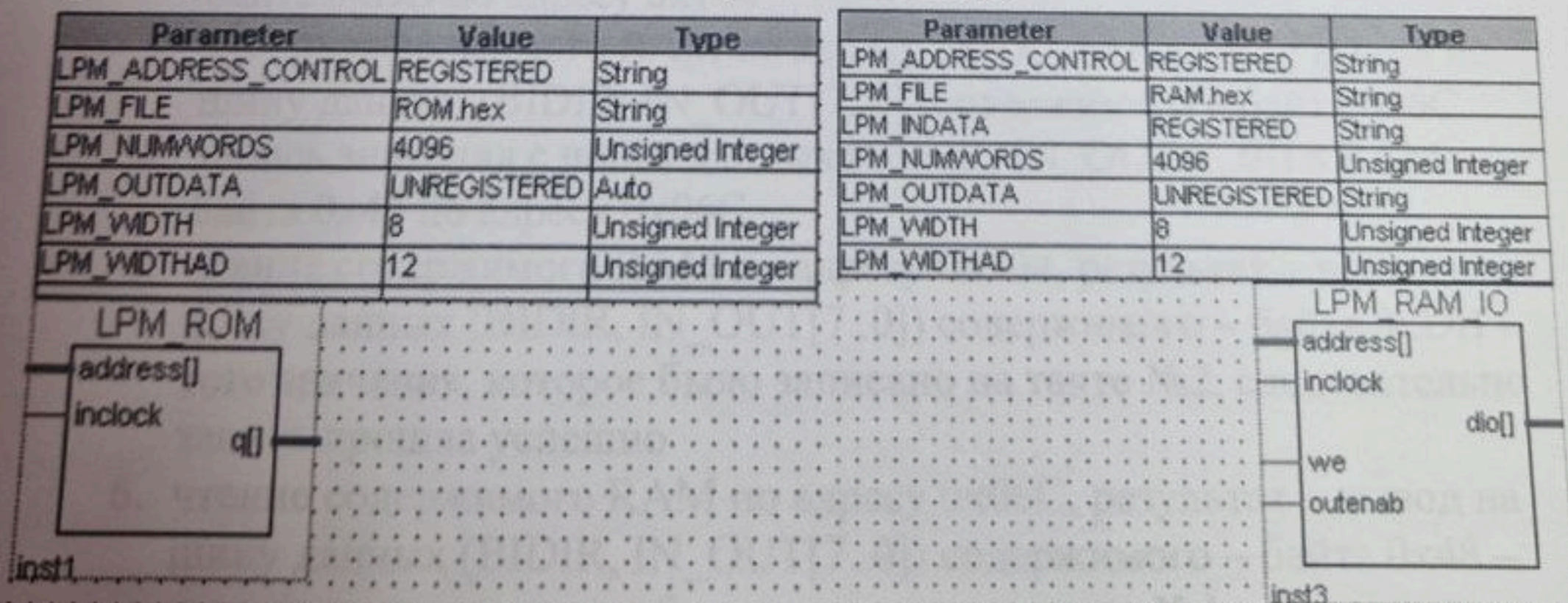


Рисунок 2.2 - Символическое представление блоков ROM и RAM

Одновременно может быть доступен только один блок (или RAM или ROM). Какой именно блок сейчас используется, задаётся управляющим входным сигналом ROM, при активном значении ('1') активен блок ROM, при неактивном значении ('0') – блок RAM.

За запись отвечает управляющий сигнал WRITE_ON, при активном значении ('1') он задаёт запись в указанный адрес, при неактивном ('0') – чтение из указанного адреса.

Тактовый сигнал CLOCK по фронту которого выполняется заданная операция, адресная шина ADDR[11..0], задающая адреса памяти и двунаправленная шина данных IN_OUT[7..0], обеспечивающая ввод-вывод в и из памяти.

Все пины содержащие слово "DMA" предназначены для сигналов с КЦДП и будут рассмотрены и промоделированы далее при рассмотрении КЦДП.

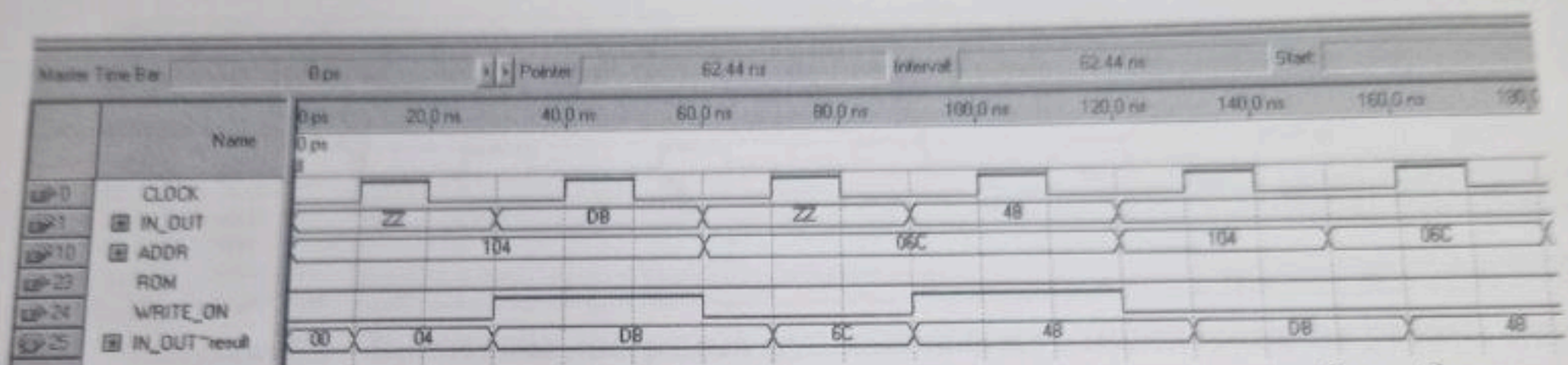


Рисунок 2.3 - Временная диаграмма функционального моделирования работы блока памяти RAM

Номер пункта соответствует номеру такта:

1. чтение содержимого RAM по адресу 0x104, результат – вывод на шину данных (BIDIR, IN_OUT[7..0]) содержимого – байта 0x04
2. запись значения с шины данных (BIDIR, IN_OUT[7..0]) в RAM – байта 0xDB по адресу 0x104
3. чтение содержимого RAM по адресу 0x06C, результат – вывод на шину данных (BIDIR, IN_OUT[7..0]) содержимого – байта 0x6C
4. запись значения с шины данных (BIDIR, IN_OUT[7..0]) в RAM – байта 0x48 по адресу 0x06C
5. чтение содержимого RAM по адресу 0x104, результат – вывод на шину данных (BIDIR, IN_OUT[7..0]) содержимого – байта 0xDB – того значения, которое было записано на такте №2, следовательно запись прошла успешно
6. чтение содержимого RAM по адресу 0x06C, результат – вывод на шину данных (BIDIR, IN_OUT[7..0]) содержимого – байта 0x48 – того значения, которое было записано на такте №4, следовательно запись прошла успешно

MAIN_MODULE.bdf

RAM_ROM_BIDIR.bdf

Compilation Report - Flo

Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	07	01	02	03	04	15	06	07	08	09	0A	0B	0C	0D	0E	0F
010	10	11	AA	13	14	15	16	B7	18	19	1A	09	1C	1D	1E	FF
020	07	5D	22	23	24	25	26	27	28	29	2A	30	2C	2D	2E	5F
030	30	31	32	6A	34	35	36	37	38	64	3A	3B	3C	3D	3E	3F
040	AA	3A	42	43	44	45	46	47	48	49	6A	4B	4C	4D	4E	4F
050	50	51	52	53	54	55	56	57	58	59	6A	5B	5C	5D	5E	5F
060	60	61	62	63	64	65	66	67	68	69	7C	6B	6C	6D	6E	6F
070	70	72	73	74	75	76	7A	7B	7C	7D	7E	7F	7C	7D	7E	7F
080	80	81	82	83	84	85	86	87	88	8A	8B	8C	8D	8E	8F	
090	90	91	92	93	94	95	96	97	98	9A	9B	9C	9D	9E	9F	
0A0	EA	A1	8F	A3	A4	A5	A6	A7	A8	A9	ED	E1	AC	AD	AE	AF
0B0	80	8A	82	8F	84	85	88	87	88	89	8A	8B	8C	8D	8E	8F
0C0	00	C1	C2	C3	C4	C5	C6	C7	C8	CA	CB	CC	CD	CE	CF	
0D0	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0E0	00	E1	EC	EA	E4	E5	E8	E7	E8	EA	EB	EC	ED	EE	EF	
0F0	AS	F1	F2	F3	F4	F5	F6	F7	F8	FB	FD	FA	FC	FE	FF	
100	67	01	02	03	04	15	06	07	08	09	0A	0B	0C	0D	0E	0F
110	10	11	AA	13	14	15	16	B7	18	19	1A	09	1C	1D	1E	FF
120	07	5D	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	5F
130	30	31	32	6A	34	35	36	37	38	64	3A	3B	3C	3D	3E	3F
140	AA	3A	42	43	44	45	46	47	48	49	6A	4B	4C	4D	4E	4F

IRAM_ROM_BIDIR[lpn_ram_inst3]alt:ram:scram[altsyncram_block1]a

Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	07	01	02	03	04	17	06	07	08	09	0A	0B	0C	0D	0E	0F
010	10	11	AA	13	14	15	16	B7	18	19	1A	09	1C	1D	1E	FF
020	07	5D	22	23	24	25	26	27	28	29	2A	30	2C	2D	2E	5F
030	30	31	32	6A	34	35	36	37	38	64	3A	3B	3C	3D	3E	3F
040	AA	3A	42	43	44	45	46	47	48	49	6A	4B	4C	4D	4E	4F
050	50	51	52	53	54	55	56	57	58	59	6A	5B	5C	5D	5E	5F
060	60	61	62	63	64	65	66	67	68	69	7C	6B	6C	6D	6E	6F
070	70	72	73	74	75	76	7A	7B	7C	7D	7E	7F	7C	7D	7E	7F
080	80	81	82	83	84	85	86	87	88	89	7A	7B	7C	7D	7E	7F
090	90	91	92	93	94	95	96	97	98	99	8A	8B	8C	8D	8E	8F
0A0	EA	E1	8F	A3	A4	A5	A6	A7	A8	ED	E1	AC	AD	AE	AF	
0B0	80	8A	82	8F	84	85	88	87	88	89	8A	8B	8C	8D	8E	8F
0C0	00	C1	C2	C3	C4	C5	C6	C7	C8	CA	CB	CC	CD	CE	CF	
0D0	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0E0	00	E1	EC	EA	E4	E5	E8	E7	E8	EA	EB	EC	ED	EE	EF	
0F0	AS	F1	F2	F3	F4	F5	F6	F7	F8	FB	FD	FA	FC	FE	FF	
100	67	01	02	03	04	17	06	07	08	09	0A	0B	0C	0D	0E	0F
110	10	11	AA	13	14	15	16	B7	18	19	1A	09	1C	1D	1E	FF
120	07	5D	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	5F
130	30	31	32	6A	34	35	36	37	38	64	3A	3B	3C	3D	3E	3F
140	AA	3A	42	43	44	45	46	47	48	49	6A	4B	4C	4D	4E	4F

Рисунок 2.4 - Дампы памяти RAM до (слева) и после (справа) функционального моделирования

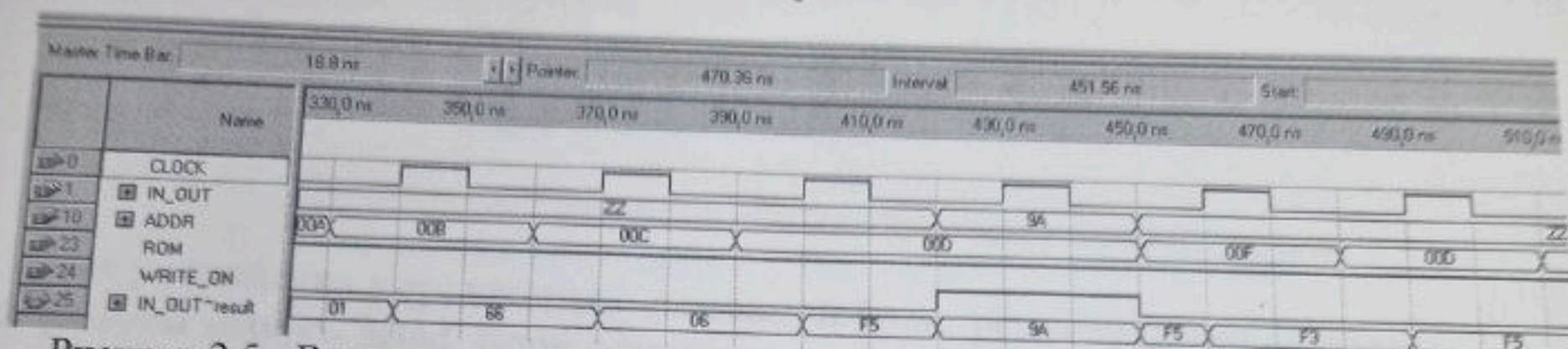


Рисунок 2.5 - Временная диаграмма функционального моделирования работы блока памяти ROM

[illegible]

Рисунок 2.6 - Дампы памяти ROM до (слева) и после (справа) функционального моделирования

Для ROM комментарии излишни: каждый очередной такт просто вычитывает содержимое памяти, заданного адреса. При этом если задать операцию записи с помощью сигнала WRITE_ON в ROM, то запись производится не будет – за этим следит схема управления блоками памяти (см. такты №3, №4 и №6 – содержимое ROM по адресу 0x00D не изменилось).

2.2 Устройство управления

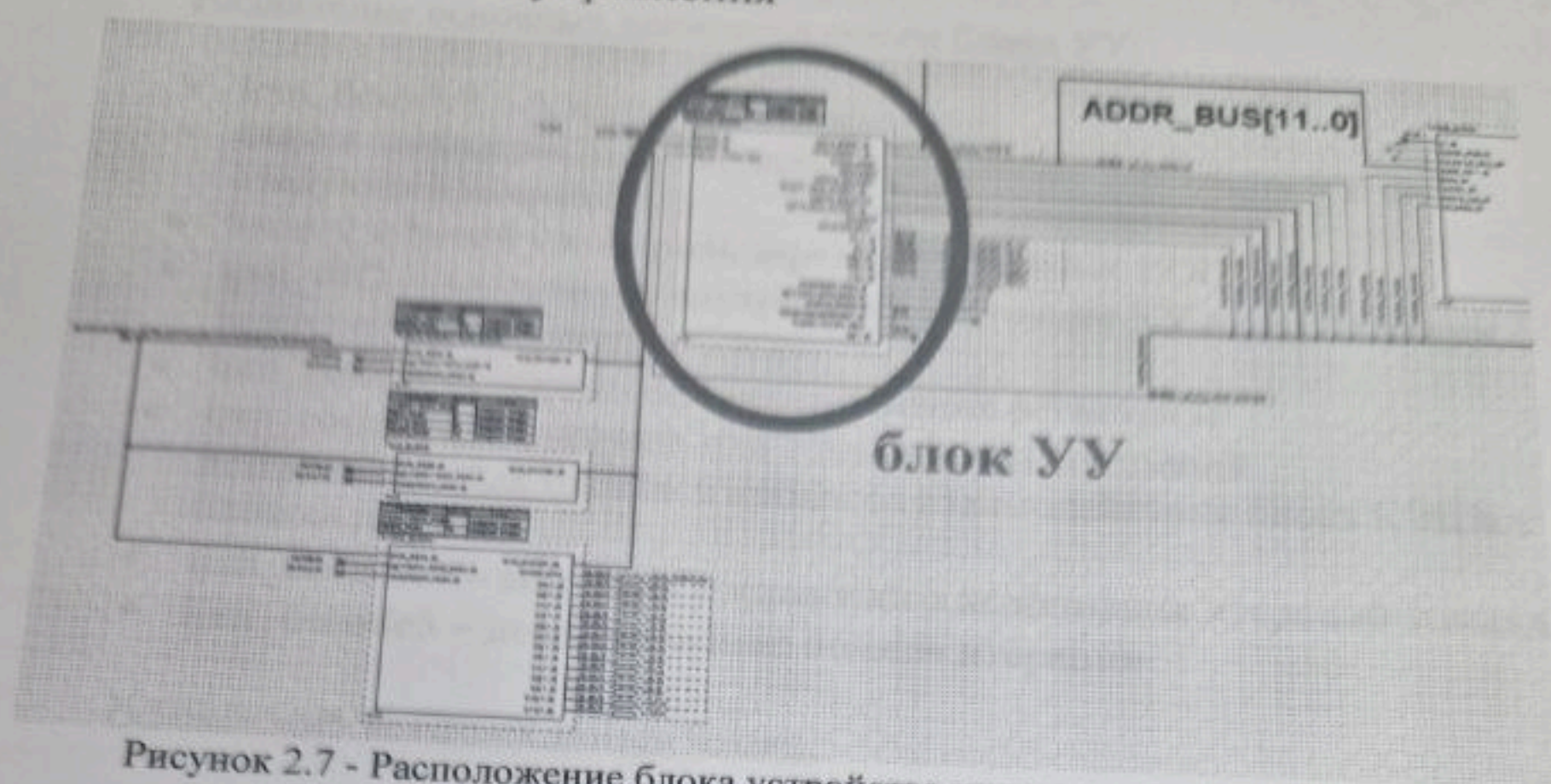


Рисунок 2.7 - Расположение блока устройства управления в корне проекта

Блок устройства управления – основной блок, реализующий общую логику работы микро-ЭВМ. Блок осуществляет:

- декодирование команды
- контроль итераций команд
- контроль состояния специальных регистров
- контроль состояния шины управления
- обеспечение доступа к регистрам специального назначения (т.к. блок регистров специального назначения встроен в устройство управления)
- контроль над сохранением состояний исполнительных блоков в процессе взаимодействия блока КЭШа с блоком памяти
- «посредничество» между исполнительными блоками и блоками КЭШа, регистров общего назначения

Схема блока устройства управления представлена в приложении В.

- WRITE_ON_GLOB – выбор режима обмена с блоком POH (запись/чтение)
- DATA_REG_GLOB[7..0] – шина данных блока POH
- EN_MEM – контроль активности выходной шины данных памяти
- EN_GLOB_REG – контроль активности выходной шины данных блока POH
- CMD[CMD_NUM..0] – шина «декодированных команд»

Назначение основных элементов схемы блока УУ:

- lpm_decode8 – декодирование операции
- связки элементов or2, not, and2 – сохранение состояния УУ на следующей итерации
- band12 и band4 – контроль зарезервированных КОП
- lpm_dff2 – сохранение внутреннего состояния УУ на протяжении выполнения операции
- lpm_compare1 – контроль переполнения регистра IP
- lpm_counter6 – контроль над сохранением состояний исполнительных блоков в процессе взаимодействия блока КЭШа с блоком памяти
- lpm_counter4 – контроль корректного исполнения итераций команд
- lpm_decode5 – декодирование номера итерации

Назначение входных пинов блока:

- CLOCK_IN – тактовый сигнал
- BUS[54..0] – входная шина управления, идущая и полностью контролируемая исполнительными блоками
- DELAY_ITERATION – сигнал задержки итерации команды (используется при “отставании” памяти от УУ и исполнительных блоков и для прерывания выполнения текущей итерации ради работы DMA)

Назначение выходных пинов блока:

- DATA_BUS[7..0] – общая шина данных
- ADDR[11..0] – общая адресная шина
- CLOCK_MEM – тактовый сигнал памяти
- ROM_MEM – выбор памяти (ROM/RAM)
- WRITE_ON_MEM – выбор режима обмена с памятью (запись/чтение)
- SEL_REG_GLOB[7..0] – адресная шина блока POH
- WRITE_ON_GLOB – выбор режима обмена с блоком POH (запись/чтение)
- DATA_REG_GLOB[7..0] – шина данных блока POH
- EN_MEM – контроль активности выходной шины данных памяти
- EN_GLOB_REG – контроль активности выходной шины данных блока POH
- CMD[CMD_NUM..0] – шина «декодированных команд»

- eq[ITERATONS_NUM..0] – шина итераций
- PUBLIC_BUS[2..0] – шина общего назначения исполнительных блоков
- COUNTER_DECODE[7..0] – отладочная шина, контроль номера итерации команды
- CLOCK_COUNTER_DEC – тактовый сигнал блока декодирования с учётом обработки УУ
- qq[1..0] – шина контроля над задержкой итерации команды
- IP[7..0], IR[3..0], AR[7..0], DR[7..0], FR[7..0] – отладочные пины для контроля содержимого специальных регистров

Блок УУ реализует операцию HALT – останов микро-ЭВМ путём блокирования тактового сигнала – реализовано элементами: lpm_decode8 и NOR2 inst16. Эта операция не была вынесена в отдельный блок, поскольку этот блок включал бы в себя лишь пару элементов, а данную операцию можно реализовать с использованием некоторых простаивающих элементов УУ при выполнении операции HALT, что экономит количество элементов схемы и упрощает её.

Блок УУ включает в себя блок регистров специального назначения.

Parameter	Value
IP	0
IR	1
AR	2
DR	3
FR	4

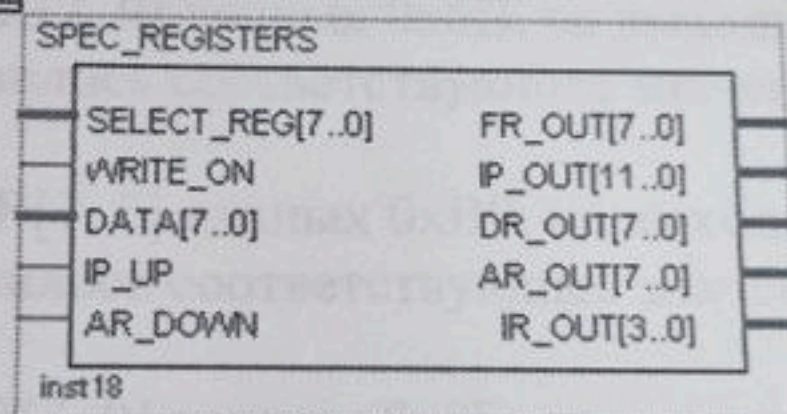


Рисунок 2.8 - Символическое представление блока регистров специального назначения

Схема блока регистров специального назначения представлена в приложении Г.

Схема тривиальна и работает по аналогичному принципу со схемой блока регистров общего назначения, детальных пояснений не требует. Детальное рассмотрение схемы блока регистров общего назначения приведено в п. 2.3.

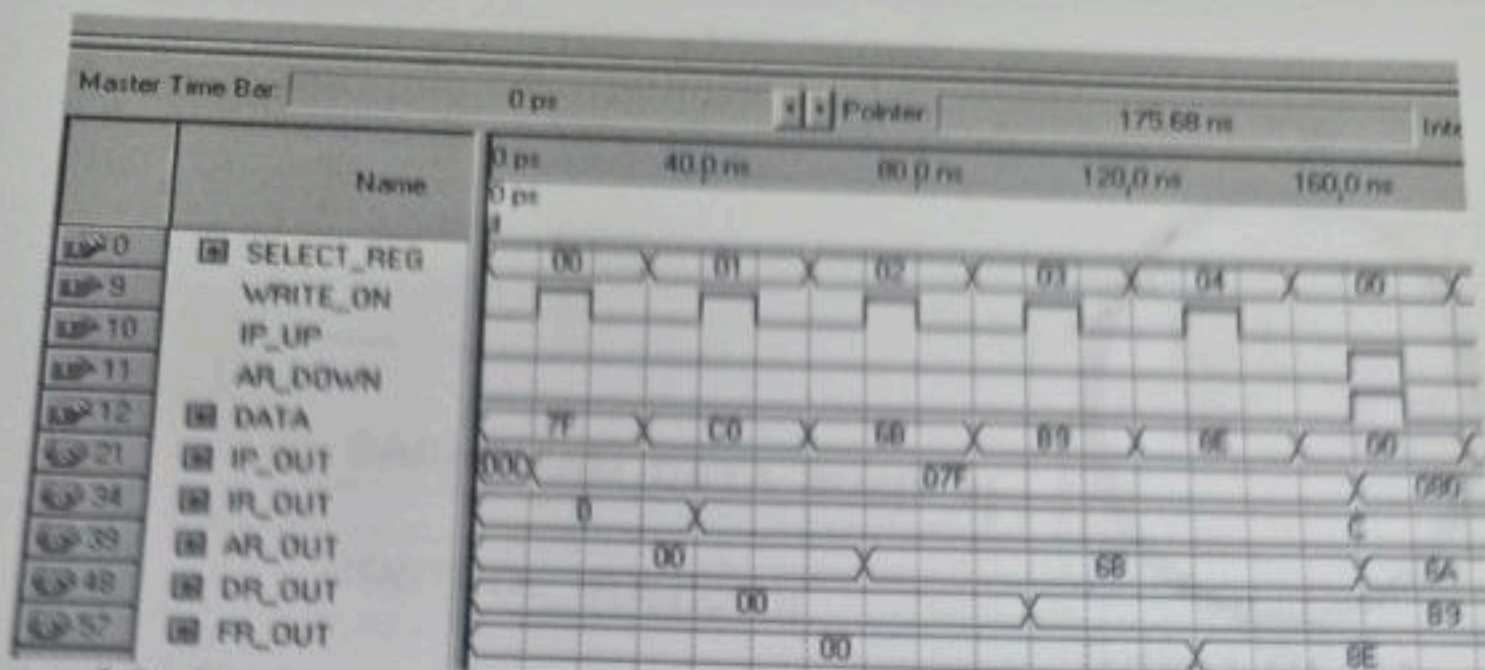


Рисунок 2.9 - Временная диаграмма функционального моделирования работы блока специальных регистров

Номер пункта соответствует номеру такта:

1. запись в регистр IP[11..0] данных 0x07F, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно
2. запись в регистр IR[3..0] данных 0xC, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно
3. запись в регистр AR[7..0] данных 0x6B, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно
4. запись в регистр DR[7..0] данных 0xB9, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно
5. запись в регистр FR[7..0] данных 0x8E, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно
6. инкрементирование содержимого регистра IP[11..0] и декрементирование содержимого регистра AR[7..0], на выходных отладочных пинах регистров появились соответствующие значения, значит, операции прошли успешно.

2.3 Регистры общего назначения

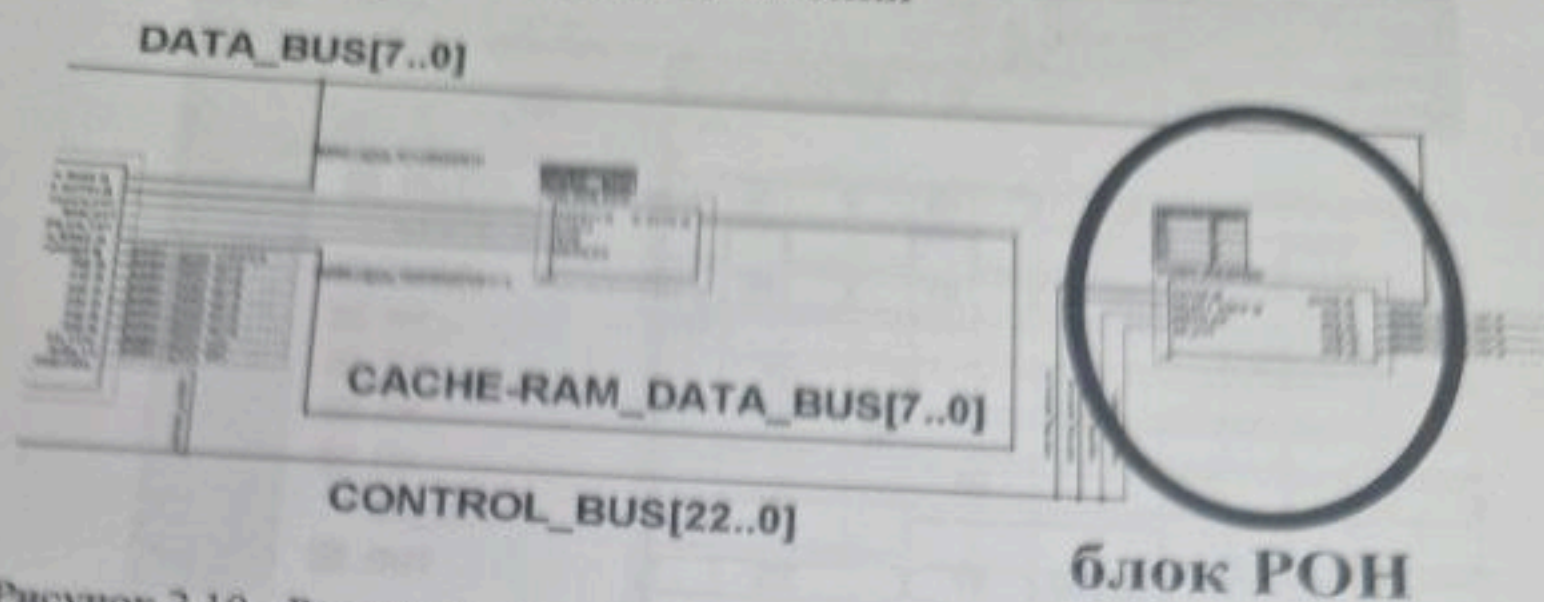


Рисунок 2.10 - Расположение блока регистров общего назначения в корне проекта

Блок POH содержит в себе регистры общего назначения в количестве 20 шт. и логику управления ими. Схема блока регистров общего назначения представлена в приложении Д.

Назначение основных элементов схемы блока POH:

- `lpm_decode4` – декодирование адреса регистра
- связки элементов `tri` – мультиплексирование данных с регистров
- `lpm_dff0` – непосредственно сами регистры общего назначения

Назначение входных пинов блока:

- `DATA[7..0]` – шина данных
- `SEL_REG[7..0]` – адресная шина выбора регистра
- `WRITE_ON` – выбор режима обмена с выбранным регистром
- `EN_OUT` – контроль активности выходной шины данных блока POH

Назначение выходных пинов блока:

- `OUT[7..0]` – общая шина данных
- `AX[7..0]`, `BX[7..0]`, `CX[7..0]`, `DX[7..0]` – отладочные шины, контроль содержимого конкретных регистров

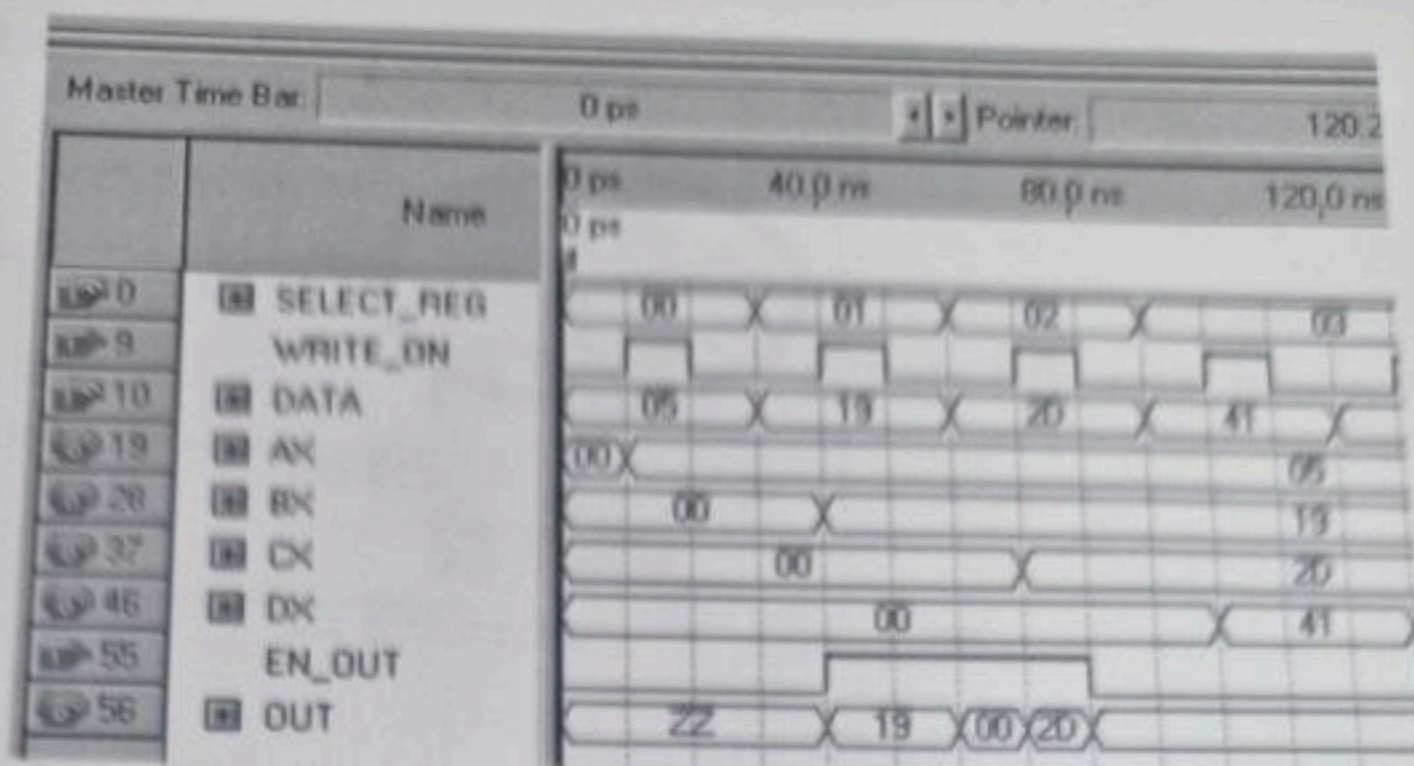


Рисунок 2.11 - Временная диаграмма функционального моделирования работы блока регистров общего назначения

Номер пункта соответствует номеру такта:

1. запись в регистр AX[7..0] данных 0x05, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно
2. запись в регистр BX[7..0] данных 0x19, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно; на пин EN_OUT подан активный сигнал ("1"), значит, на общую шину данных проходят данные с выбранного (адресуется шиной SELECT_REG[7..0]) регистра
3. запись в регистр CX[7..0] данных 0x2D, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно; на пин EN_OUT подан активный сигнал ("1"), значит, на общую шину данных проходят данные с выбранного (адресуется шиной SELECT_REG[7..0]) регистра – первоначальное содержимое регистра CX[7..0] - 0x00 (выводится на общую шину данных), после записи это значение изменилось и стало – 0x2D
4. запись в регистр DX[7..0] данных 0x41, на выходном отладочном пине регистра появилось соответствующее значение, значит, запись прошла успешно.

2.4 Блок общих операций

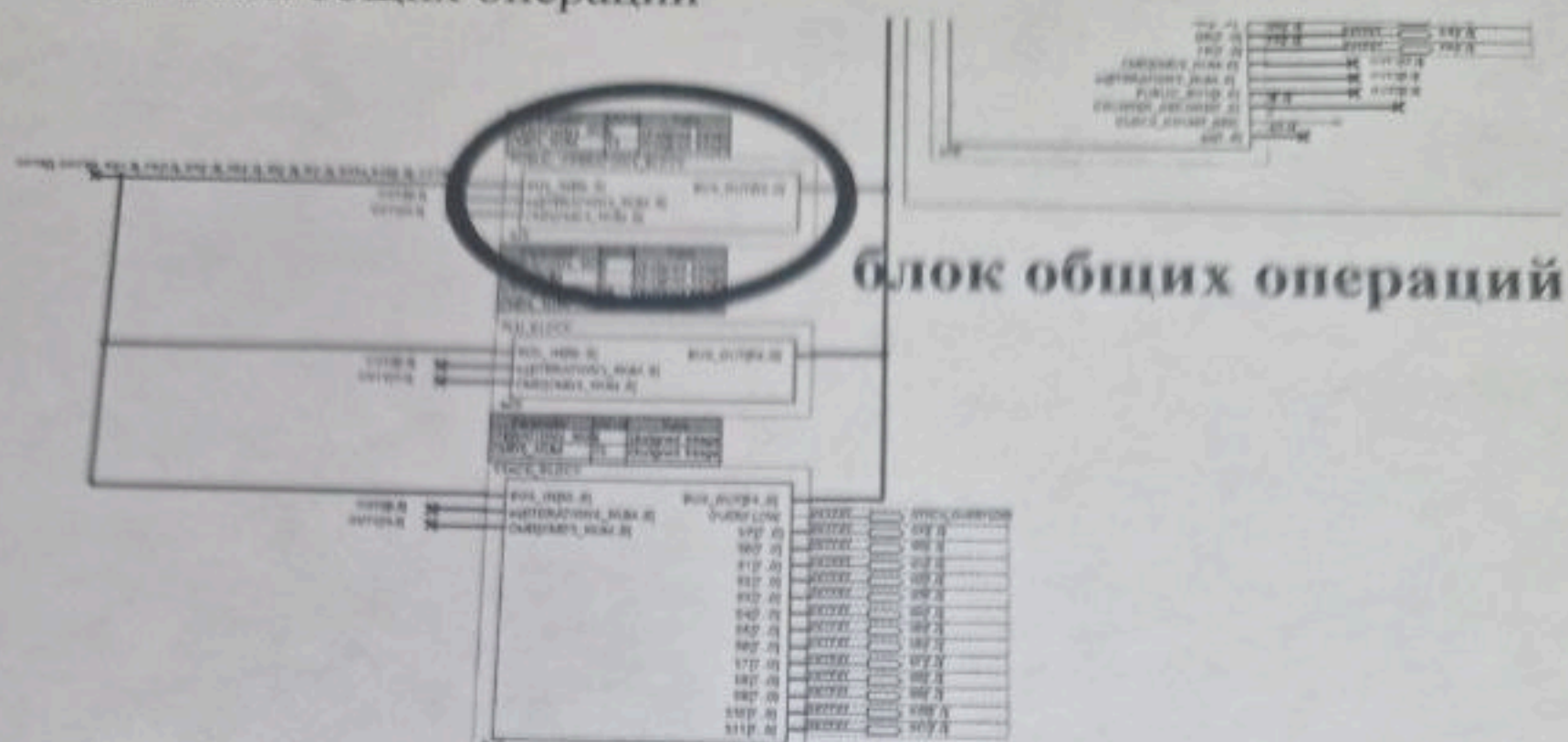


Рисунок 2.12 - Расположение блока общих операций в корне проекта

Схема блока общих операций представлена в приложении Е.

Состав блока и назначение основных элементов схемы:

- блоки CHECK_IN_BLOCK, блоки CHECK_OUT_BLOCK – обеспечение арбитража к общей шине BUS_OUT[54..0]
- ZERO_ITERATION_BLOCK – блок 0-ой итерации
- NOP_BLOCK – блок «НЕТ ОПЕРАЦИИ»
- JMP_BLOCK – блок операции «НЕПОСРЕДСТВЕННЫЙ ПЕРЕХОД»
- M-R_DIRECT_BLOCK – блок операции чтения из RAM в POH
- R-M_DIRECT – блок операции записи из POH в RAM

Далее опишем назначение пинов блока:

- BUS_IN[50..0] – входная общая шина данных (на неё прокидываются некоторые управляющие сигналы, а так же содержимое служебных регистров и общая шина данных)
- eq[ITERATIONS_NUM..0] – входная шина сигналов итераций
- CMD[CMDS_NUM..0] – входная шина декодированных команд
- BUS_OUT[54..0] – выходная общая совмещённая шина управления.

Все исполнительные блоки общих итераций имеют сходные входные и выходные пины.

Входные пины исполнительного блока общих итераций и их назначение:

- CLOCK_IN – тактовый сигнал
- DATA_IN[7..0] – общая шина данных
- IP_IN[11..0] – шина данных регистра IP (содержимое регистра)

- IR_IN[7..0] – шина данных регистра IR (содержимое регистра)
- AR_IN[7..0] – шина данных регистра AR (содержимое регистра)
- DR_IN[7..0] – шина данных регистра DR (содержимое регистра)
- FR_IN[7..0] – шина данных регистра FR (содержимое регистра)
- FALSE_TRUE_IN[1..0] – шина общих уровней логической единицы '1' и нуля '0' (сделано для синхронизации и исключения задержки с иными общими управляющими сигналами с блоком устройства управления)
- DECODE_IN[N..0] – декодированная шина итерации операции, разрядность зависит от того, сколько тактов занимает конкретная операция.

Выходные пины исполнительного блока общих итераций и их назначение:

- SEL_REG_SPEC_OUT[7..0] – адресная шина блока регистров специального назначения
- WRITE_ON_SPEC_OUT – выбор режима обмена с блоком регистров специального назначения (запись/чтение)
- IP_UP_SPEC_OUT – сигнал инкрементирования значения регистра IP
- AR_DOWN_SPEC_OUT – сигнал декрементирования значения регистра AR
- DATA_SPEC_OUT[7..0] – шина данных блока регистров специального назначения
- ADDR_MEM_OUT[11..0] – общая адресная шина
- WRITE_ON_MEM_OUT – выбор режима обмена с памятью
- CLOCK_MEM_OUT – тактовый сигнал памяти
- ROM_MEM_OUT – выбор памяти (ROM/RAM)
- (запись/чтение)
- SEL_REG_GLOB_OUT[7..0] – адресная шина блока POH
- WRITE_ON_GLOB_OUT – выбор режима обмена с блоком POH (запись/чтение)
- DATA_REG_GLOB_OUT[7..0] – шина данных блока POH
- ACLR_COUNTER_OUT – сигнал сброса счётчика итераций устройства управления
- STOP – сигнал останова микро-ЭВМ
- EN_OUT_MEM – контроль активности выходной шины данных памяти
- EN_OUT_REG – контроль активности выходной шины данных блока POH

Далее описание и назначение пинов исполнительных блоков общих итераций приводиться не будут.

2.4.1 Блок нулевой итерации

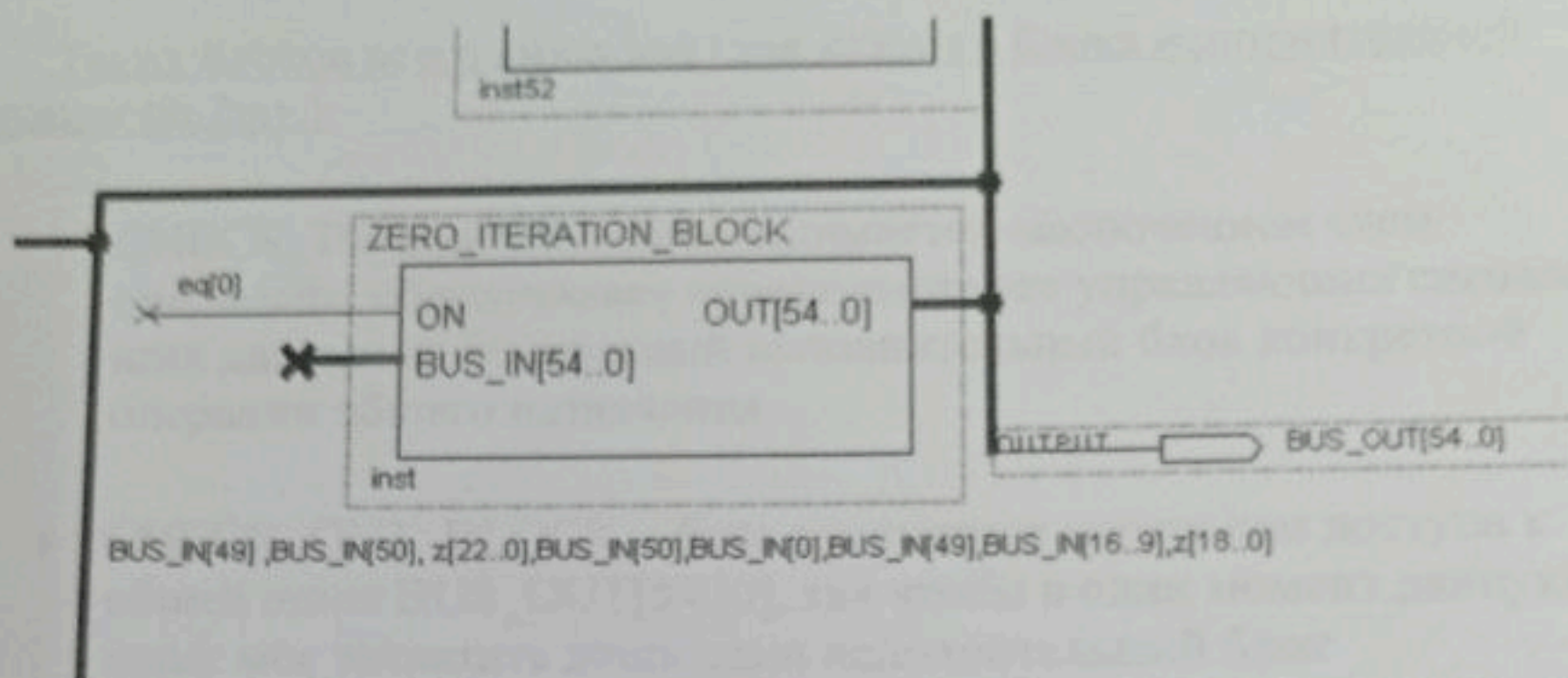


Рисунок 2.13 - Символическое представление блока нулевой итерации

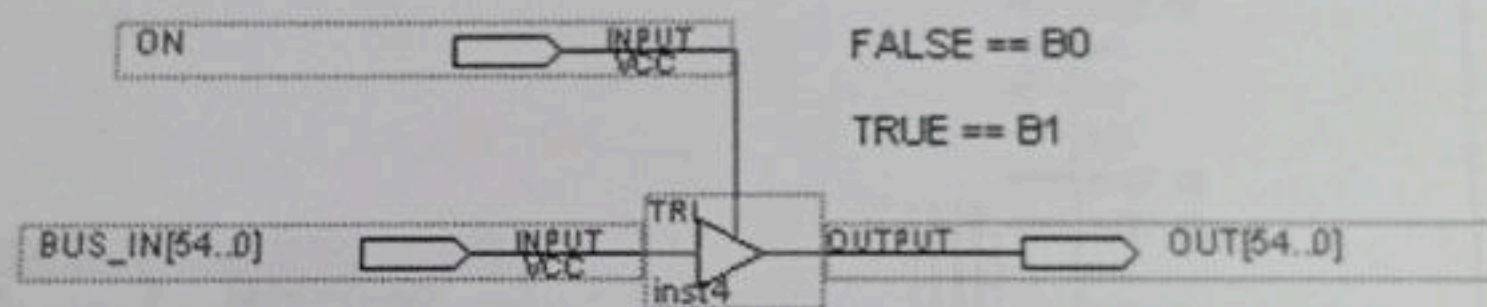


Рисунок 2.14 - Схема блока нулевой итерации

Комментарии по текущему блоку и его схеме излишни, всем управлением занимаются фиксировано заданные сигналы на входе блока, сигнал ON занимается контролем «пропуска» сигналов входной шины на выходную шину блока.

Значимые фиксированные сигналы блока нулевой итерации (активны только при нулевой итерации: $eq[0] = '1'$):

- ROM_MEM_OUT = '1'
- ADDR_MEM_OUT[11..0] = IP_IN[11..0]
- CLOCK_MEM_OUT = CLOCK_IN
- IP_UP_SPEC_OUT = '1'
- значения остальных сигналов и шин соответствуют неактивному состоянию (логические нули в данном случае)

2.4.2 Блок арбитража к общей шине

Данный блок имеет доступ лишь для двух каналов. Блок исполнительный (исполнительный блок).

- **CHECK_IN_BLOCK** – Блок занимается «вытеснением» с шины данных на конкретный исполнительный блок конкретной операции общего назначения.
- **CHECK_OUT_BLOCK** – Блок занимается контролем доступа к общей шине **BUS_OUT[32,0]**, так чтобы в один момент времени шину мог занимать лишь один исполнительный блок.

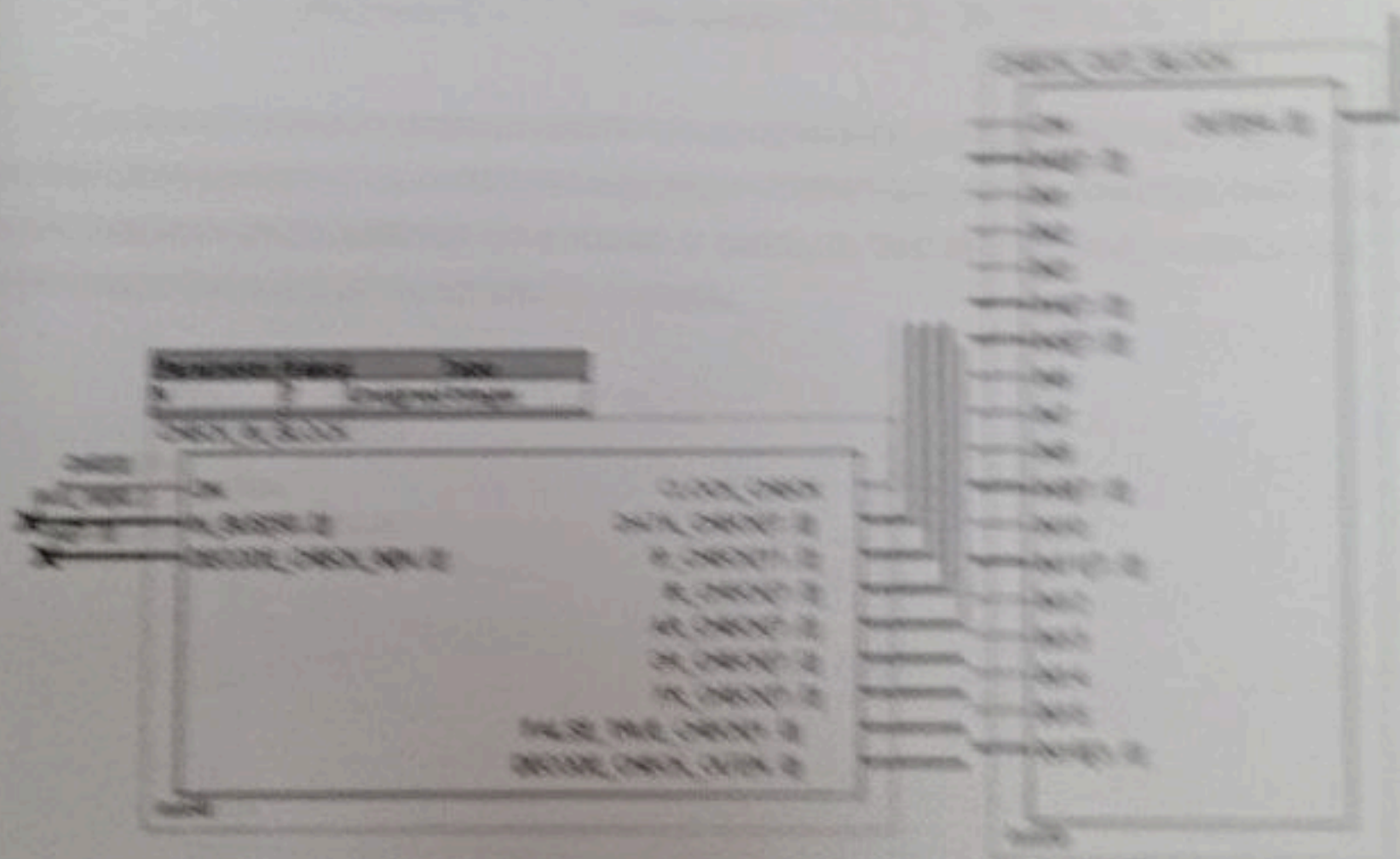


Рисунок 2.15 - Символическое представление блока арбитража к общей шине

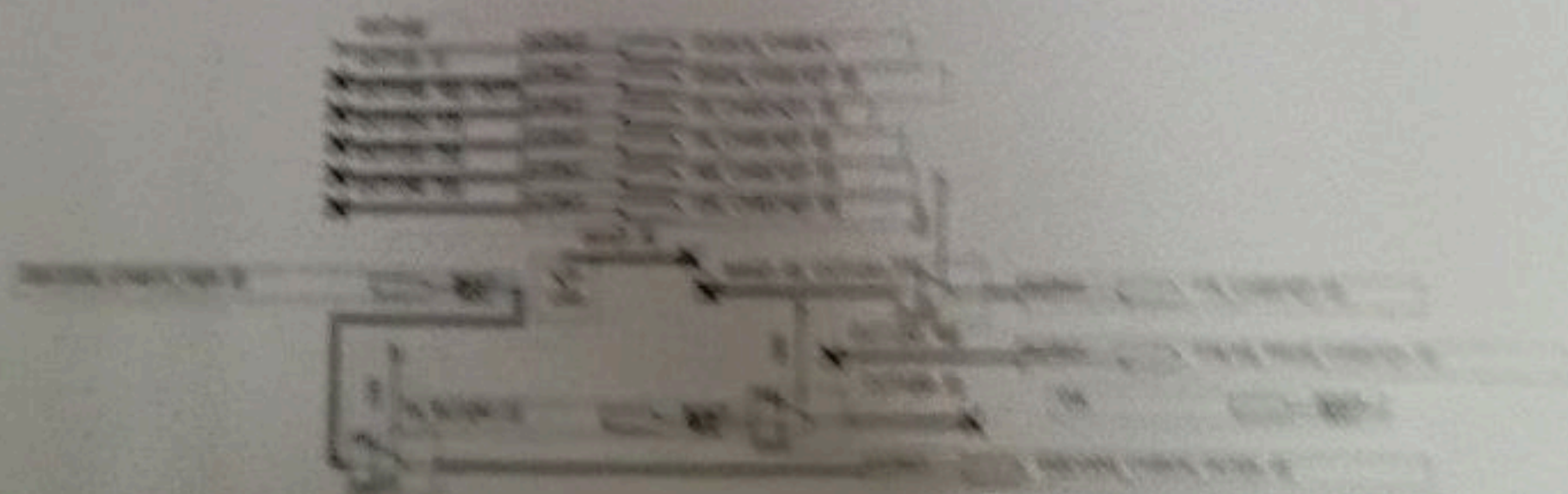


Рисунок 2.16 - Схема блока CHECK_IN_BLOCK

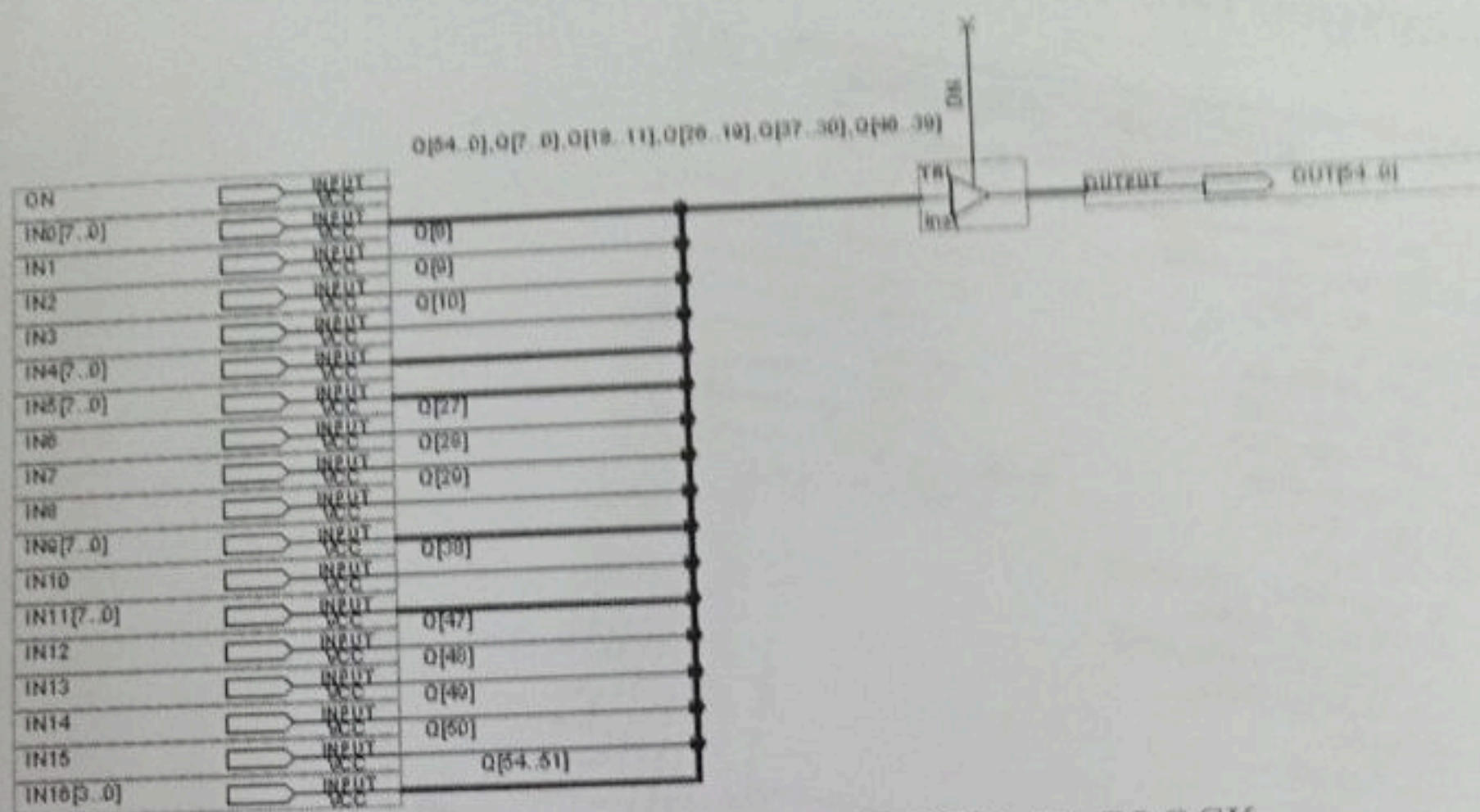


Рисунок 2.17 - Схема блока CHECK_IN_BLOCK

Схемы блоков тривиальны и подробному описанию не подлежат, всем, чем они занимаются, так это распределением (мультиплексированием и демультиплексированием) сигналов и шин, а так же «пропусканьем» конкретных сигналов на шины и линии.

2.4.3 Блок операции NOP (нет операции)

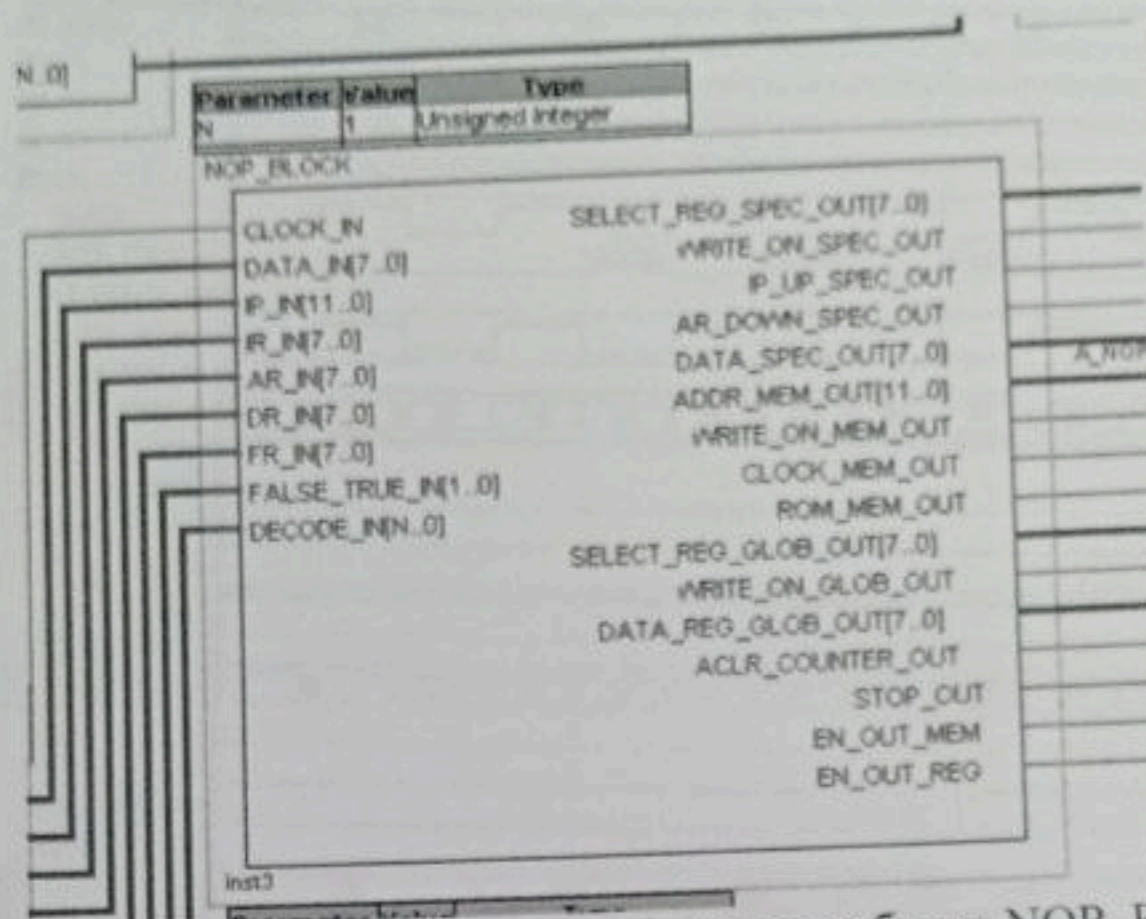


Рисунок 2.18 - Символическое представление блока NOP_BLOCK

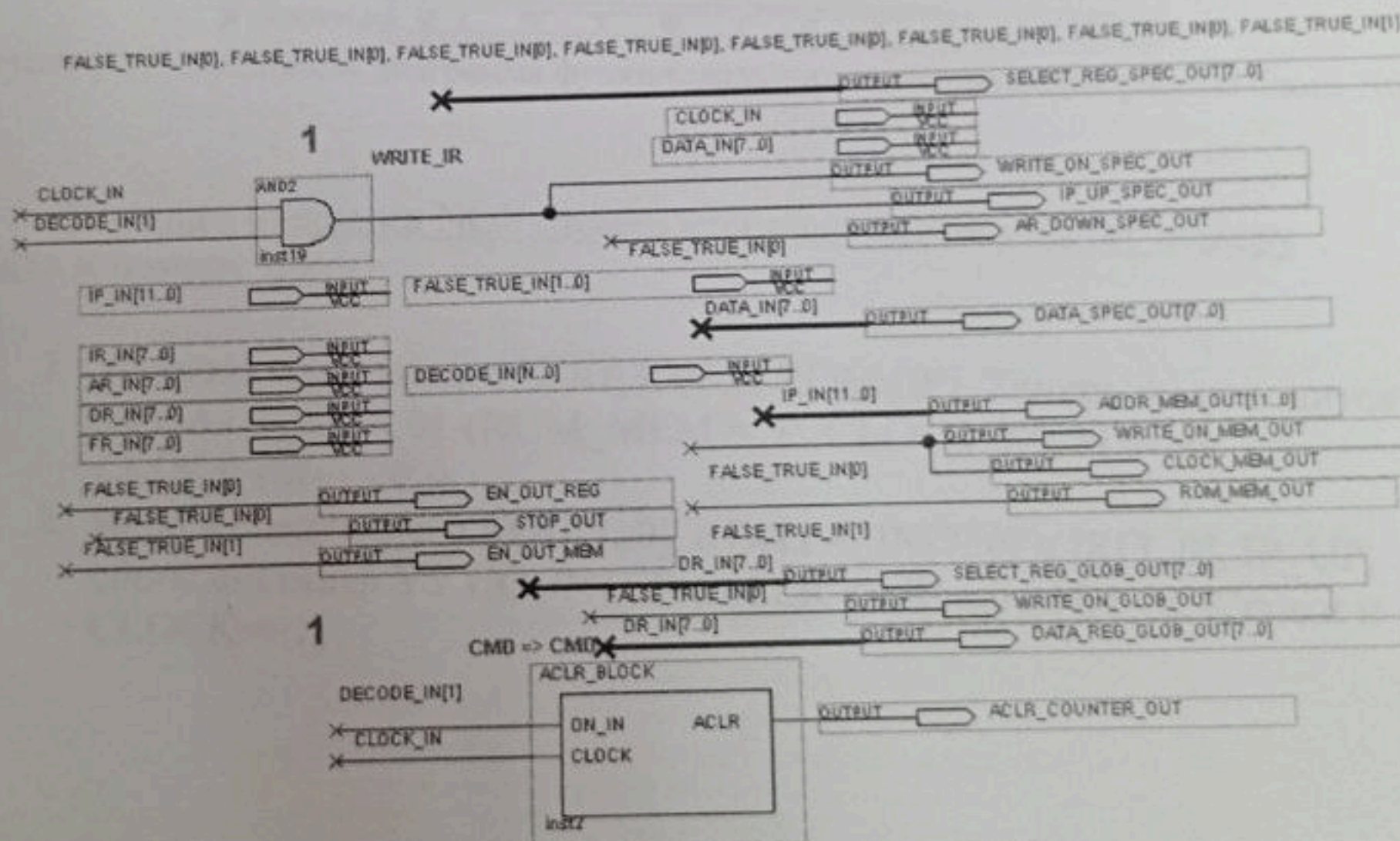


Рисунок 2.19 - Схема блока NOP_BLOCK

Time	Process	PCB	State	Priority	Wait Time	Turnaround Time
0	P1	1	Ready	1	0	0
1	P1	1	Running	1	0	1
2	P1	1	Running	1	0	2
3	P1	1	Running	1	0	3
4	P1	1	Running	1	0	4
5	P1	1	Running	1	0	5
6	P1	1	Running	1	0	6
7	P1	1	Running	1	0	7
8	P1	1	Running	1	0	8
9	P1	1	Running	1	0	9
10	P1	1	Running	1	0	10
11	P1	1	Running	1	0	11
12	P1	1	Running	1	0	12
13	P1	1	Running	1	0	13
14	P1	1	Running	1	0	14
15	P1	1	Running	1	0	15
16	P1	1	Running	1	0	16
17	P1	1	Running	1	0	17
18	P1	1	Running	1	0	18
19	P1	1	Running	1	0	19
20	P1	1	Running	1	0	20
21	P1	1	Running	1	0	21
22	P1	1	Running	1	0	22
23	P1	1	Running	1	0	23
24	P1	1	Running	1	0	24
25	P1	1	Running	1	0	25
26	P1	1	Running	1	0	26
27	P1	1	Running	1	0	27
28	P1	1	Running	1	0	28
29	P1	1	Running	1	0	29
30	P1	1	Running	1	0	30
31	P1	1	Running	1	0	31
32	P1	1	Running	1	0	32
33	P1	1	Running	1	0	33
34	P1	1	Running	1	0	34
35	P1	1	Running	1	0	35
36	P1	1	Running	1	0	36
37	P1	1	Running	1	0	37
38	P1	1	Running	1	0	38
39	P1	1	Running	1	0	39
40	P1	1	Running	1	0	40
41	P1	1	Running	1	0	41
42	P1	1	Running	1	0	42
43	P1	1	Running	1	0	43
44	P1	1	Running	1	0	44
45	P1	1	Running	1	0	45
46	P1	1	Running	1	0	46
47	P1	1	Running	1	0	47
48	P1	1	Running	1	0	48
49	P1	1	Running	1	0	49
50	P1	1	Running	1	0	50
51	P1	1	Running	1	0	51
52	P1	1	Running	1	0	52
53	P1	1	Running	1	0	53
54	P1	1	Running	1	0	54
55	P1	1	Running	1	0	55
56	P1	1	Running	1	0	56
57	P1	1	Running	1	0	57
58	P1	1	Running	1	0	58
59	P1	1	Running	1	0	59
60	P1	1	Running	1	0	60
61	P1	1	Running	1	0	61
62	P1	1	Running	1	0	62
63	P1	1	Running	1	0	63
64	P1	1	Running	1	0	64
65	P1	1	Running	1	0	65
66	P1	1	Running	1	0	66
67	P1	1	Running	1	0	67
68	P1	1	Running	1	0	68
69	P1	1	Running	1	0	69
70	P1	1	Running	1	0	70
71	P1	1	Running	1	0	71
72	P1	1	Running	1	0	72
73	P1	1	Running	1	0	73
74	P1	1	Running	1	0	74
75	P1	1	Running	1	0	75
76	P1	1	Running	1	0	76
77	P1	1	Running	1	0	77
78	P1	1	Running	1	0	78
79	P1	1	Running	1	0	79
80	P1	1	Running	1	0	80
81	P1	1	Running	1	0	81
82	P1	1	Running	1	0	82
83	P1	1	Running	1	0	83
84	P1	1	Running	1	0	84
85	P1	1	Running	1	0	85
86	P1	1	Running	1	0	86
87	P1	1	Running	1	0	87
88	P1	1	Running	1	0	88
89	P1	1	Running	1	0	89
90	P1	1	Running	1	0	90
91	P1	1	Running	1	0	91
92	P1	1	Running	1	0	92
93	P1	1	Running	1	0	93
94	P1	1	Running	1	0	94
95	P1	1	Running	1	0	95
96	P1	1	Running	1	0	96
97	P1	1	Running	1	0	97
98	P1	1	Running	1	0	98
99	P1	1	Running	1	0	99

Figure 1.1: A Gantt chart illustrating the execution of a single process (P1) over time. The chart shows the process running continuously from time 0 to 100, with no idle time or context switches.

Process Scheduling: APT (Adaptive Priority Time) Scheduling

Control Flow Table

1. When a process is ready to execute, its priority is calculated based on its arrival time and its current priority. The process with the highest priority is selected for execution.
2. When a process is running, its priority is updated based on its execution time and its current priority. The process with the highest priority is selected for execution.

2.4.4 Блок операции JMP (безусловный переход)

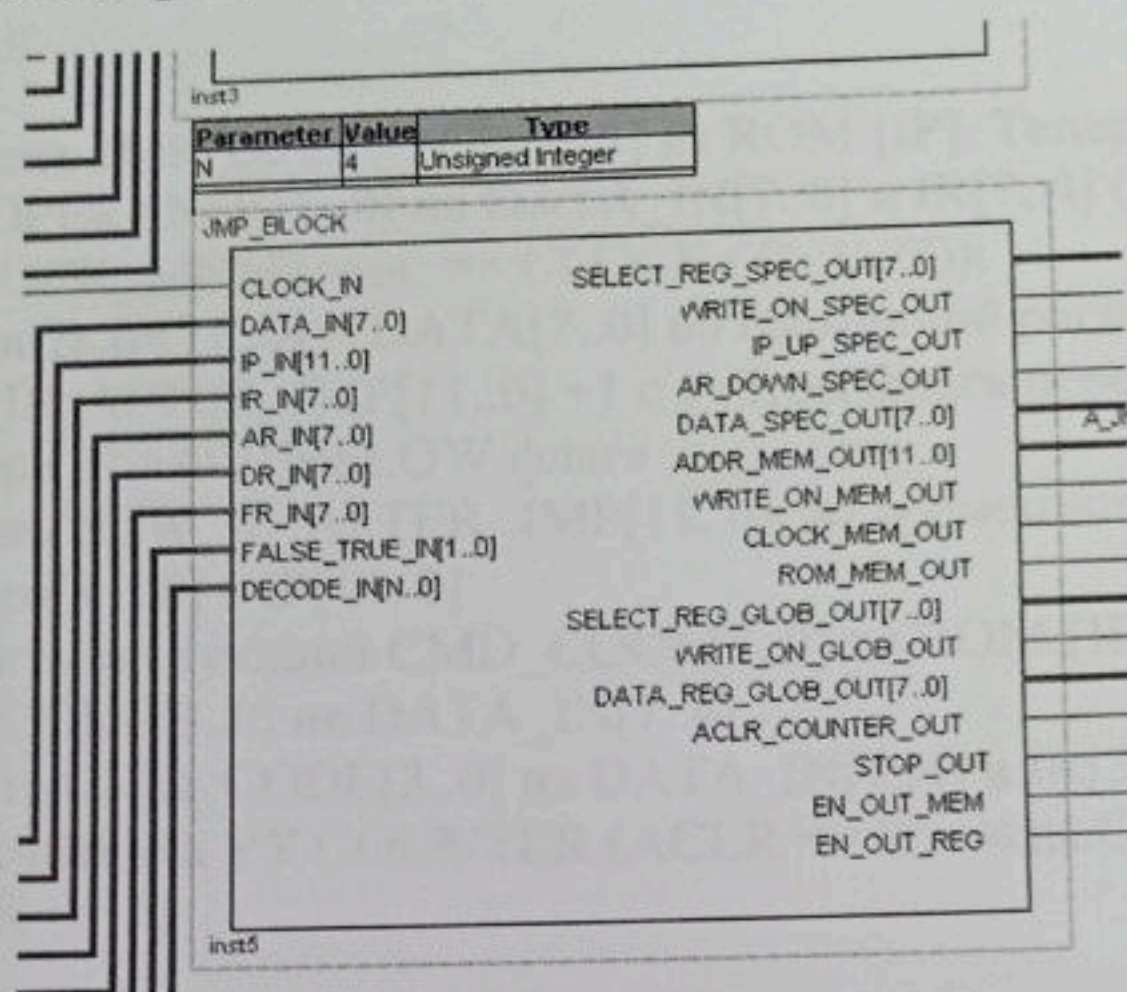


Рисунок 2.21 - Символическое представление блока JMP_BLOCK

Схема блока JMP_BLOCK представлена в приложении Ж.

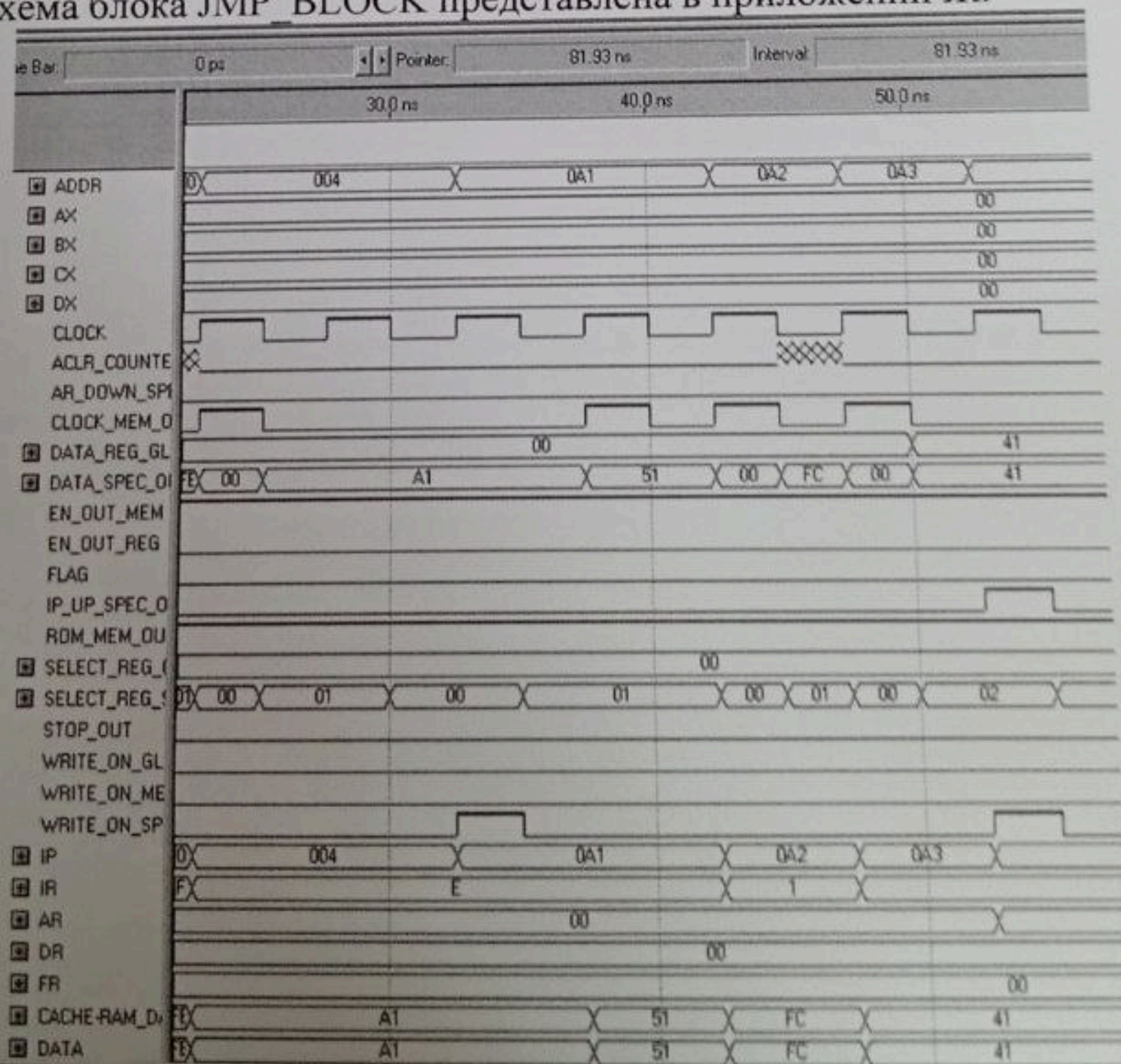


Рисунок 2.22 - Временная диаграмма функционального моделирования работы блока JMP BLOCK

Итерации операции JMP (номер итерации соответствует номеру пункта и номеру такта):

1. загрузить JMP_OPERAND[11..0] из ROM:[IP]. Теперь JMP_OPERAND[11..0] на DATA_IN[7..0] и IR[7..4] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'));
2. сохранить IP[11..0] + DATA[7..0] во временной регистр[11..0] = NEW_IP_AFTER_JMP[11..0] + 1 с учётом флага переполнения регистра IP – OVERFLOW флага
3. записать NEW_IP_AFTER_JMP[11..0] из временного регистра[11..0] в IP[11..0]
4. загрузить следующий CMD_CODE[3..0] из ROM:[IP]. Теперь CMD_CODE[3..0] на DATA_IN[7..4]
5. записать CMD_CODE[3..0] из DATA_IN[7..4] в IR[3..0], IP_UP, сброс счётчика УУ COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

2.4.5 Блок операции M->R (чтение из RAM в POH)

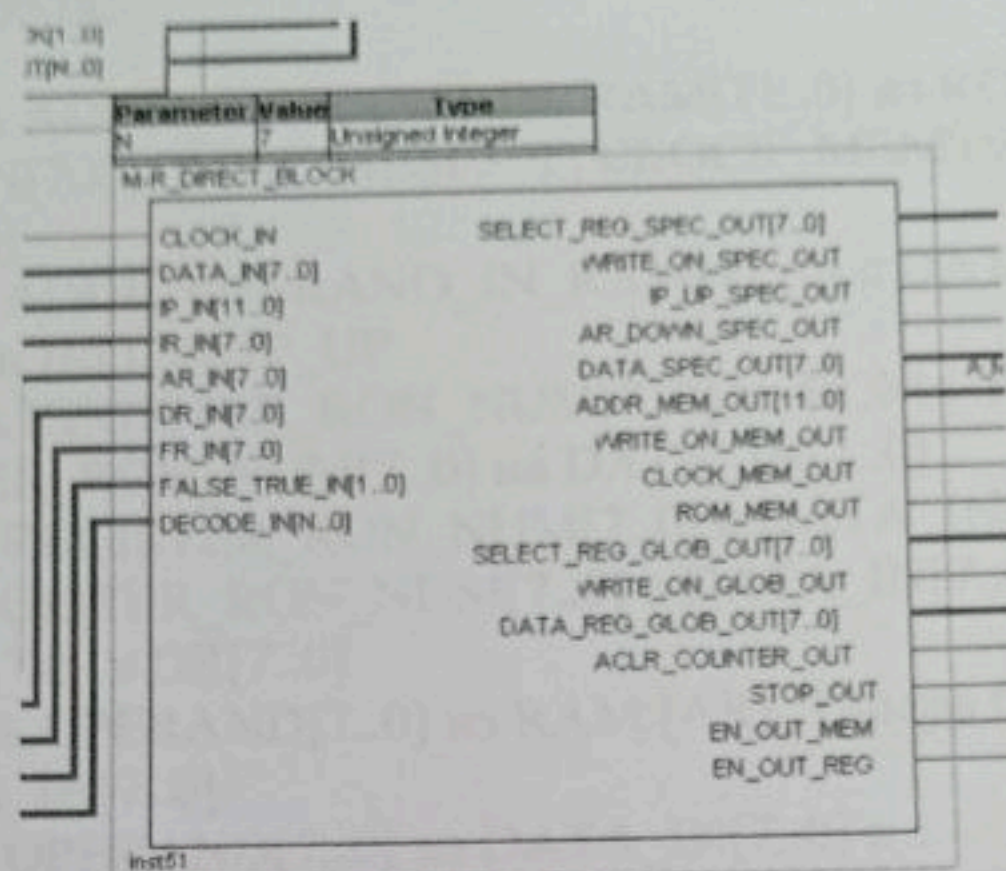


Рисунок 2.23 - Символическое представление блока M-R_DIRECT_BLOCK

Схема блока M-R_DIRECT_BLOCK представлена в приложении И.

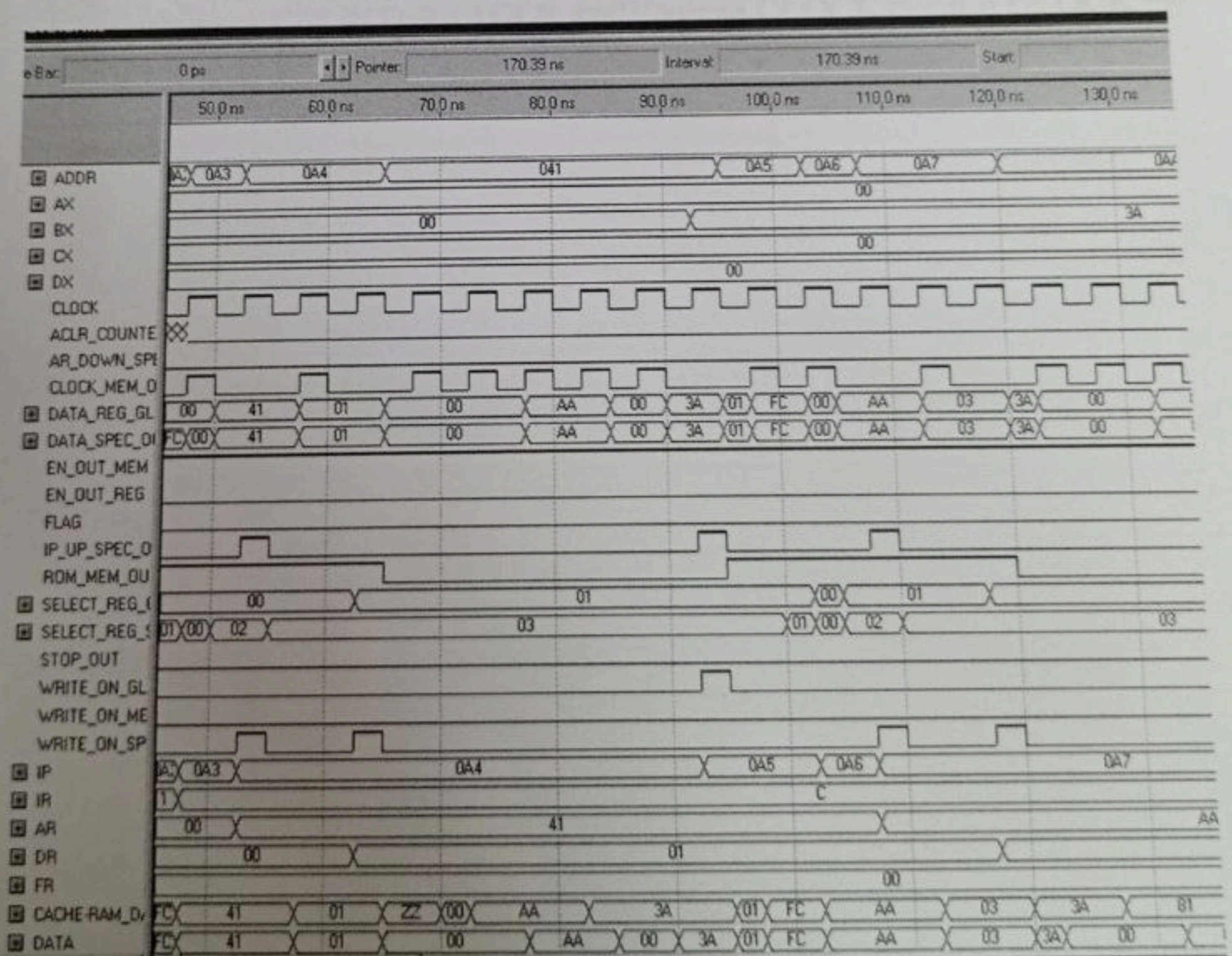


Рисунок 2.24 - Временная диаграмма функционального моделирования работы блока M-R_DIRECT_BLOCK

Итерации операции M->R (номер итерации соответствует номеру пункта и номеру такта):

1. загрузить ADDR_OPERAND_IN_RAM[11..0] из ROM:[IP] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. записать ADDR_OPERAND_IN_RAM[11..0] с DATA_IN[7..0] в AR[7..0] и IR[3..0], IP_UP
3. загрузить REGISTER_RON_NUM[7..0] из ROM:[IP]. Теперь REGISTER_RON_NUM[7..0] на DATA_IN[7..0]
4. записать REGISTER_RON_NUM[7..0] с DATA_IN[7..0] в DR[7..0]. Если REGISTER_RON_NUM[7..0] с DATA_IN[7..0] > '19' то записать '00' в DR[7..0]
5. загрузить OPERAND[7..0] из RAM:[AR]. Теперь OPERAND[7..0] на DATA_IN[7..0]
6. записать OPERAND[7..0] из DATA_IN[7..0] в REGISTER_RON_NUM[7..0] код которого в DR[7..0], IP_UP
7. загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь следующий CMD_CODE[3..0] на DATA_IN[7..0]
8. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сброс счётчика COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

2.4.6 Блок операции R->M (запись из РОН в RAM)

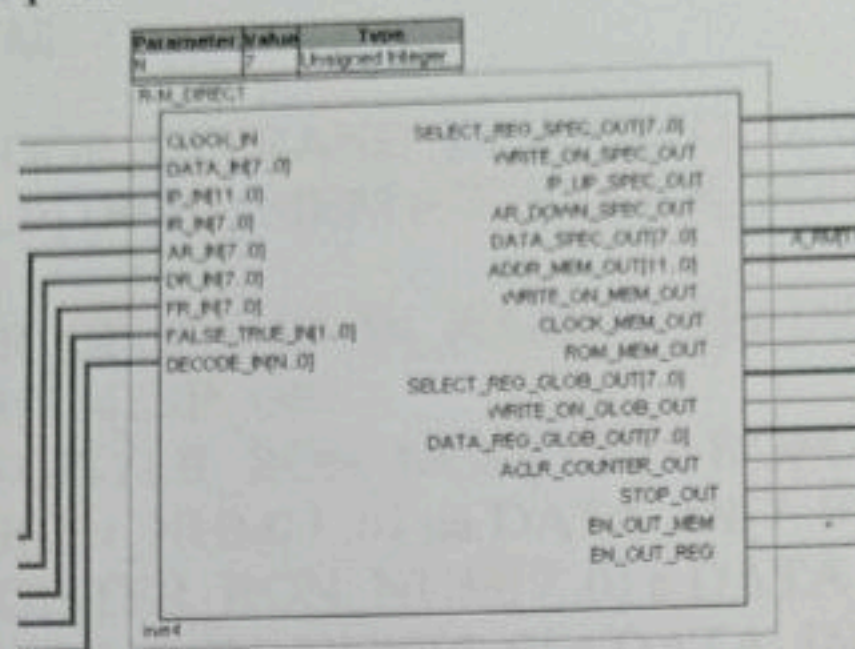


Рисунок 2.25 - Символическое представление блока R-M_DIRECT

Схема блока R-M_DIRECT представлена в приложении К.

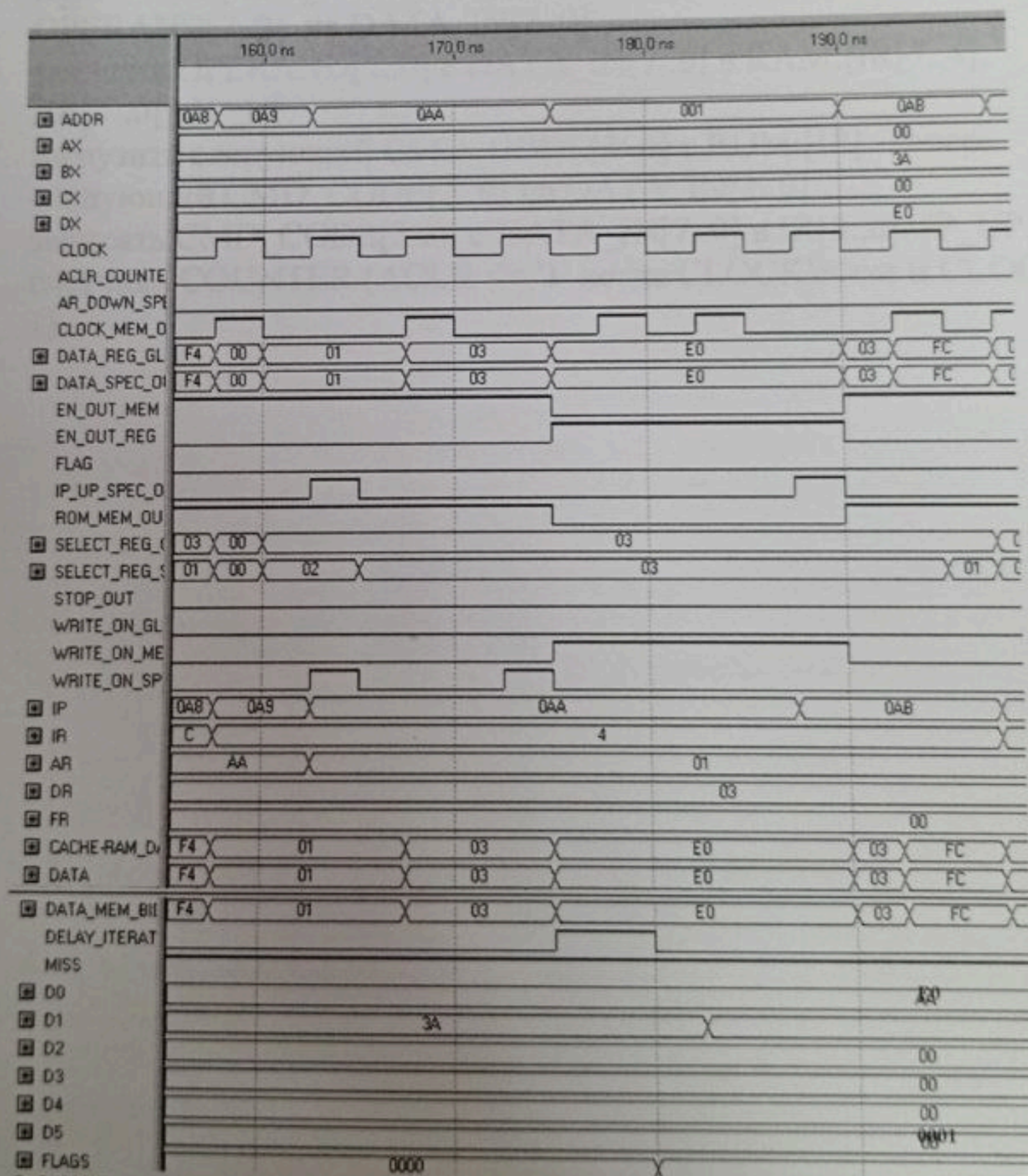


Рисунок 2.26 - Временная диаграмма функционального моделирования работы блока R-M_DIRECT

Итерации операции R->M (номер итерации соответствует номеру пункта и номеру такта):

1. загрузить ADDR_OPERAND_IN_RAM[11..0] с ROM:[IP] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. запись ADDR_OPERAND_IN_RAM[11..0] с DATA_IN[7..0] в AR[7..0] и IR[7..4], IP_UP
3. загрузить REGISTER_RON_NUM[7..0] с ROM:[IP]. Теперь REGISTER_RON_NUM[7..0] на DATA_IN[7..0]
4. записать REGISTER_RON_NUM[7..0] с DATA_IN[7..0] в DR[7..0]. Если REGISTER_RON_NUM[7..0] с DATA_IN[7..0] > '19' то записать '00' в DR[7..0]
5. загрузить OPERAND[7..0] с POH в соответствии с REGISTER_RON_NUM[7..0] код которого в DR[7..0]. Теперь OPERAND[7..0] на DATA_IN[7..0]
6. записать OPERAND[7..0] с DATA_IN[7..0] в RAM:[IR[7..4], AR[7..0]], IP_UP
7. загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь следующий CMD_CODE[3..0] на DATA_IN[7..0]
8. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сброс счётчика COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

2.5 Арифметико-логическое устройство

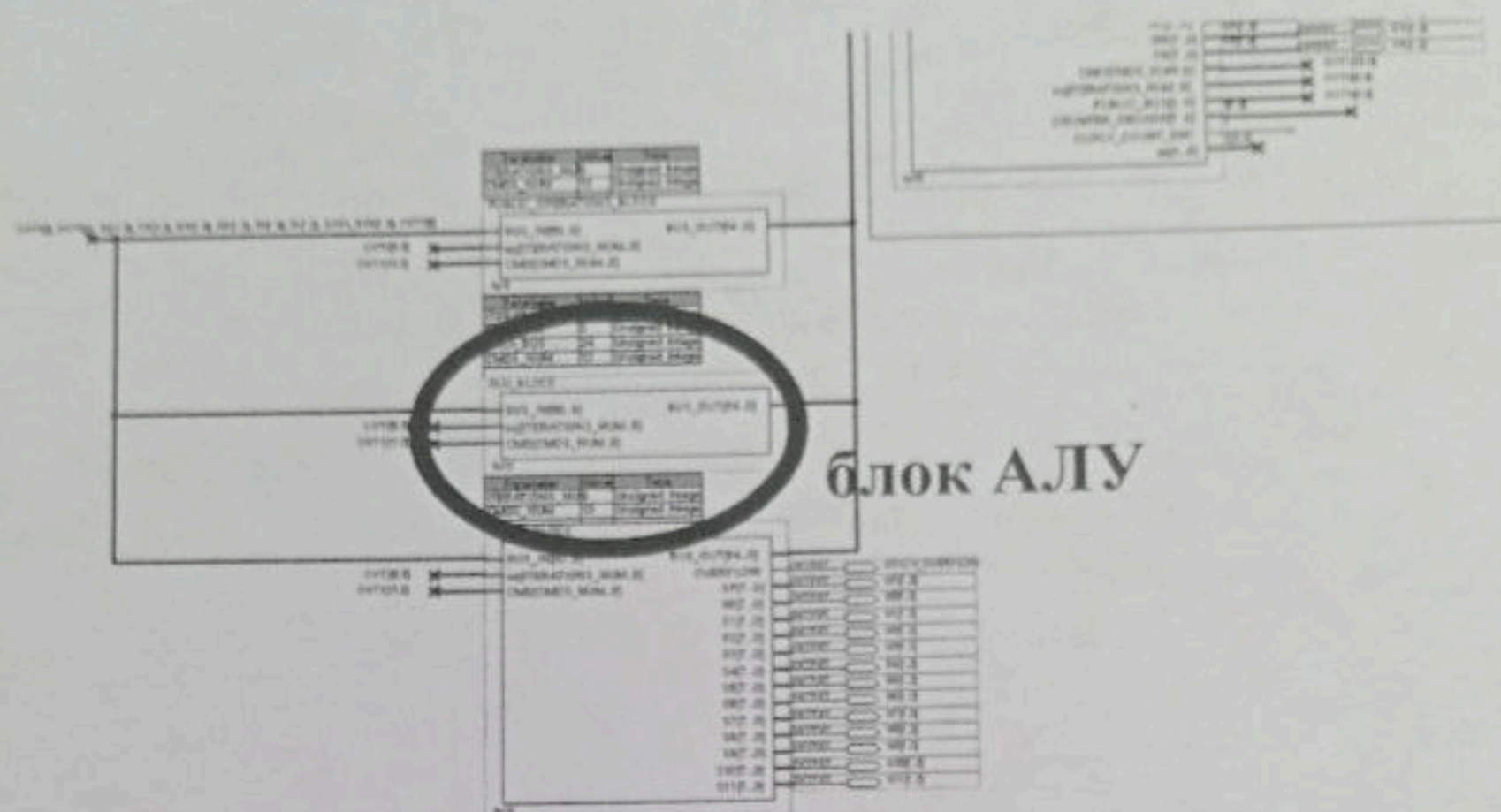


Рисунок 2.27 - Расположение блока арифметико-логического устройства в корне проекта

Схема блока арифметико-логического устройства представлена в приложении Л.

Состав блока и назначение основных элементов схемы:

- блоки CHECK_IN_ALU, блоки CHECK_OUT_ALU – обеспечение арбитража к общей шине BUS_OUT[54..0]
- ALU_CONTROL – блок регистров АЛУ и схема управления ими
- ROL_BLOCK – блок операции «ЦИКЛИЧЕСКИЙ СДВИГ ВПРАВО»
- INCS_DIRECT_BLOCK – блок операции «СЛОЖЕНИЕ С ПЕРЕНОСОМ»
- INCS_REGISTERS_BLOCK – блок операции «ИНКРЕМЕНТ ПО ФЛАГУ S»
- NAND_DIRECT_BLOCK – блок «ПОБИТОВОЕ ИНВЕРТИРОВАНИЕ»
- NAND_IMMEDIATE_BLOCK – блок операции побитового «И»

Состав пинов исполнительных блоков АЛУ и их назначение аналогичен составу пинов и их назначению исполнительных блоков общих операций. Однако добавились некоторые дополнительные пины, которые будут приведены далее.

Все исполнительные блоки АЛУ имеют сходные входные и выходные пины.

Дополнительные входные пины исполнительного блока АЛУ и их назначение:

- A_IN[7..0] – шина данных регистра АЛУ A[7..0]
- B_IN[7..0] – шина данных регистра АЛУ B[7..0]
- C_IN[7..0] – шина данных регистра АЛУ C[7..0]

Дополнительные выходные пины исполнительного блока АЛУ и их назначение:

- CLOCK_CNTRL[REG_NUM-1..0] – общая тактовая шина блоков регистров АЛУ
- DATA_CNTRL[DATA_BUS-1..0] – общая шина данных блока регистров АЛУ

2.5.1 Блоки арбитража к общим шинам

Таких блоков всего лишь два (для каждого блока исполнительской операции по 2шт.):

- CHECK_IN_ALU – блок занимается «включением схем операций», обеспечивает «прокидывание» управляющих сигналов и шин данных на конкретный исполнительный блок конкретной операции АЛУ
- CHECK_OUT_ALU – блок занимается контролем доступа к общей шине BUS_OUT[54..0], так чтобы в один момент данную шину мог захватить лишь один исполнительный блок АЛУ

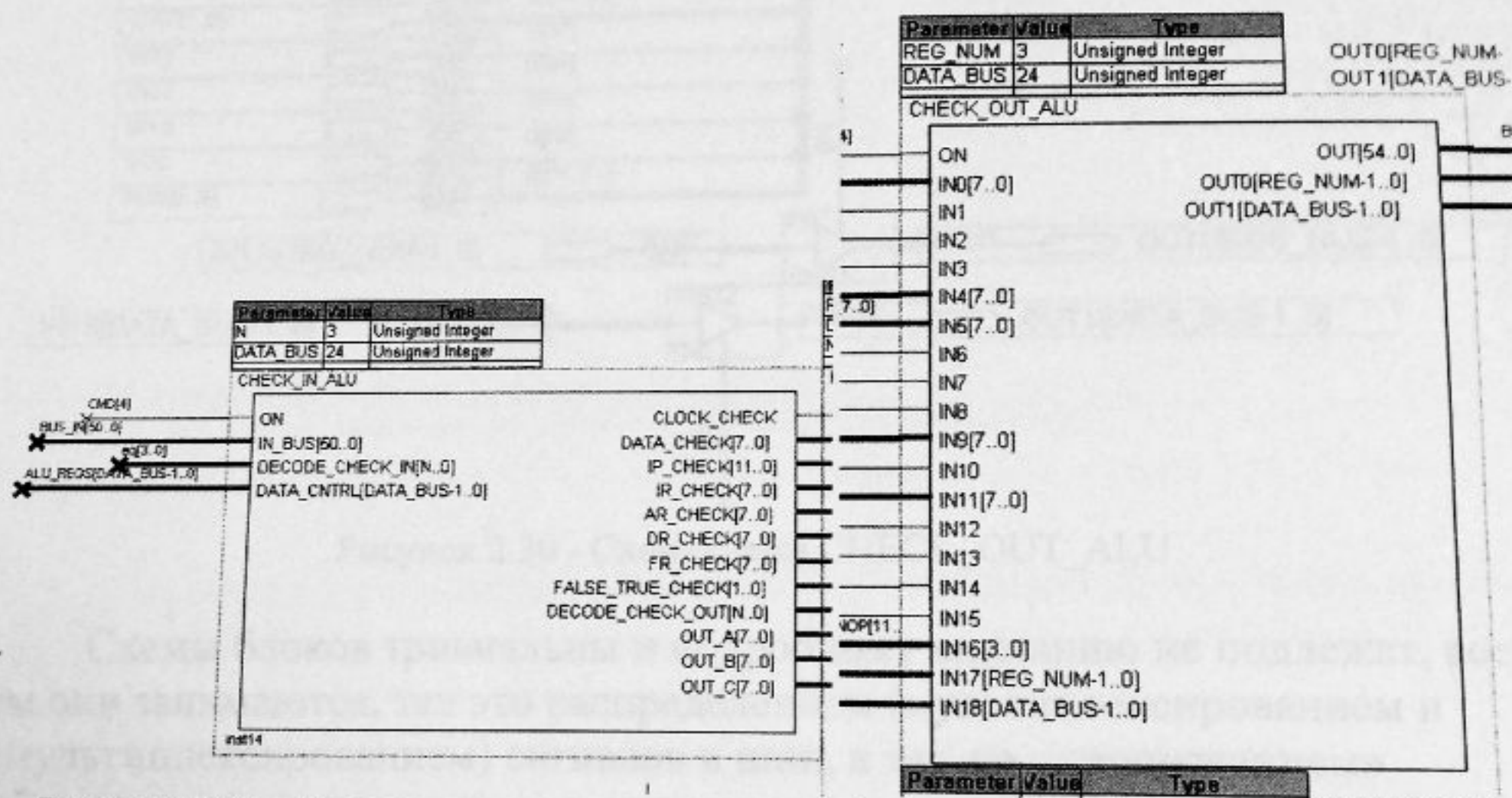


Рисунок 2.28 - Символическое представление блоков арбитража к общим шинам

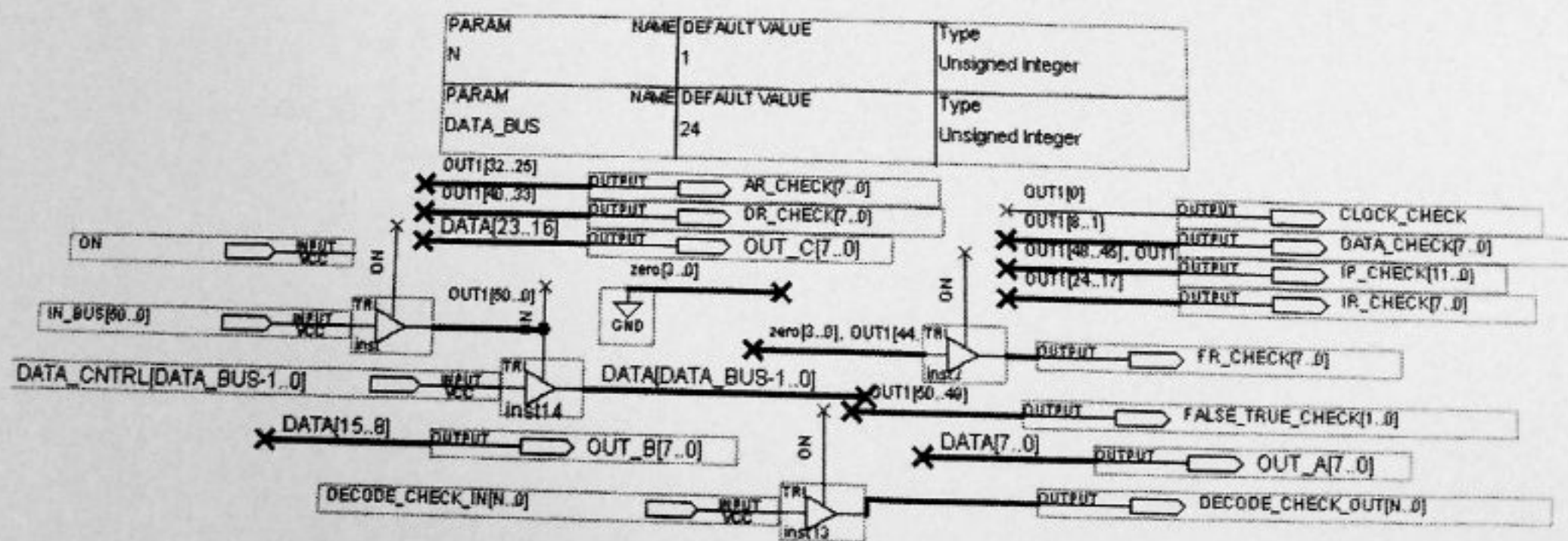


Рисунок 2.29 - Схема блока CHECK_IN_ALU

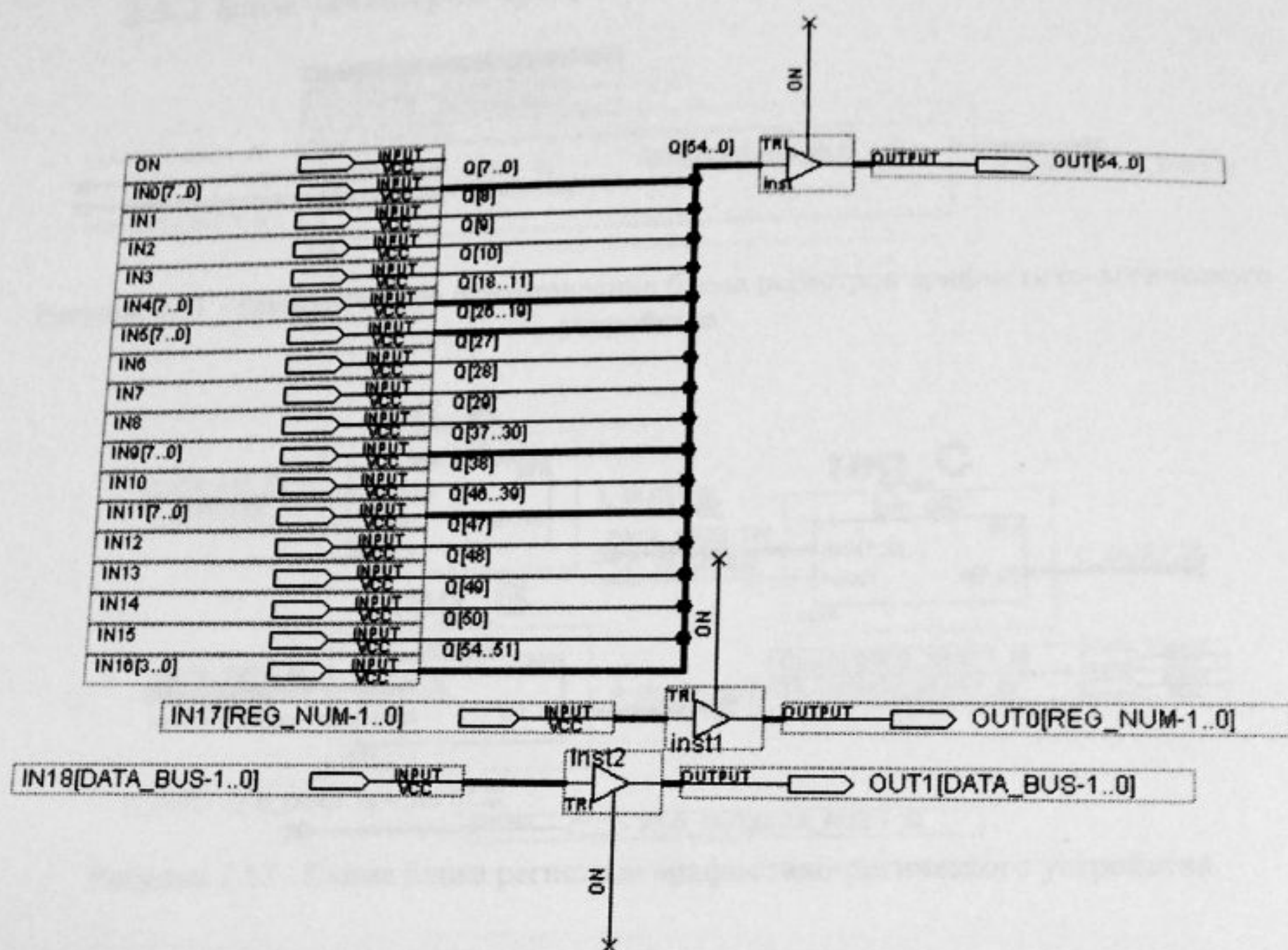


Рисунок 2.30 - Схема блока CHECK_OUT_ALU

Схемы блоков тривиальны и подробному описанию не подлежат, всем, чем они занимаются, так это распределением (мультиплексированием и демультиплексированием) сигналов и шин, а так же «пропусканием» конкретных сигналов на шины и линии.

2.5.2 Блок регистров арифметико-логического устройства

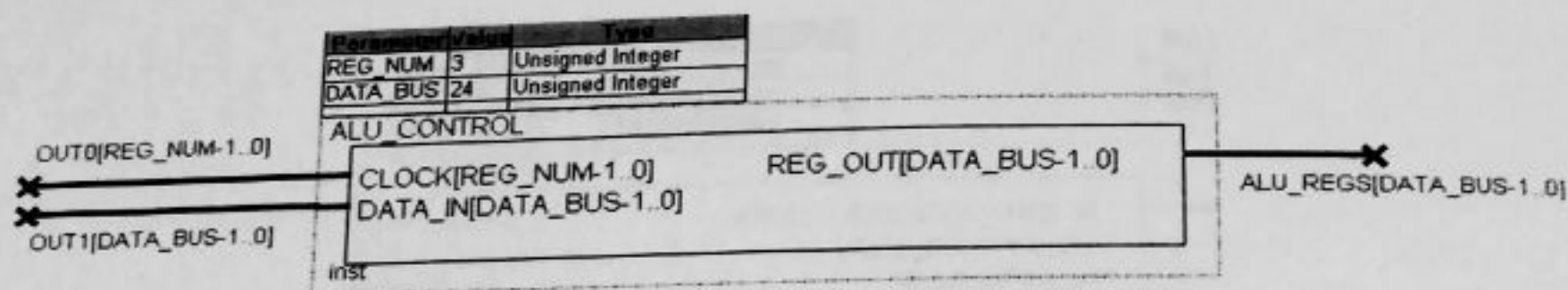


Рисунок 2.31 - символическое представление блока регистров арифметико-логического устройства

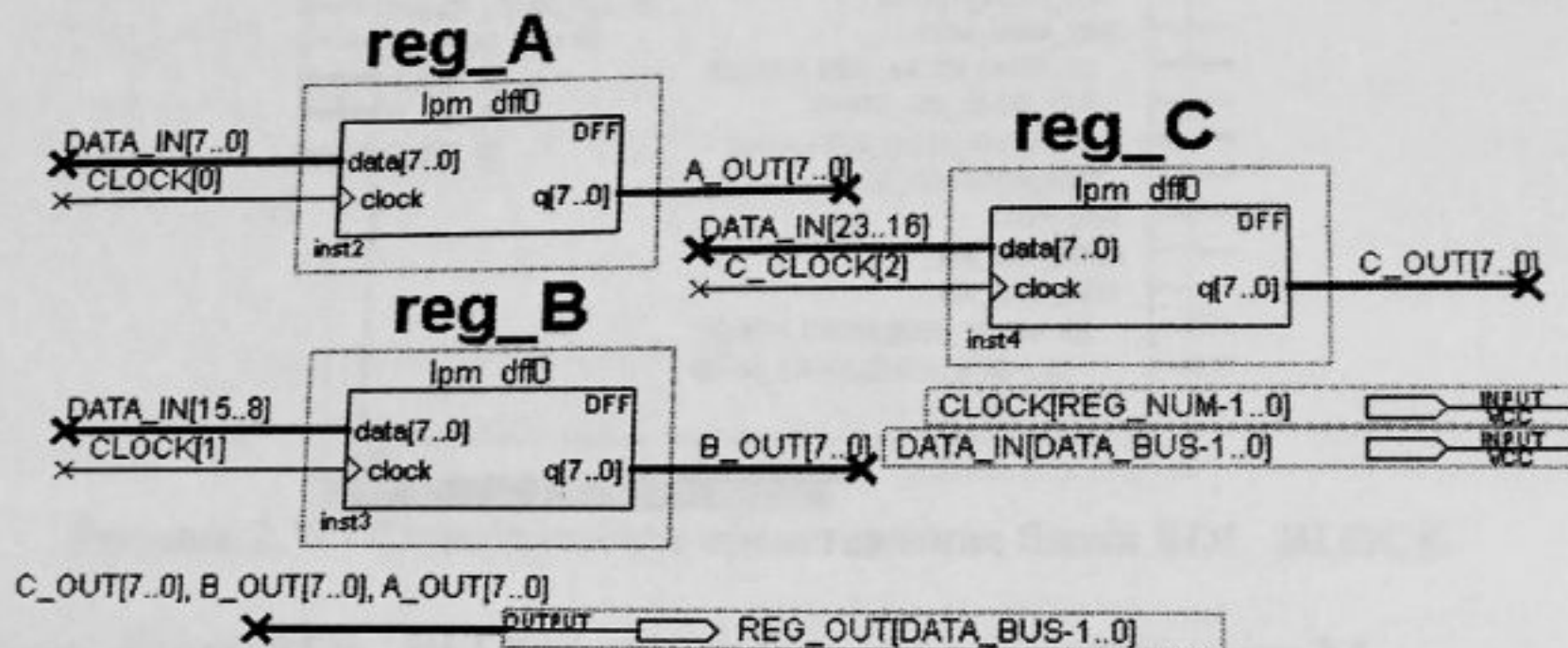


Рисунок 2.32 - Схема блока регистров арифметико-логического устройства

Схема блока тривиальна и подробному описанию не подлежит, всем, чем она занимаются, так это запись в регистры А, В, С и считывание их содержимого в любой момент времени.

2.5.3 Блок операции ROR (циклический сдвиг вправо)

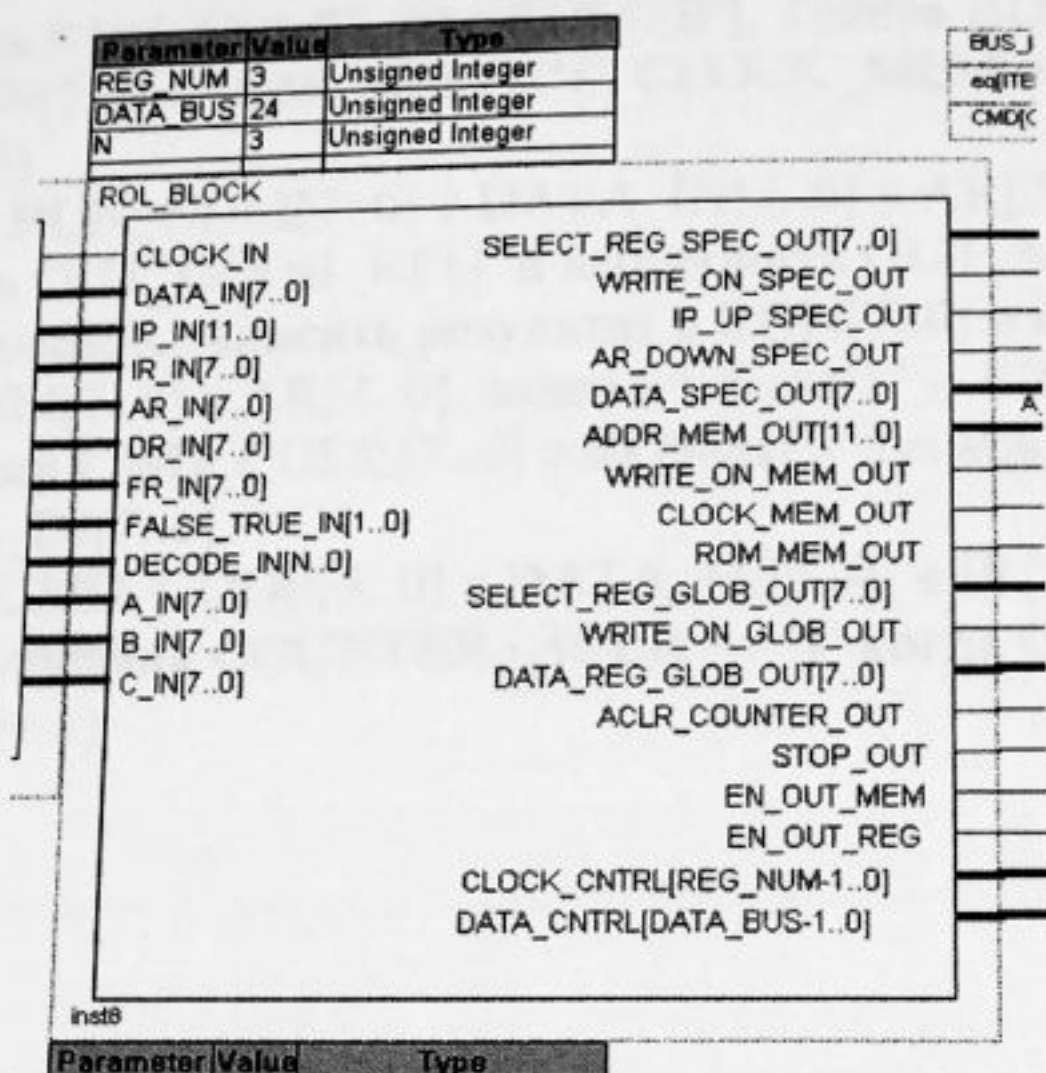


Рисунок 2.33 - Символическое представление блока ROL_BLOCK

Схема блока ROL_BLOCK представлена в приложении М.

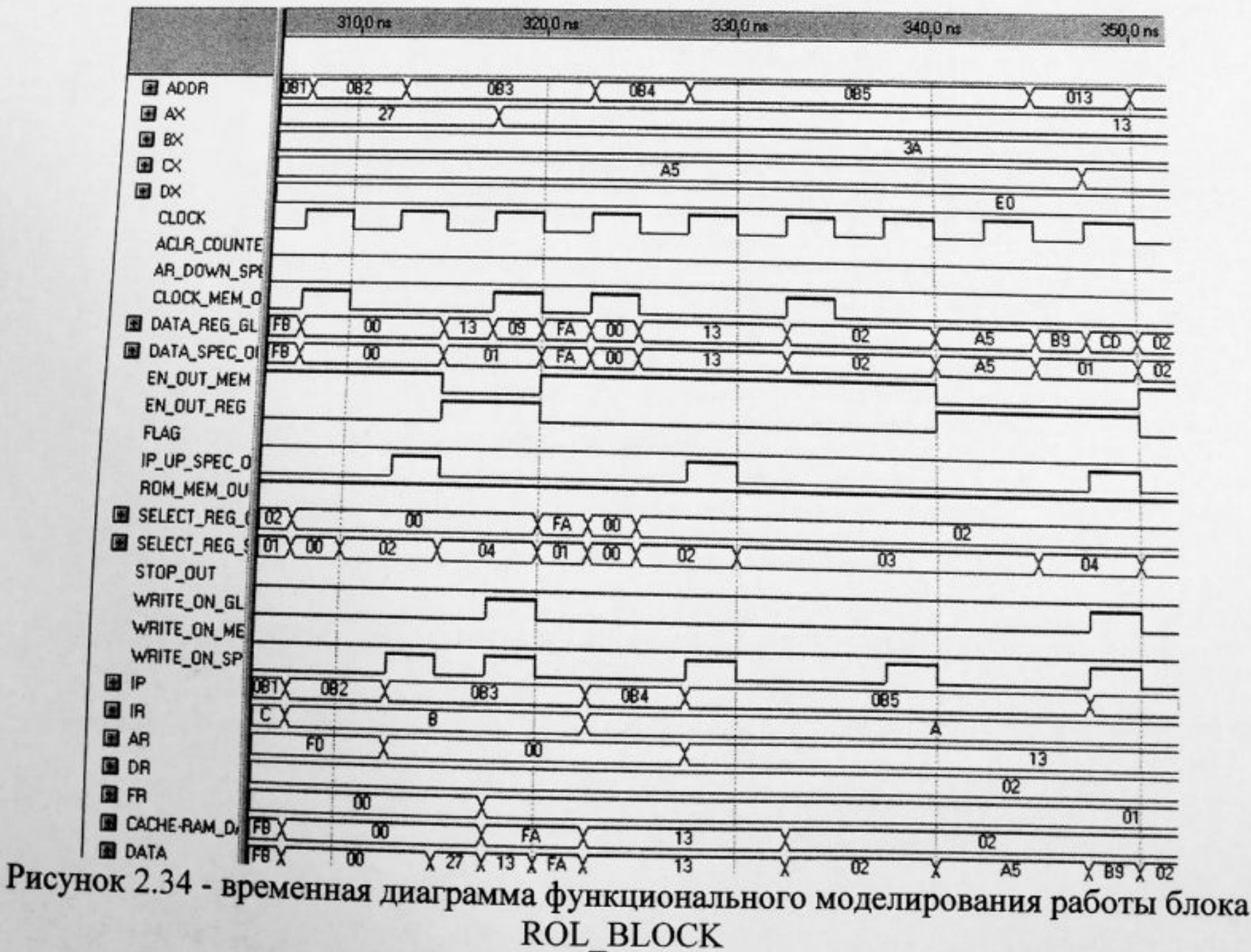


Рисунок 2.34 - временная диаграмма функционального моделирования работы блока ROL_BLOCK

Итерации операции ROR (номер итерации соответствует номеру пункта и номеру такта):

1. Загрузить RON_NUM[7..0] с ROM:[IP]. Теперь RON_NUM[7..0] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. записать RON_NUM[7..0] с DATA_IN[7..0] в AR[7..0], IP_UP
3. включить OUT_GLOB_REG и выключить OUT_MEM, выполнить ROR операцию, записать результат в RON[7..0] в соответствии с RON_NUM[7..0] в AR[7..0], записать флаг 'S' в FR[0], загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь CMD_CODE[3..0] на DATA_IN[7..0]
4. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сбросить счётчик COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

2.5.4 Блок операции ADDC (сложение с переносом)

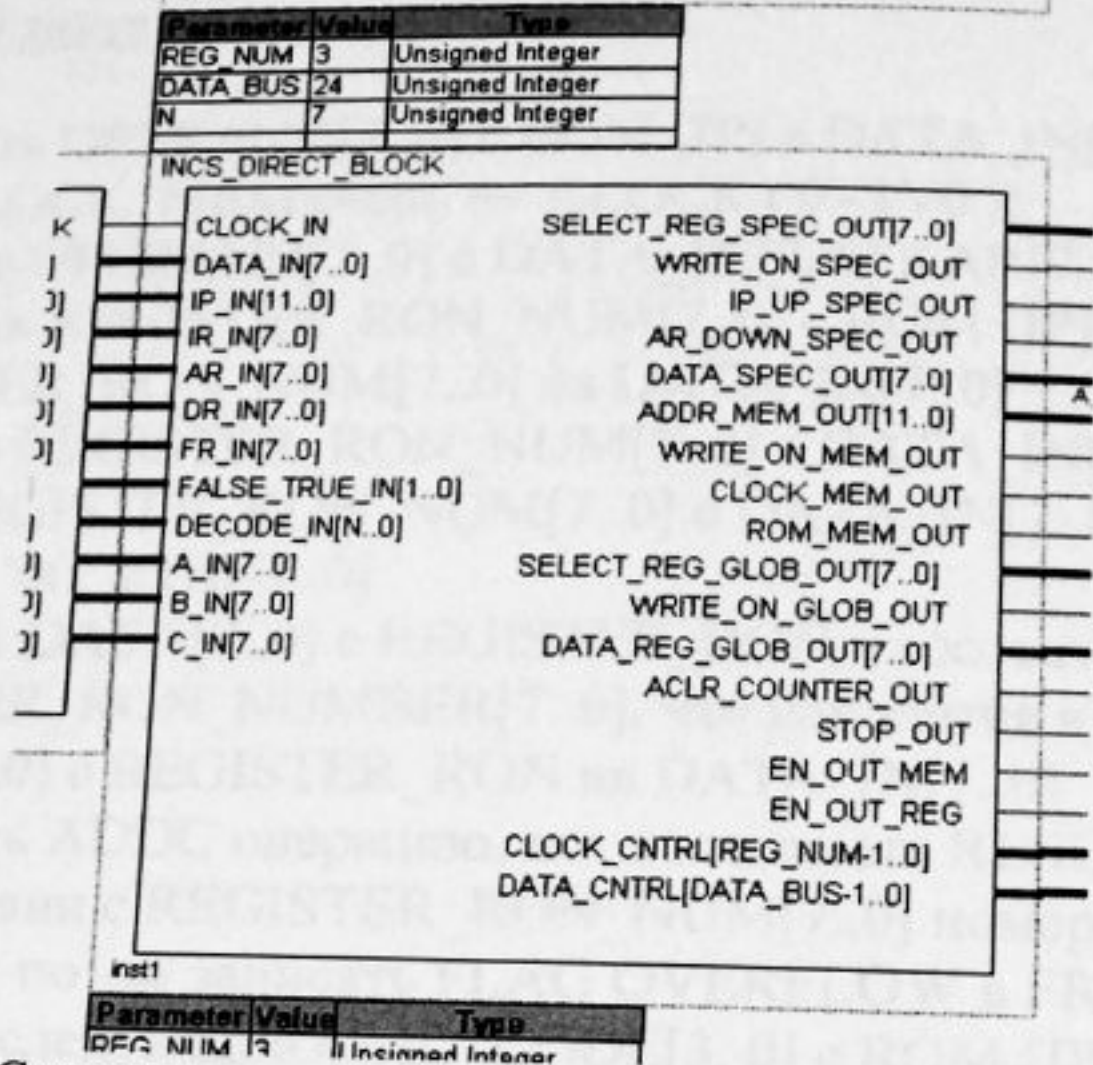


Рисунок 2.35 - Символическое представление блока INCS_DIRECT_BLOCK

Схема блока INCS_DIRECT_BLOCK представлена в приложении Н.

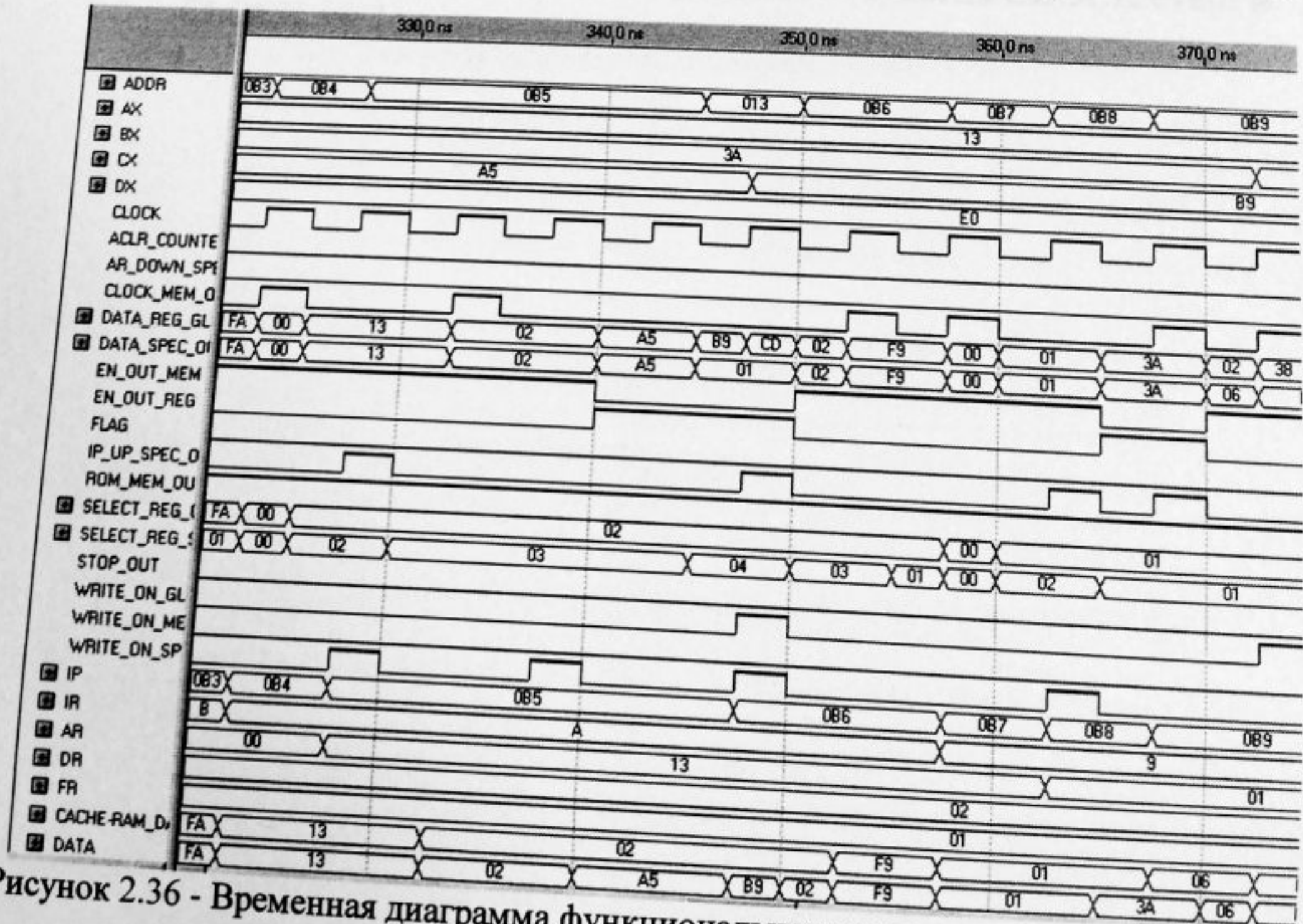


Рисунок 2.36 - Временная диаграмма функционального моделирования работы блока INCS_DIRECT_BLOCK

Итерации операции ADDC (номер итерации соответствует номеру пункта и номеру такта):

1. загрузить OPERAND[7..0] с ROM:[IP] в DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. записать OPERAND[7..0] с DATA_IN[7..0] в AR[7..0], IP_UP
3. загрузить REGISTER_RON_NUM[7..0] с ROM:[IP]. Теперь REGISTER_RON_NUM[7..0] на DATA_IN[7..0]
4. записать REGISTER_RON_NUM[7..0] с DATA_IN[7..0] в DR[7..0].
Если REGISTER_RON_NUM[7..0] с DATA_IN[7..0] > '19' то записать '00' в DR[7..0]
5. загрузить DATA[7..0] с REGISTER_RON в соответствии с REGISTER_RON_NUMBER[7..0], что находится в DR[7..0], теперь DATA[7..0] с REGISTER_RON на DATA_IN[7..0]
6. выполнить ADDC операцию, потом записать RESULT[7..0] в POH в соответствии с REGISTER_RON_NUM[7..0] номер которого в DR[7..0], потом записать FLAG_OVERFLOW в FR[1], IP_UP
7. загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь CMD_CODE[7..0] на DATA_IN[7..0]
8. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сбросить счётчик COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

2.5.5 Блок операции INCS (инкремент по флагу S)

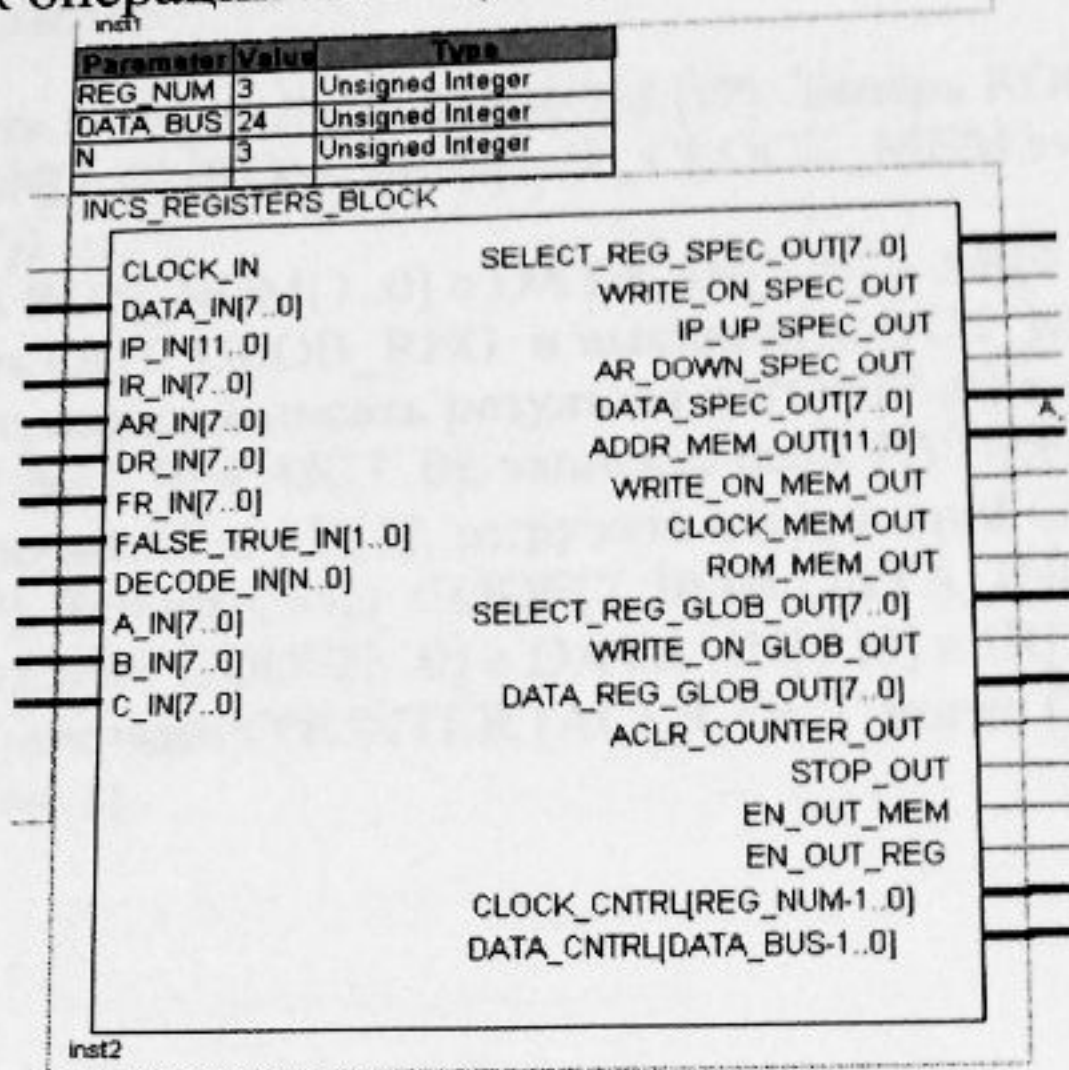


Рисунок 2.37 - Символическое представление блока INCS_REGISTERS_BLOCK

Схема блока INCS_REGISTERS_BLOCK представлена в приложении

П.

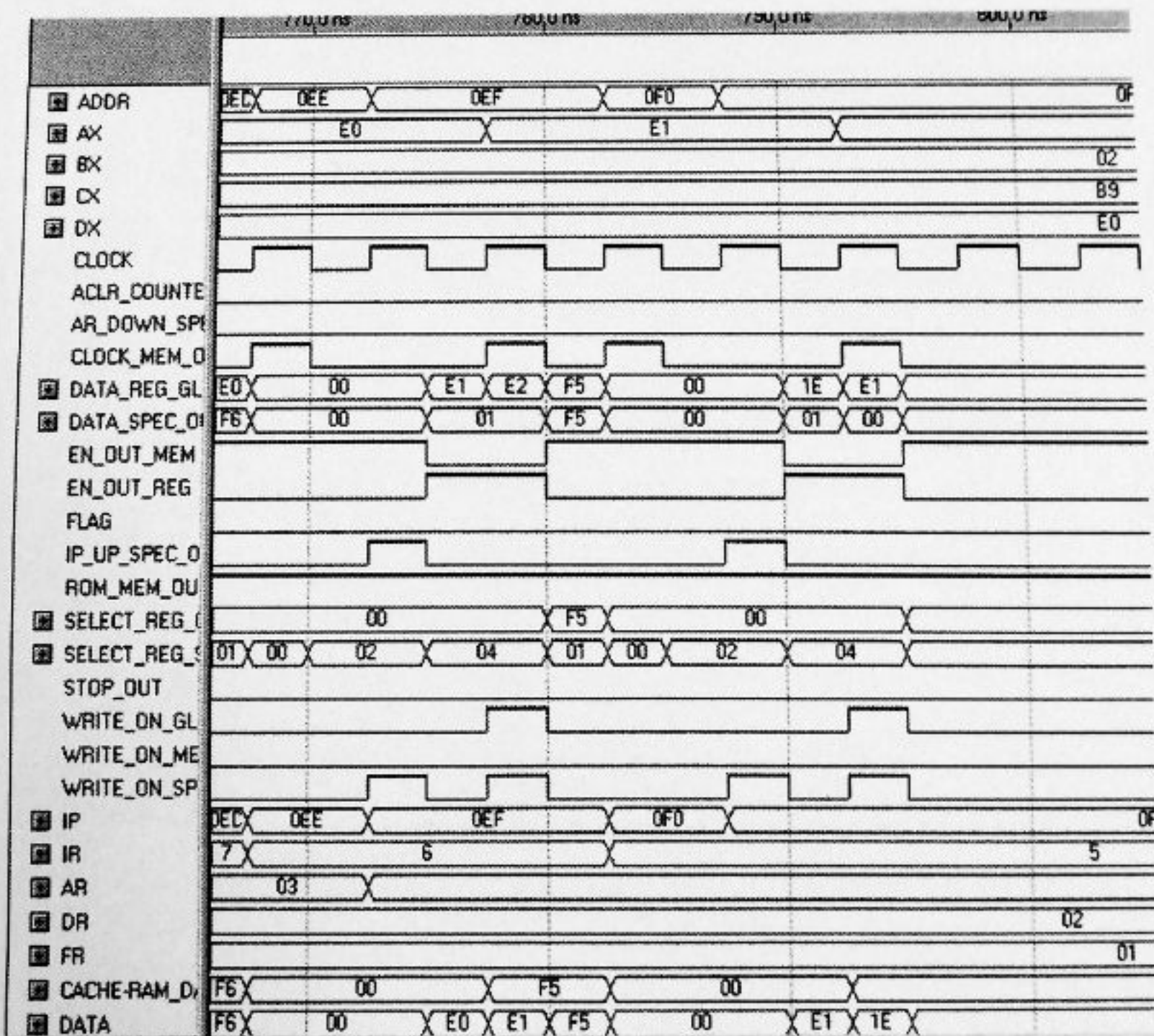


Рисунок 2.38 - Временная диаграмма функционального моделирования работы блока INCS_REGISTERS_BLOCK

Итерации операции INCS (номер итерации соответствует номеру пункта и номеру такта):

1. загрузить RON_NUM[7..0] с ROM:[IP]. Теперь RON_NUM[7..0] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. записать RON_NUM[7..0] с DATA_IN[7..0] в AR[7..0], IP_UP
3. включить OUT_GLOB_REG и выключить OUT_MEM, выполнить INCS операцию, записать результат в RON[7..0] в соответствии с RON_NUM[7..0] в AR[7..0], записать флаг OVERFLOW именованный 'O' в FR[1], загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь CMD_CODE[7..0] на DATA_IN[7..0]
4. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сбросить счётчик COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK = 1)

2.5.6 Блок операции NOT (побитовое инвертирование)

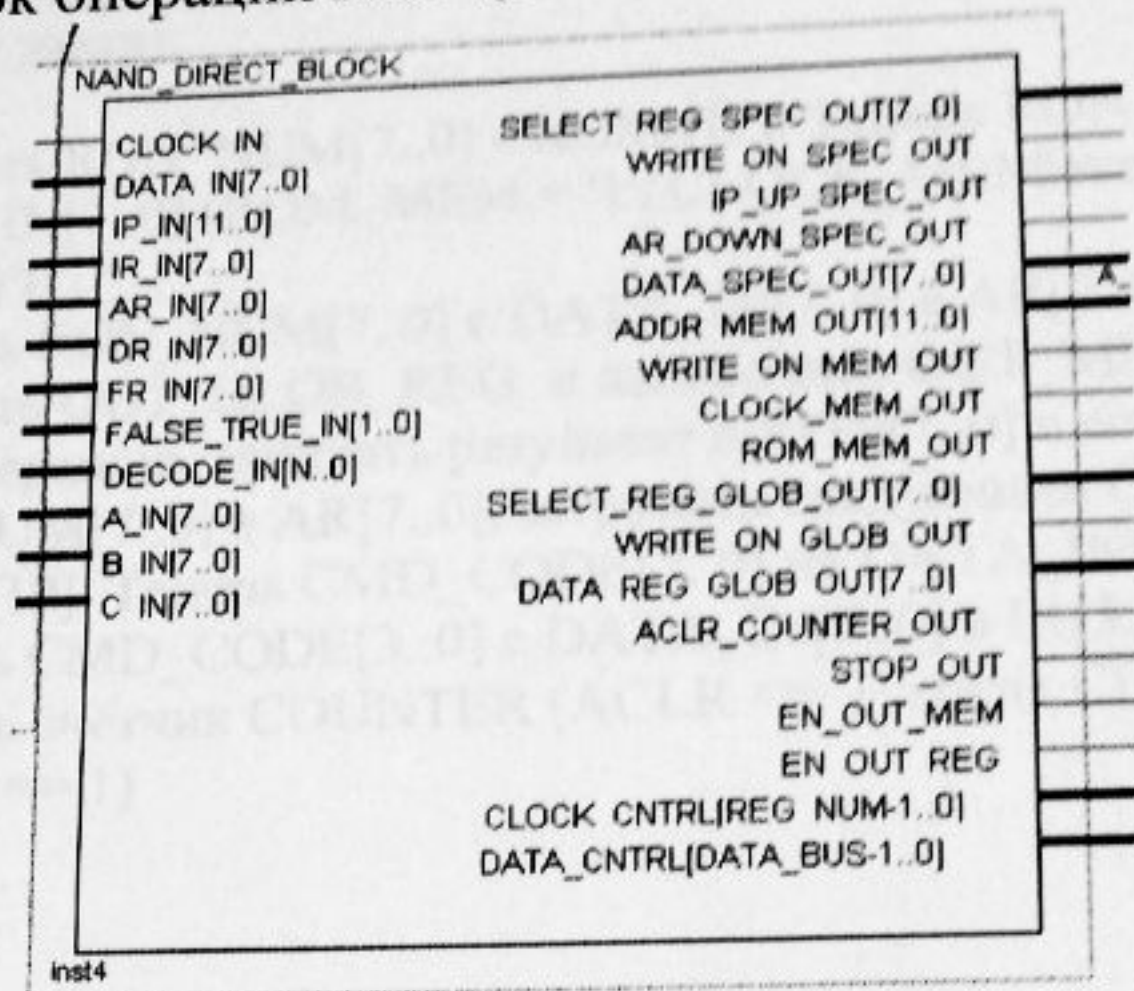


Рисунок 2.39 - Символическое представление блока NAND_DIRECT_BLOCK

Схема блока NAND_DIRECT_BLOCK представлена в приложении Р.

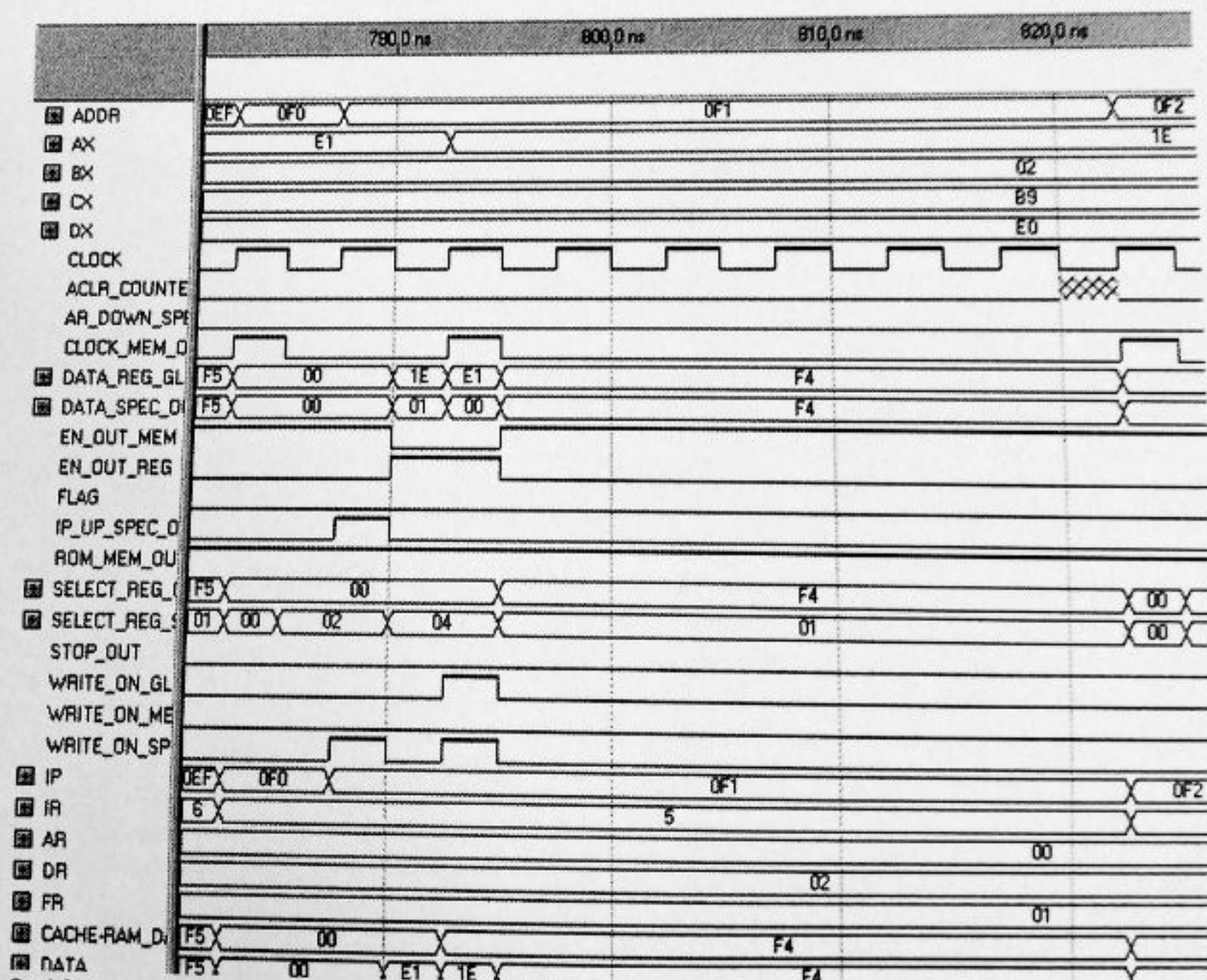


Рисунок 2.40 - Временная диаграмма функционального моделирования работы блока NAND_DIRECT_BLOCK

Итерации операции NOT (номер итерации соответствует номеру пункта и номеру такта):

1. загрузить RON_NUM[7..0] с ROM:[IP]. Теперь RON_NUM[7..0] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. записать RON_NUM[7..0] с DATA_IN[7..0] в AR[7..0], IP_UP
3. включить OUT_GLOB_REG и выключить OUT_MEM, выполнить NOT операцию, записать результат в RON[7..0] в соответствии с RON_NUM[7..0] в AR[7..0], загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь CMD_CODE[7..0] на DATA_IN[7..0]
4. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сбросить счётчик COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

2.5.7 Блок операции AND (побитовое «И»)

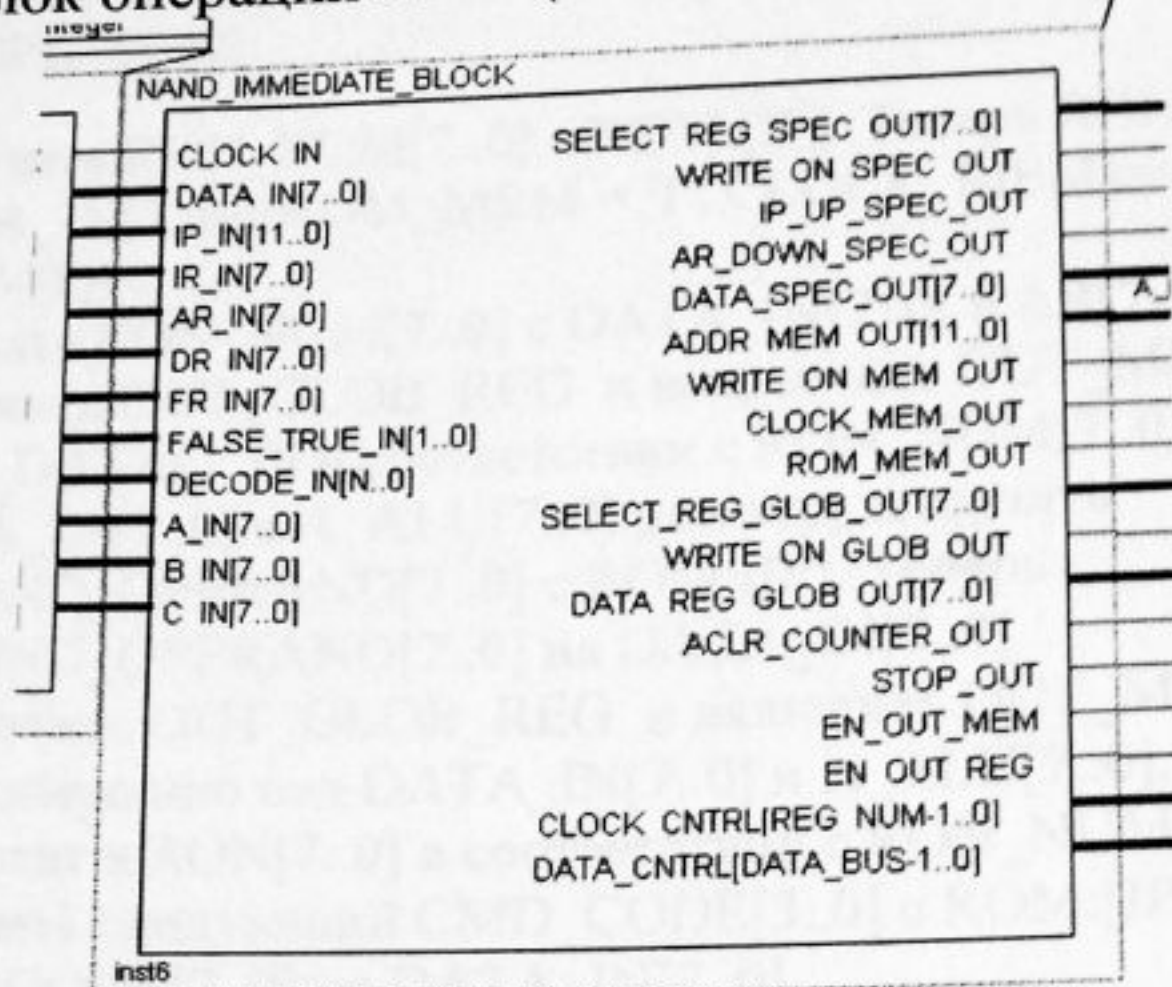


Рисунок 2.41 - Символическое представление блока NAND_IMMEDIATE_BLOCK

Схема блока NAND_IMMEDIATE_BLOCK представлена в приложении С.

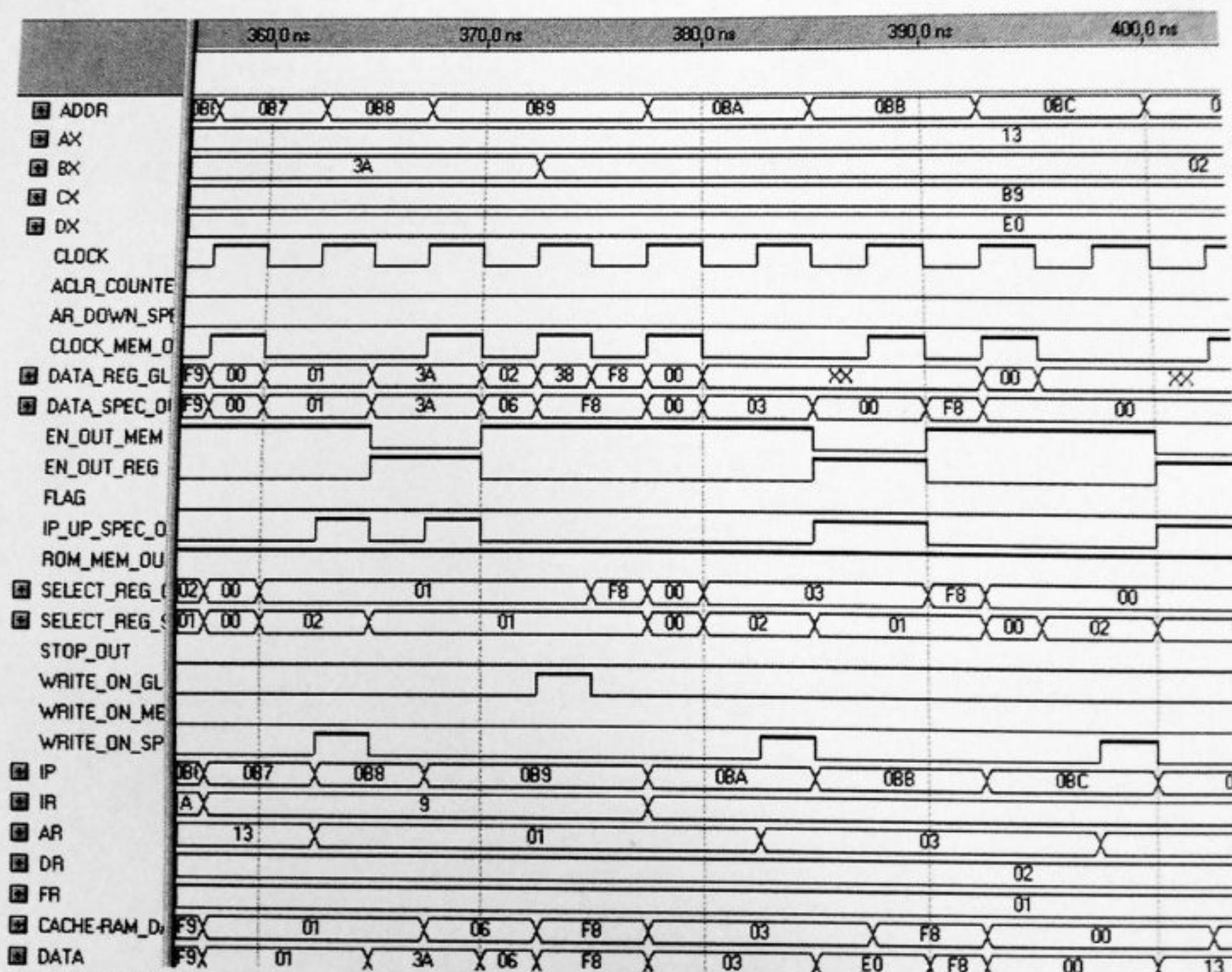
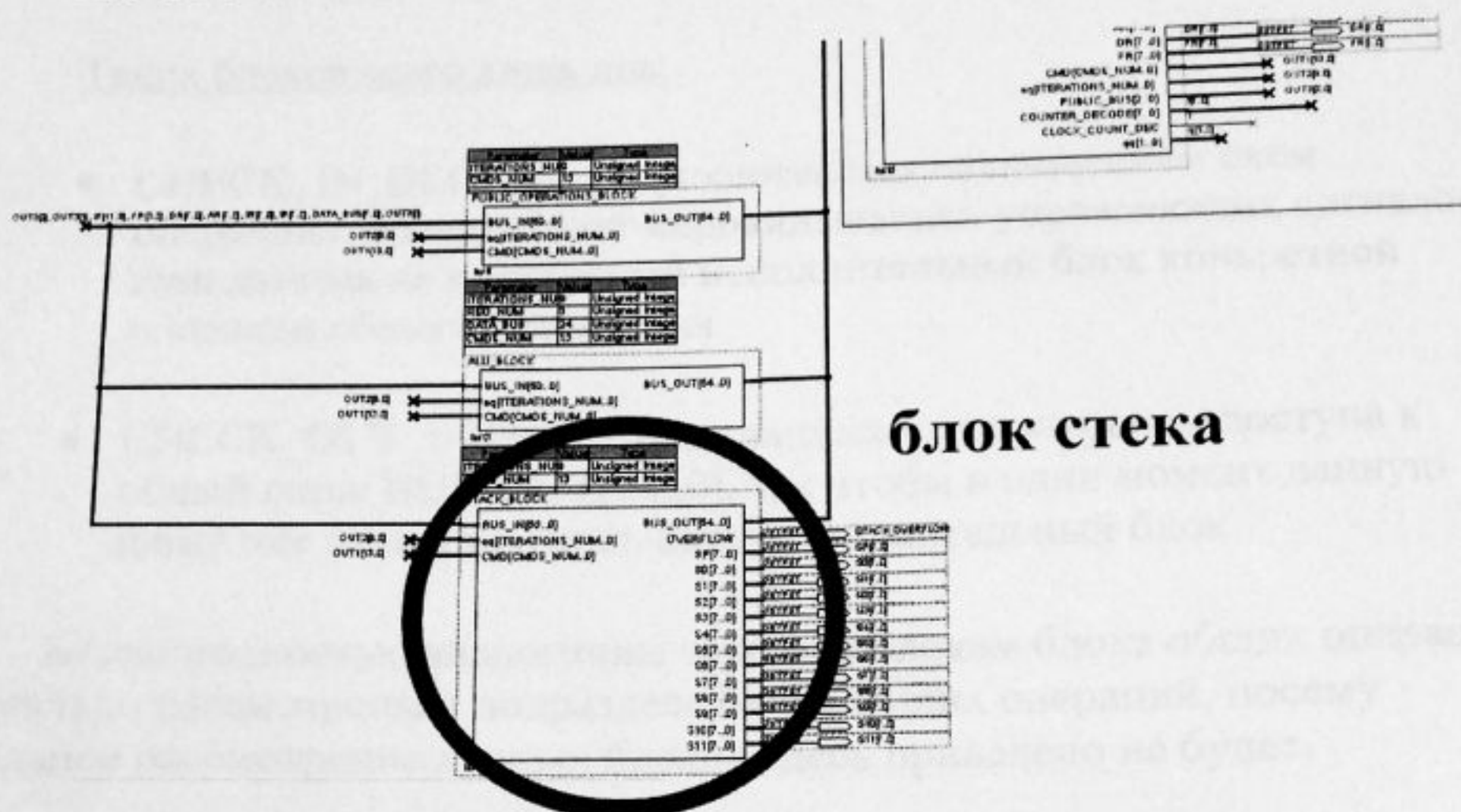


Рисунок 2.42 - Временная диаграмма функционального моделирования работы блока NAND_IMMEDIATE_BLOCK

Итерации операции AND (номер итерации соответствует номеру пункта и номеру такта):

1. загрузить RON_NUM[7..0] с ROM:[IP]. Теперь RON_NUM[7..0] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. записать RON_NUM[7..0] с DATA_IN[7..0] в AR[7..0], IP_UP
3. включить OUT_GLOB_REG и выключить OUT_MEM, записать RON_DATA[7..0] в соответствии с RON_NUM[7..0] в AR[7..0] с DATA_IN[7..0] в A_ALU[7..0] регистр, загрузить SECOND_OPERAND[7..0] с ROM:[IP]. Теперь SECOND_OPERAND[7..0] на DATA_IN[7..0]
5. выключить OUT_GLOB_REG и включить OUT_MEM, выполнить AND операцию над DATA_IN[7..0] и A_ALU[7..0], записать результат в RON[7..0] в соответствии с RON_NUM[7..0] в AR[7..0], загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь CMD_CODE[7..0] на DATA_IN[7..0]
4. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сбросить счётчик COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

2.6 Стековое запоминающее устройство



блок стека

Рисунок 2.43 - Расположение блока стекового устройства в корне проекта

Схема блока арифметико-логического устройства представлена в приложении Т.

Состав блока и назначение основных элементов схемы:

- блоки **CHECK_IN_BLOCK**, **CHECK_OUT_BLOCK** – обеспечение арбитража к общей шине **BUS_OUT[54..0]**
- **STACK_MAIN** – исполнительный блок стека

Состав пинов исполнительного блока стека и их назначение аналогичны составу пинов и их назначению исполнительных блоков общих операций.

2.6.1 Блоки арбитража к общим шинам

Таких блоков всего лишь два:

- CHECK_IN_BLOCK – блок занимается «включением схем операций», обеспечивает «прокидывание» управляющих сигналов и шин данных на конкретный исполнительный блок конкретной операции общего назначения
- CHECK_OUT_BLOCK – блок занимается контролем доступа к общей шине BUS_OUT[54..0], так чтобы в один момент данную шину мог захватить лишь один исполнительный блок

Блоки полностью аналогичны таковым блокам блока общих операций и детально рассмотрены в подразделе блока общих операций, поэтому детальное рассмотрение данных блоков здесь приведено не будет.

2.6.2 Блок схемы стека

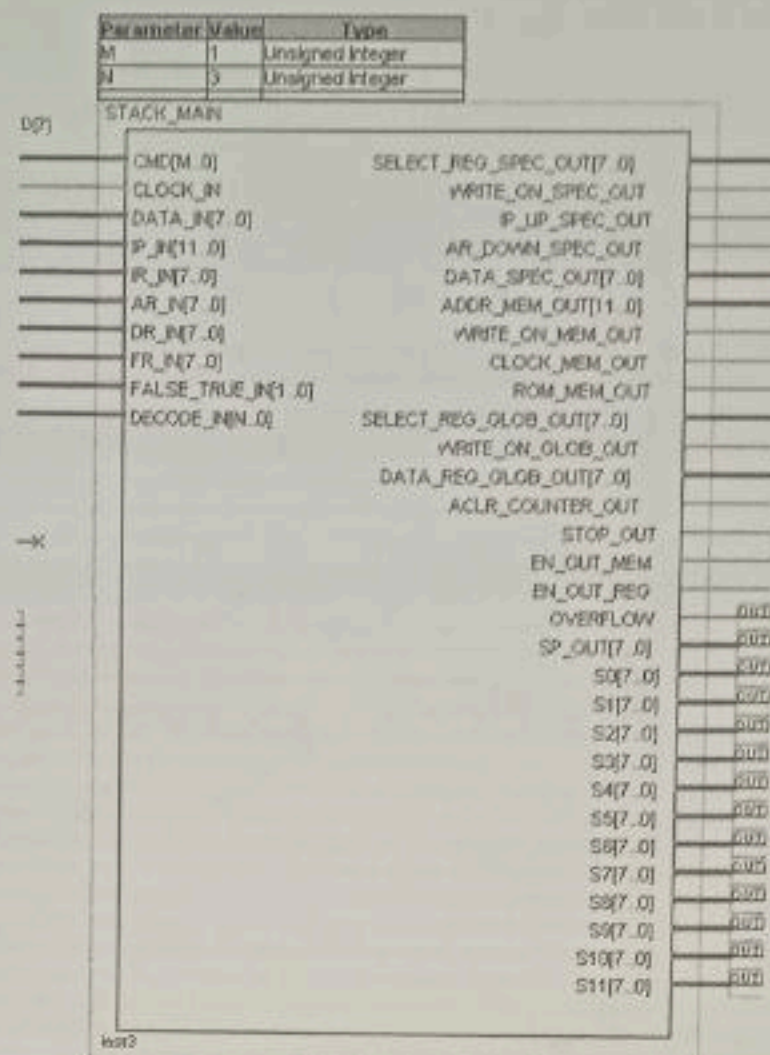


Рисунок 2.44 - Символическое представление исполнительного блока стека

Схема блока исполнительного блока стека представлена в приложении У.

Основной состав схемы исполнительного блока стека:

- блоки `lpm_dff0` – регистры, реализующие память стека, т.к. стек регистровый (реализован на регистрах)
- `lpm_dff1` – регистр флага переполнения стека (`OVERFLOW_BIT`)
- `lpm_counter` – счётчик, реализующий регистр указателя SP (stack pointer) стека
- `lpm_decode6` – блок декодирования содержимого счётчика стека
- остальные элементы – логика управления стеком

Стек имеет контроль переполнения: если стековая память исчерпана, то флаг `OVERFLOW` становится активным ("1"), при последующих операциях `PUSH` запись в стек производиться просто не будет, так же как и регистр `SP` не изменит своего значения. Флаг `OVERFLOW` становится неактивным сразу как только освобождается одна ячейка памяти стекового ЗУ.

В ситуации, когда стек пуст, но выполняется операция `POP`, операция будет выполнена успешно и будет извлечено значение самого верхнего элемента памяти стекового ЗУ, что бы там не находилось.

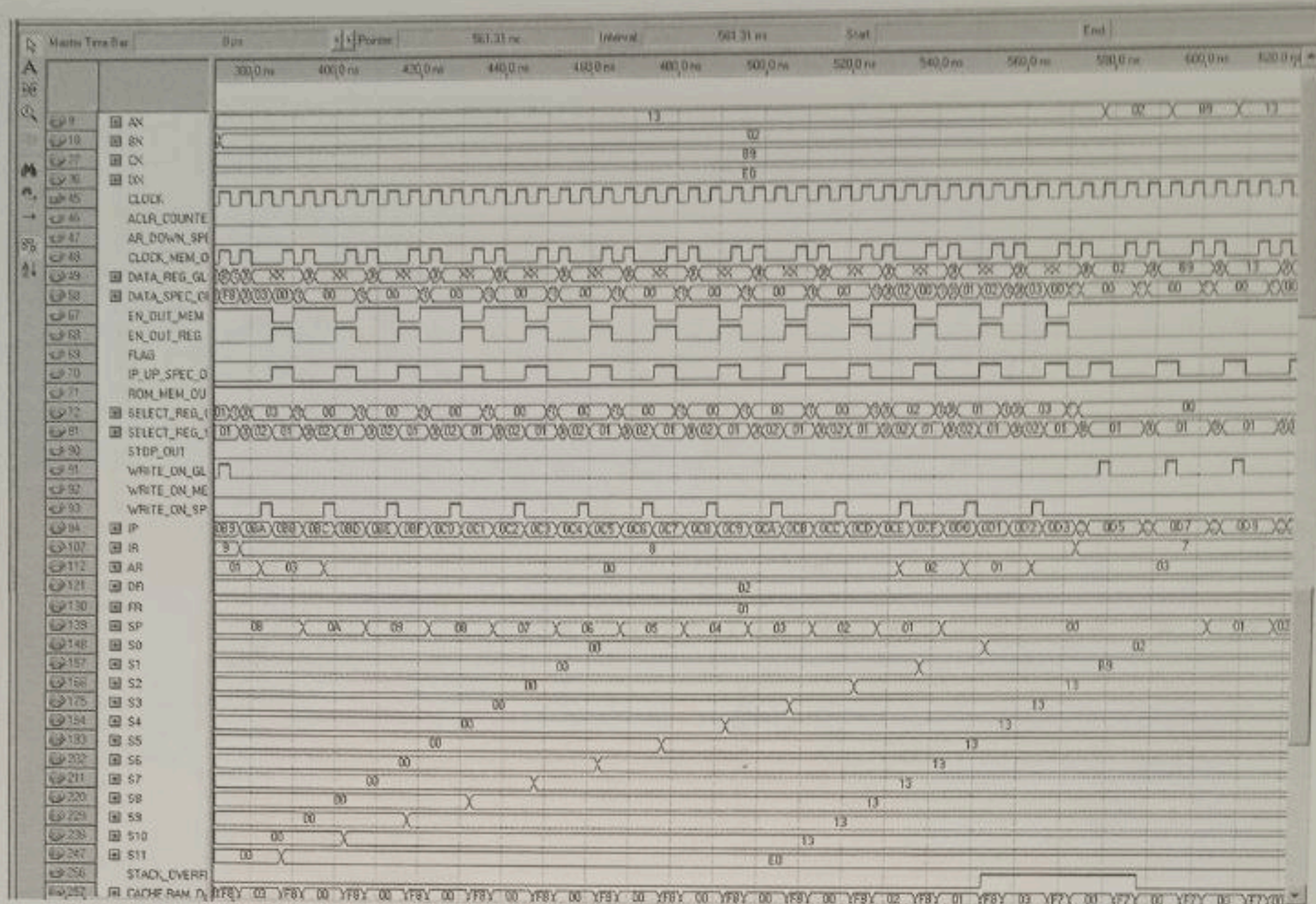


Рисунок 2.45 - Временная диаграмма функционального моделирования заполнения стекового ЗУ

Итерации операции PUSH (поместить в стек), номер пункта соответствует номеру итерации и такта:

1. загрузить REGISTER_RON_NUM[7..0] с ROM:[IP]. Теперь REGISTER_RON_NUM[7..0] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0'))
2. записать RON REGISTER_NUMBER[7..0] с DATA_IN[7..0] в AR[7..0]. Если REGISTER_RON_NUM[7..0] с DATA_IN[7..0] > '19' то записать '00' в AR[7..0]
3. загрузить DATA[7..0] с GLOBAL_REGISTERS в соответствии с REGISTER_RON_NUM[7..0]. Теперь DATA[7..0] на DATA_IN[7..0]. Затем сохранить DATA_IN[7..0] в STACK:[SP]. IP_UP. Перезагрузить OVERFLOW_BIT, загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь CMD_CODE[3..0] на DATA_IN[7..0], но заблокирован, т.к. EN_OUT_MEM == '0'
4. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сбросить счётчик COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1)

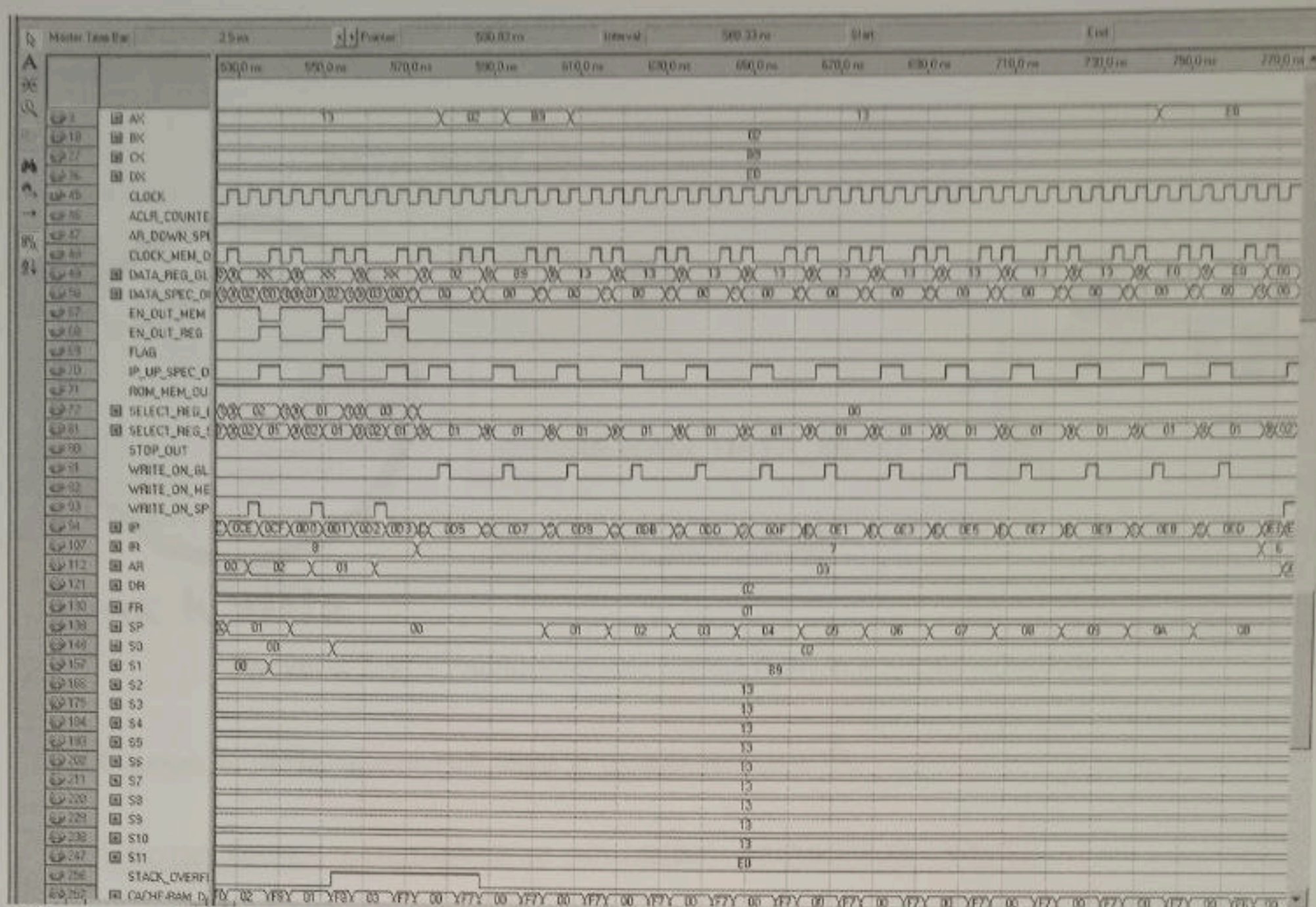
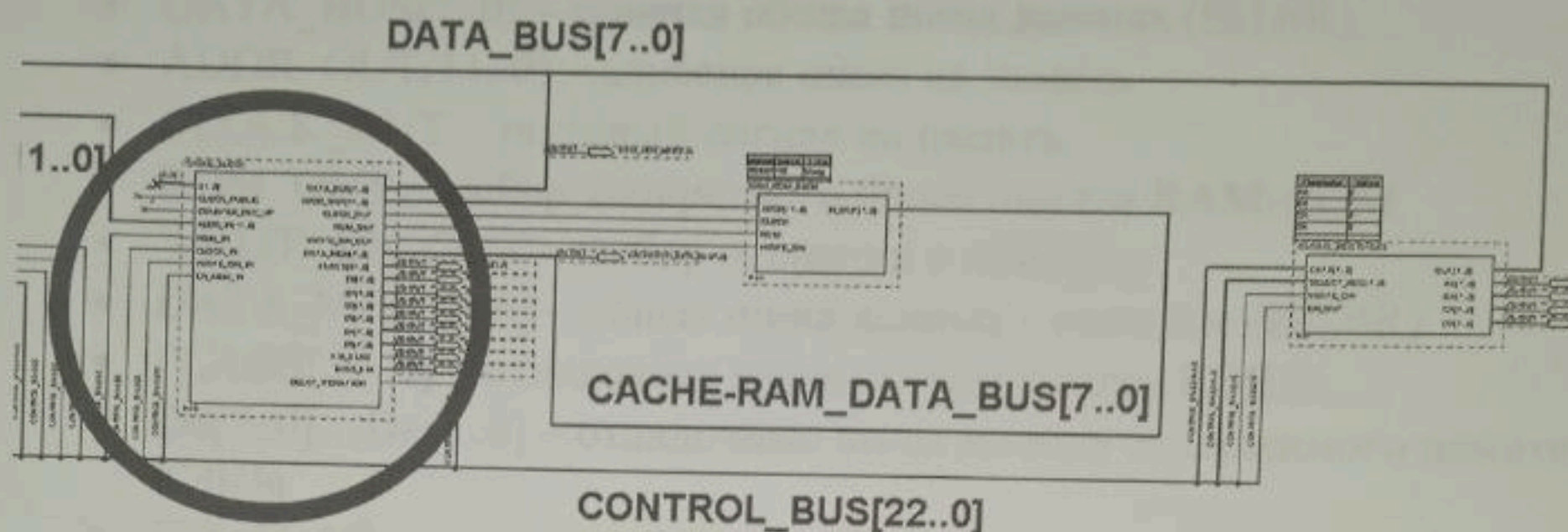


Рисунок 2.46 - Временная диаграмма функционального моделирования извлечения из стекового ЗУ

Итерации операции POP (извлечь из стека), номер пункта соответствует номеру итерации и такта:

1. загрузить REGISTER_RON_NUM[7..0] с ROM:[IP]. Теперь REGISTER_RON_NUM[7..0] на DATA_IN[7..0] (ROM_MEM = '1', CLOCK_MEM'event == CLOCK ('0'-'1'-'0')) Если REGISTER_RON_NUM[7..0] с DATA_IN[7..0] > '19' то REGISTER_RON_NUM[7..0] с DATA_IN[7..0] изменяем на '00' (REGISTER_RON_NUM[7..0] == '00')
2. записать DATA[7..0] с STACK:[SP] в GLOBAL_REGISTER в соответствии с REGISTER_RON_NUM[7..0] в SELECT_GLOB_REG[7..0], IP_UP.
3. загрузить следующий CMD_CODE[3..0] с ROM:[IP]. Теперь CMD_CODE[3..0] на DATA_IN[7..0]
4. записать CMD_CODE[3..0] с DATA_IN[7..0] в IR[3..0], IP_UP, сбросить счётчик COUNTER (ACLR <= '1' когда CLOCK'event и CLOCK == 1) перезагрузить OVERFLOW_BIT

2.7 КЭШ-память



блок КЭШа

Рисунок 2.47 - Расположение блока КЭШа в корне проекта

Схема КЭШа представлена в приложении Ф.

Состав блока и назначение основных элементов схемы:

- CACHE_MAIN – блок данных КЭШа
- CACHE_CONTROL – блок управления КЭШа
- FLAG_BLOCK – блок контроля флагов КЭШа
- SAVE_BLOCK – блок сохранения состояния выводов блока данных КЭШа
- COUNTER – блок счётчика итераций КЭШа
- MUX_BLOCK – блок контроля за задержкой устройства управления
- блоки BUSMUX и прочие элементы схемы - обеспечение арбитража к общим шинам RAM/ROM.

Далее опишем состав и назначение пинов блока. Входные пины блока КЭШа и их назначение:

- CLOCK_PUBLIC – тактовый сигнал всей схемы микро-ЭВМ
- COUNTER_DEC_UP – тактовый сигнал контроля выполнения задержки блока УУ
- ADDR_IN[11..0] – главная общая адресная шина
- ROM_IN – выбор конкретного блока памяти ROM-RAM
- CLOCK_IN – тактовый сигнал на память
- WRITE_ON_IN – выбор операции с памятью
- EN_MEM_IN – контроль захвата блоком КЭШа главной общей шины данных

Выходные (и BIDIR) пины блока КЭШа и их назначение:

- DATA_BUS[7..0] – главная общая шина данных (BIDIR)
- ADDR_OUT[11..0] – адресная шина на память
- CLOCK_OUT – тактовый сигнал на память
- ROM_OUT – выбор конкретного блока памяти RAM-ROM
- WRITE_ON_OUT – выбор операции с памятью
- DATA_MEM[7..0] – общая шина данных с памятью (BIDIR)
- FLAGS[31..0] – отладочная шина вывода флагов КЭШа
- D0[7..0]...Dx[7..0] – отладочные пины вывода содержимого памяти КЭШа
- FIX_FLAG – состояние флага текущего way КЭШа
- MISS_FIX – состояние попадания-промаха последней операции КЭШа
- DELAY_ITERATION – контроль приостановки выполнения итерации устройства управления и исполнительных блоков

Все обращения исполнительных блоков и блока УУ к памяти осуществляются только через блок КЭШа, прямого доступа к памяти эти блоки не имеют.

2.7.1 Блок данных КЭШа

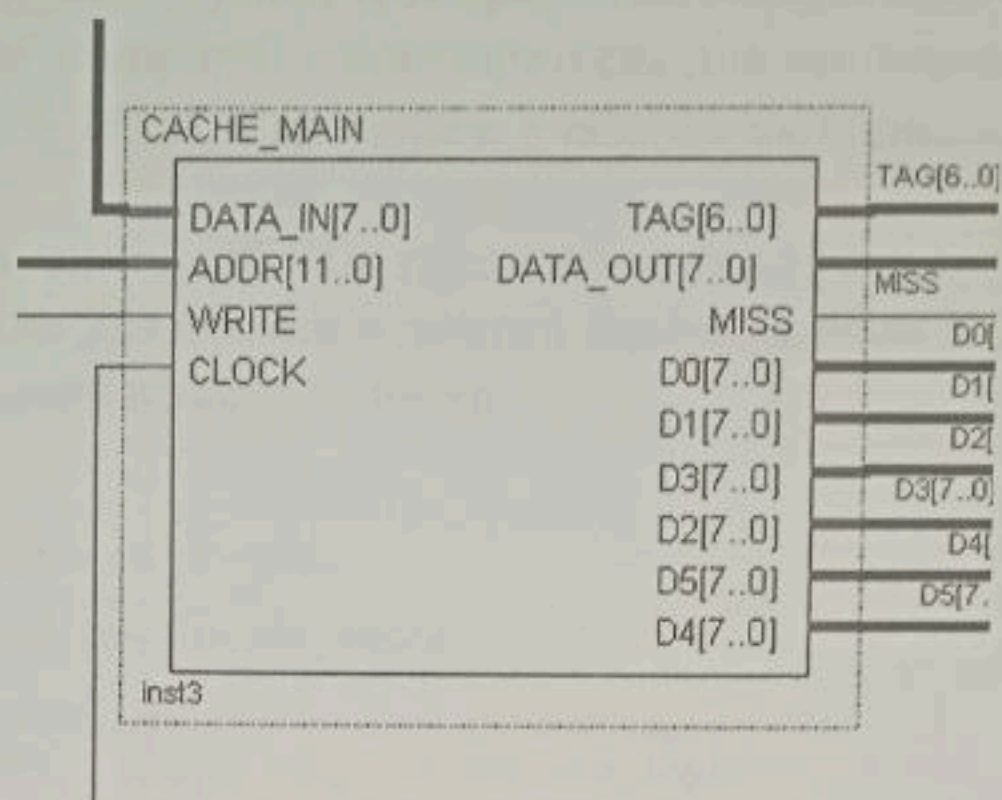


Рисунок 2.48 - Символическое представление блока данных КЭШа

Входные пины блока данных КЭШа и их назначение:

- DATA_IN[7..0] – входная шина данных
- ADDR_DEC[11..0] - адресная шина
- WRITE - выбор операции
- CLOCK – тактовый сигнал блока данных КЭШа

Выходные пины блока данных КЭШа и их назначение:

- TAG[6..0] – шина вывода тэга
- DATA_OUT[7..0] – шина вывода содержимого ячейки КЭШа
- MISS – пин, показывающий результат выполнения последней операции чтения из блока данных КЭШа
- D0[7..0]...Dx[7..0] – отладочные пины вывода содержимого ways блока данных КЭШа

Состав и назначение основных элементов схемы:

- lpm_dff7 – регистр хранения входного адреса на протяжении выполнения всей заданной операции
- lpm_decode1 – декодирование номера строки КЭШа
- INV_BLOCK – блок задержки для записи в память блока данных КЭШа
- lpm_counter8 – счётчик way КЭШа, обеспечивает «псевдо-random» выбора way строки КЭШа
- lpm_compare1 – компаратор для контроля выхода счётчика way за пределы области его значения

- блоки `lmp_dff3` – регистры хранения тэга одного way КЭШа
- блоки `lmp_dff0` – регистры хранения содержимого КЭШа
- блоки `lmp_compare0` – компараторы для проверки на попадание тэга
- блоки `BUS_MUX` – арбитраж к общим выходным шина

Схема блока данных КЭШа представлена в приложении X.

Блок задержки для записи в память блока данных КЭШа - `INV_BLOCK` написан на VHDL, код приведён ниже:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity INV_BLOCK is    -- INV_BLOCK
    port (
        CLOCK, WR      : in std_logic;
        INV             : out std_logic
    );
end INV_BLOCK ;

architecture INV_MAIN of INV_BLOCK is
begin
    process (CLOCK, WR)
    begin
        if(WR='1') then
            if(CLOCK'event and CLOCK='0') then
                INV <= '1';
            end if;
        else
            INV <= '0';
        end if;
    end process;
end INV_MAIN;
```

ADDR[11..0] BITS MEANS:		
[11; 5]	[4; 1]	[0]
TAG	number of row	OFFSET bit

Рисунок 2.49 - Назначение битов адресной шины

Кэш 2-way ассоциативный, в каждой строке 2 way блоков, в каждом блоке 2 ячейки КЭШа, offset указывает смещение в way. Всего строк КЭШа – $2^4 = 16$. Под тег отведено 7 бит из-за достаточно большой адресной шины – 12 бит.

Операции запись и чтение из памяти (синхронизация) – только пакетные и занимают 2 такта, память оперирует целым блоком way КЭШа как при чтении так и при записи. Это значит, что при валидности какой-либо ячейки КЭШа с активным флагом отложенной записи при записи в этот блок

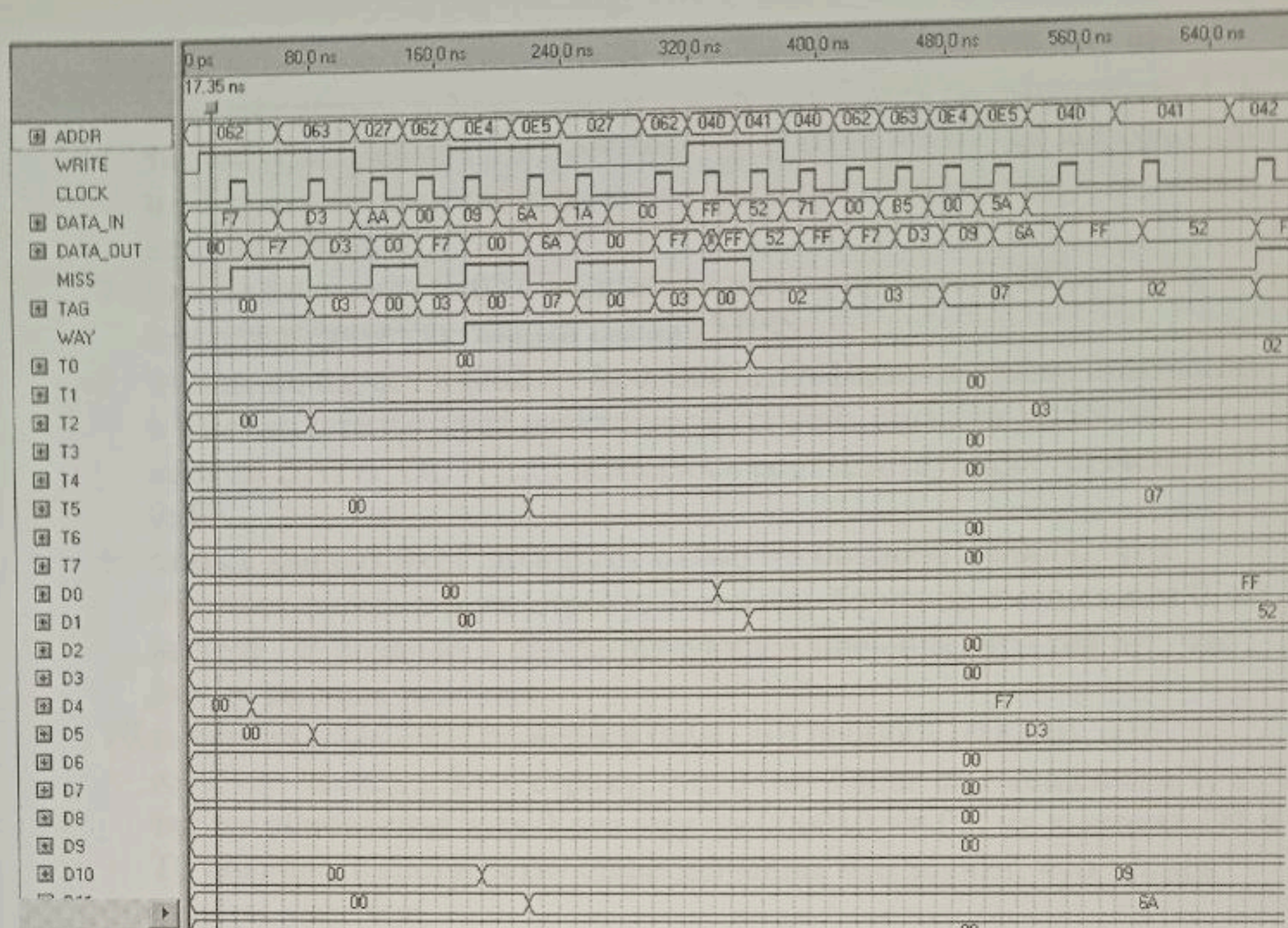


Рисунок 2.50 - Временная диаграмма функционального моделирования работы блока данных КЭШа

Номер пункта соответствует номеру такта:

1. запись данных $0xF7$ по адресу $0x062 = 0b000001100010$, что означает запись в 1-ую строку, 0-ого way блока со смещением 0, на выходе D4 появилось соответствующее значение, что значит, что запись прошла успешно
2. запись данных $0xD3$ по адресу $0x063 = 0b000001100011$, что означает запись в 1-ую строку, 0-ого way блока со смещением 1, а так же обновление тэга 0-ого way $= 0000011 = 03$, на выходах D5 и T2 появилось соответствующее значение, что значит, что запись прошла успешно
3. чтение по адресу $0x027$, на выходе имеем промах КЭШа, о чём свидетельствует активный сигнал MISS
4. чтение по адресу $0x062 = 0b000001100010$, произошло попадание, о чём свидетельствует неактивный уровень сигнала MISS; на выходе $DATA_OUT[7..0]$ появилось соответствующее значение – $0xF7$
5. запись данных $0x09$ по адресу $0x0E4 = 0b000011100100$, что означает запись в 2-ую строку, 1-ого way блока со смещением 0, на выходе D10 появилось соответствующее значение, что значит, что запись прошла успешно

6. запись данных 0x6A по адресу 0x063 == 0b000011100101, что означает запись в 2-ую строку, 1-ого way блока со смещением 1, а так же обновление тэга 1-ого way == 000111 == 07, на выходах D11 и T5 появилось соответствующее значение, что значит, что запись прошла успешно
7. чтение по адресу 0x027, на выходе имеем промах КЭШа, о чём свидетельствует активный сигнал MISS
8. чтение по адресу 0x062 == 0b000001100010, произошло попадание, о чём свидетельствует неактивный уровень сигнала MISS; на выходе DATA_OUT[7..0] появилось соответствующее значение – 0xF7
9. запись данных 0xFF по адресу 0x040 == 0b000001000000, что означает запись в 0-ую строку, 0-ого way блока со смещением 0, на выходе D0 появилось соответствующее значение, что значит, что запись прошла успешно
10. запись данных 0x52 по адресу 0x041 == 0b000001000001, что означает запись в 0-ую строку, 0-ого way блока со смещением 1, а так же обновление тэга 0-ого way == 000010 == 02, на выходах D1 и T0 появилось соответствующее значение, что значит, что запись прошла успешно
11. чтение по адресу 0x040 == 0b000001000000, произошло попадание, о чём свидетельствует неактивный уровень сигнала MISS; на выходе DATA_OUT[7..0] появилось соответствующее значение – 0xFF
12. чтение по адресу 0x062 == 0b000001100010, произошло попадание, о чём свидетельствует неактивный уровень сигнала MISS; на выходе DATA_OUT[7..0] появилось соответствующее значение – 0xF7
13. чтение по адресу 0x063 == 0b000001100011, произошло попадание, о чём свидетельствует неактивный уровень сигнала MISS; на выходе DATA_OUT[7..0] появилось соответствующее значение – 0xD3

2.7.2 Второстепенные блоки КЭШа

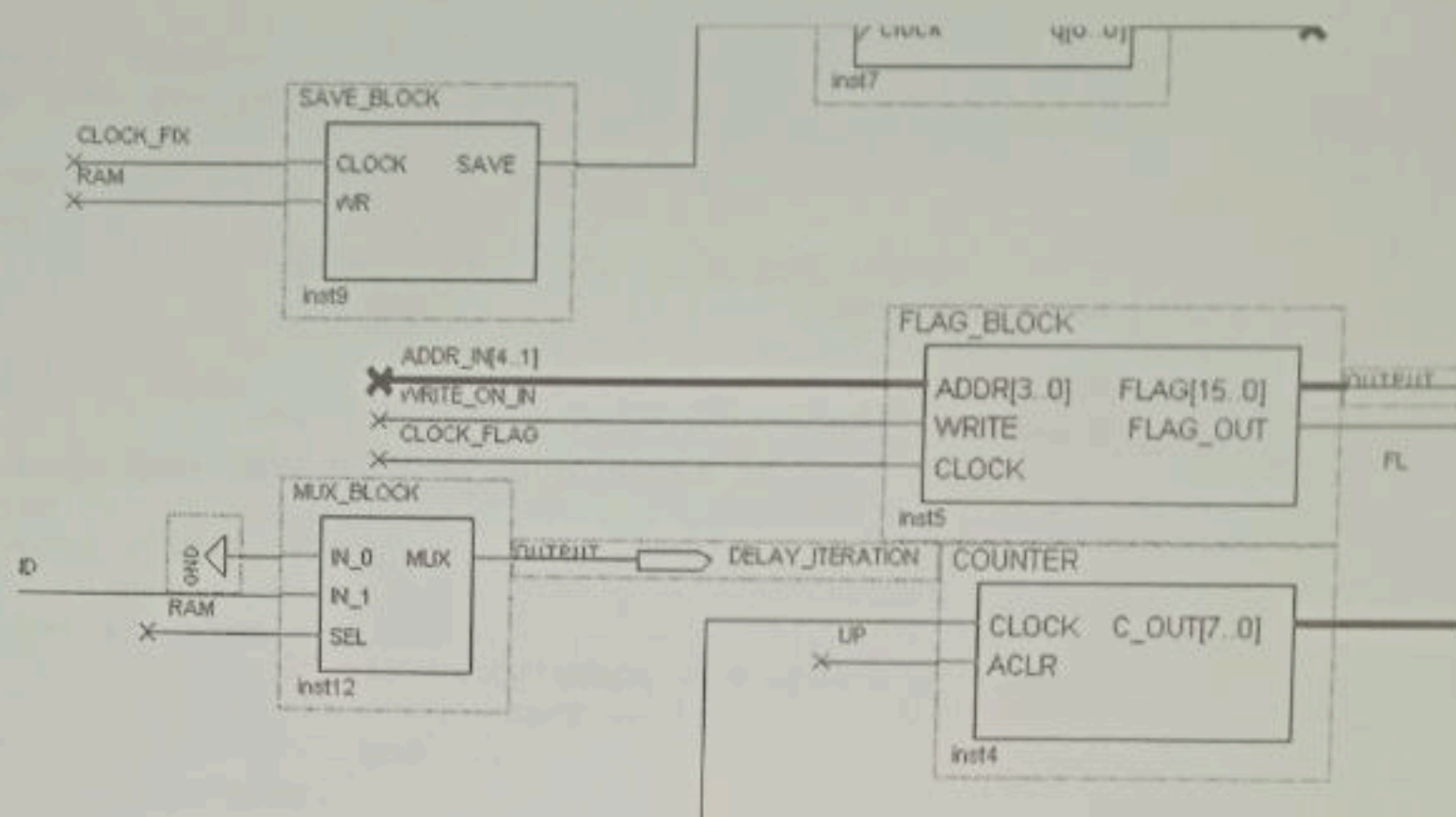


Рисунок 2.51 - Символическое представление второстепенных блоков КЭШа

Перечислим эти блоки:

- SAVE_BLOCK – блок сохранения состояния выводов блока данных КЭШа
- FLAG_BLOCK – блок контроля флагов КЭШа
- COUNTER – блок счётчика итераций КЭШа
- MUX_BLOCK – блок контроля за задержкой устройства управления

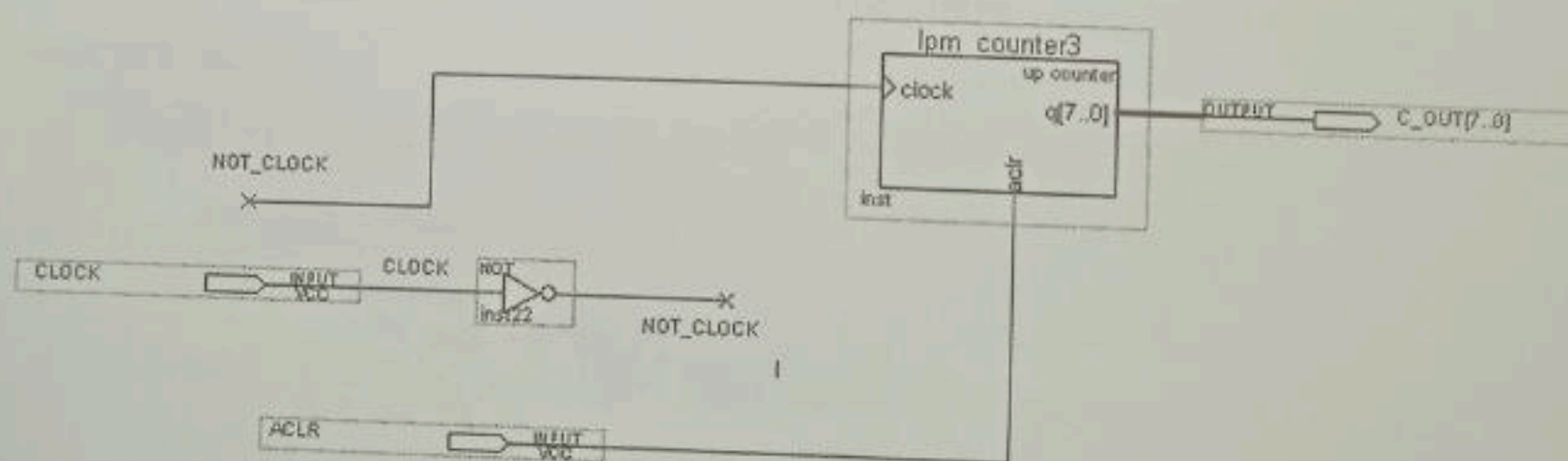


Рисунок 2.52 - Схема блока COUNTER

Блок сохранения состояния выводов блока данных КЭШа -
SAVE_BLOCK написан на VHDL, код приведён ниже:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity SAVE_BLOCK is  -- SAVE_BLOCK
port (
    CLOCK, WR          : in std_logic;
    SAVE                : out std_logic
);
end SAVE_BLOCK ;

architecture SAVE_MAIN of SAVE_BLOCK is
begin
    process (CLOCK, WR)
    begin
        if(WR='1') then
            if(CLOCK'event and CLOCK='0') then
                SAVE <= '1';
            end if;
        else
            SAVE <= '0';
        end if;
    end process;
end SAVE_MAIN;
```

Схема блока FLAG_BLOCK КЭШа представлена в приложении Ц.

Блок контроля за задержкой устройства управления - MUX_BLOCK
написан на VHDL, код приведён ниже:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity MUX_BLOCK is  -- MUX_BLOCK
port (
    IN_0, IN_1, SEL    : in std_logic;
    MUX                 : out std_logic
);
end MUX_BLOCK ;

architecture MUX_MAIN of MUX_BLOCK is
begin
    process (SEL)
    begin
        if(SEL='1') then
            MUX <= IN_1;
        else
            MUX <= IN_0;
        end if;
    end process;
end MUX_MAIN;
```


2.7.3 Блок управления КЭШа

Это основной блок КЭШа, являющийся инициатором всех итераций конкретной заданной операции в зависимости от состояния КЭШа.

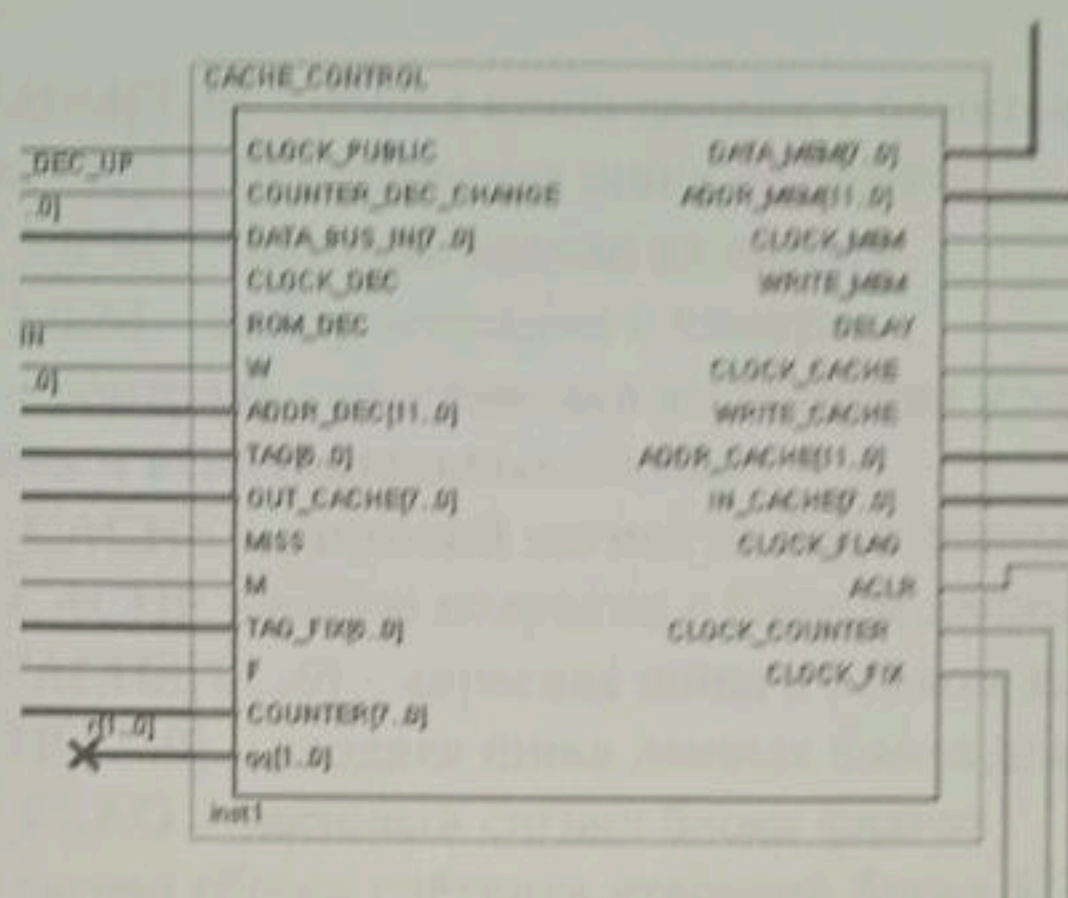


Рисунок 2.53 - Символическое представление управляющего блока КЭШа

Далее опишем состав и назначение пинов блока. Входные пины блока управления КЭШа и их назначение:

- CLOCK_PUBLIC – тактовый сигнал всей схемы микро-ЭВМ
- COUNTER_DEC_CHANGE – тактовый сигнал контроля выполнения задержки блока УУ
- DATA_BUS_IN[7..0] – главная общая шина данных
- CLOCK_DEC – тактовый сигнал блока УУ
- ROM_DEC – выбор конкретного блока памяти ROM-RAM сигнал с УУ
- W - выбор операции с памятью
- ADDR_DEC[11..0] - главная общая адресная шина
- TAG[6..0] – шина тэга с блока данных КЭШа
- OUT_CACHE[7..0] – шина данных с блока КЭШа
- MISS – результат последней операции с блоком данных КЭШа (попадание-промах)
- M – зафиксированный сигнал MISS на всё последующее выполнение заданной операции
- TAG_FIX[6..0] – зафиксированная шина TAG[6..0] на всё последующее выполнение заданной операции
- F – зафиксированный сигнал FLAG на всё последующее выполнение заданной операции
- COUNTER[7..0] – значение счётчика итераций блока КЭШа

- qq[1..0] – значение контролирующего счётчика задержки блока УУ и исполнительных блоков.

Выходные (и BIDIR) пины управляющего блока КЭШа и их назначение:

- DATA_MEM[7..0] – общая шина данных с памятью (BIDIR)
- ADDR_MEM[11..0] – адресная шина на память
- CLOCK_MEM – тактовый сигнал на память
- WRITE_MEM – выбор операции с памятью
- DELAY – контроль приостановки выполнения итерации устройства управления и исполнительных блоков
- CLOCK_CACHE – тактовый сигнал на блок данных КЭШа
- WRITE_CACHE – выбор операции с блоком данных КЭШа
- ADDR_CACHE[11..0] – адресная шина с блоком данных КЭШа
- IN_CACHE[7..0] – входная шина данных блока данных КЭШа
- CLOCK_FLAG – тактовый сигнал блока флагов
- ACLR – сигнал сброса счётчика итераций блока КЭШа
- CLOCK_COUNTER – тактовый сигнал счётчику итераций блока КЭШа
- CLOCK_FIX – тактовый сигнал блоку SAVE_BLOCK – сохранения выходного состояния пинов блока данных КЭШа на протяжении выполнения всей заданной операции КЭШа

Управляющий блок КЭШа написан на VHDL, код приведён ниже:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity CACHE_CONTROL is    -- CACHE_CONTROL
  port (
    -- nodes and buses for RAM_ROM_BIDIR
    CLOCK_PUBLIC      : in std_logic;
    COUNTER_DEC_CHANGE : in std_logic;
    DATA_BUS_IN      : in std_logic_vector(7 downto 0);
    CLOCK_DEC, ROM_DEC, W : in std_logic;
    ADDR_DEC          : in std_logic_vector(11 downto 0);
    DATA_MEM         : inout std_logic_vector(7 downto 0);
    ADDR_MEM          : out std_logic_vector(11 downto 0);
    CLOCK_MEM, WRITE_MEM, DELAY : out std_logic;
    -- nodes and buses for CACHE_MAIN
    TAG                : in std_logic_vector(6 downto 0);
    OUT_CACHE          : in std_logic_vector(7 downto 0);
    MISS, M             : in std_logic;
    TAG_FIX            : in std_logic_vector(6 downto 0);
    CLOCK_CACHE, WRITE_CACHE : out std_logic;
    ADDR_CACHE         : out std_logic_vector(11 downto 0);
    IN_CACHE           : out std_logic_vector(7 downto 0);
    -- nodes and buses for FLAG_BLOCK
```



```

F                                : in std_logic;

CLOCK_FLAG                      : out std_logic;
                                -- nodes and buses for COUNTER
COUNTER                        : in std_logic_vector(7 downto 0);
ACLR, CLOCK_COUNTER : out std_logic;
                                -- nodes and buses for other
CLOCK_FIX                      : out std_logic;
qq                             : in std_logic_vector(1 downto 0)
);
end CACHE_CONTROL ;

architecture CACHE_ARCH of CACHE_CONTROL is
begin
  process (COUNTER, ROM_DEC, COUNTER_DEC_CHANGE, CLOCK_PUBLIC)
  begin
    DATA_MEM <= "ZZZZZZZZ";
    if(ROM_DEC='0') then

case COUNTER is
  when "00000000" => -- 0
    if (qq="00") then
      DELAY <= '1';
      ACLR <= '0';
      ADDR_CACHE <= ADDR_DEC;
      WRITE_CACHE <= '0';
      CLOCK_CACHE <= CLOCK_DEC;
      CLOCK_FLAG <= '0';
      CLOCK_COUNTER <= CLOCK_DEC;
      CLOCK_FIX <= CLOCK_DEC;
    end if;
  when "00000001" => -- 1
    if(W='1' and (M='1' or (M='1' and F='0')))) then
      DELAY <= '0'; -- delete
      ACLR <= '0';

      ADDR_CACHE <= ADDR_DEC;
      WRITE_CACHE <= '1';
      IN_CACHE <= DATA_BUS_IN;
      CLOCK_FLAG <= '1';
      CLOCK_CACHE <= not(CLOCK_DEC);
    end if;
    if(M='1' and F='1') then
      DELAY <= '1';
      ACLR <= '0';
      ADDR_MEM(11 downto 5) <= TAG(6 downto 0);
      ADDR_MEM(4 downto 0) <= ADDR_DEC(4 downto 0);
      DATA_MEM <= OUT_CACHE;
      WRITE_MEM <= '1';
      WRITE_CACHE <= '0';
      CLOCK_CACHE <= '0';
      CLOCK_MEM <= CLOCK_DEC;
    end if;
    if(M='0' and W='0') then
      CLOCK_CACHE <= '0';
      DELAY <= '0';
      ACLR <= '1';
    end if;
    if(W='0' and F='0' and M='1') then
      CLOCK_CACHE <= '0';
      DELAY <= '1';
      ACLR <= '0';
      ADDR_MEM(11 downto 1) <= ADDR_DEC(11 downto 1);
      ADDR_MEM(0) <= not(ADDR_DEC(0));
      WRITE_MEM <= '0';
      CLOCK_MEM <= CLOCK_DEC;
    end if;
    CLOCK_FIX <= '0';
    CLOCK_COUNTER <= CLOCK_DEC;
  end case;
end process;

```



```

when "00000010" => -- 2
    if(W='1' and (M='0' or (M='1' and F='0')) then
        ADDR_CACHE <= ADDR_DEC;
        WRITE_CACHE <= '1';
        IN_CACHE <= DATA_BUS_IN;
        DELAY <= '0';

        ACLR <= '1';

        CLOCK_CACHE <= '0';
        CLOCK_FLAG <= '0';
    end if;
    if(W='1' and M='1' and F='1') then
        CLOCK_MEM <= '0';
        DELAY <= '1';
        ACLR <= '0';
        ADDR_CACHE(0) <= not(ADDR_DEC(0));
        ADDR_CACHE(11 downto 1) <= ADDR_DEC(11 downto 1);
        WRITE_CACHE <= '0';
        CLOCK_CACHE <= CLOCK_DEC;
    end if;
    if((W='0' and M='1')) then
        CLOCK_MEM <= '0';
        DELAY <= '1';
        ACLR <= '0';
        ADDR_CACHE(11 downto 1) <= ADDR_DEC(11 downto 1);
        ADDR_CACHE(0) <= not(ADDR_DEC(0));
        if(F='0') then
            IN_CACHE <= DATA_MEM;
            WRITE_CACHE <= '1';
        else
            WRITE_CACHE <= '0';
        end if;
        CLOCK_CACHE <= not(CLOCK_DEC);
    end if;
    if(M='0' and W='0') then
        CLOCK_CACHE <= '0';
        DELAY <= '0';
        ACLR <= '1';
    end if;
    CLOCK_COUNTER <= CLOCK_DEC;
when "00000011" => -- 3
    DELAY <= '1';
    ACLR <= '0';

    if(F='1') then
        ADDR_MEM(0) <= not(ADDR_DEC(0));
        ADDR_MEM(11 downto 5) <= TAG(6 downto 0);
        ADDR_MEM(4 downto 1) <= ADDR_DEC(4 downto 1);
        DATA_MEM <= OUT_CACHE;
        WRITE_MEM <= '1';
        CLOCK_CACHE <= '0';
        WRITE_CACHE <= '0';
    else
        ADDR_CACHE <= ADDR_DEC;
        IN_CACHE <= DATA_MEM;
        WRITE_CACHE <= '1';
        CLOCK_CACHE <= not(CLOCK_DEC);
        ADDR_MEM(11 downto 1) <= ADDR_DEC(11 downto 1);
        ADDR_MEM(0) <= ADDR_DEC(0);
        WRITE_MEM <= '0';
    end if;
    CLOCK_MEM <= CLOCK_DEC;
    CLOCK_COUNTER <= CLOCK_DEC;
when "00000100" => -- 4
    ACLR <= '0';

    if(F='1' and M='1') then

```



```

    DELAY                <= '1';
    --ADDR_MEM            <= ADDR_DEC;
    if(W='1') then

        ADDR_CACHE        <= ADDR_DEC;
        IN_CACHE           <= DATA_BUS_IN;
        WRITE_CACHE        <= '1';
        CLOCK_CACHE        <= CLOCK_PUBLIC;

    else

        ADDR_CACHE(0) <= not(ADDR_DEC(0));
        CLOCK_CACHE    <= '0';
        ADDR_MEM(11 downto 1) <= ADDR_DEC(11 downto 1);
        ADDR_MEM(0) <= not(ADDR_DEC(0));
        WRITE_MEM      <= '0';
        IN_CACHE        <= DATA_MEM;
        CLOCK_MEM        <= CLOCK_DEC;
        WRITE_CACHE      <= '0';

    end if;
end if;
if(W='0' and M='1' and F='0') then
    CLOCK_CACHE    <= (CLOCK_DEC);
    ADDR_CACHE     <= ADDR_DEC;
    DELAY          <= '0'; -- delete
    WRITE_CACHE     <= '1';
    IN_CACHE        <= DATA_MEM;
end if;
CLOCK_COUNTER <= CLOCK_DEC;
when "00000101" => -- 5
    if(W='1' and F='1' and M='1') then
        DELAY          <= '0';
        ACLR           <= '1';
        ADDR_CACHE(11 downto 1) <= ADDR_DEC(11 downto 1);
        ADDR_CACHE(0) <= not(ADDR_DEC(0));
        IN_CACHE        <= DATA_MEM;
        WRITE_CACHE      <= '1';
        CLOCK_CACHE      <= CLOCK_PUBLIC;
    end if;
    if(W='0' and M='1' and F='0') then
        ADDR_CACHE     <= ADDR_DEC;
        CLOCK_CACHE     <= '0';
        CLOCK_MEM       <= '0';
        DELAY          <= '0';
        ACLR           <= '1';
    end if;
    if(W='0' and M='1' and F='1') then
        ADDR_CACHE(0) <= not(ADDR_DEC(0));
        DELAY          <= '1';
        ACLR           <= '0';
        ADDR_CACHE(11 downto 1) <= ADDR_DEC(11 downto 1);
        IN_CACHE        <= DATA_MEM;
        WRITE_CACHE     <= '1';
        CLOCK_CACHE     <= not(CLOCK_DEC);
    end if;
    CLOCK_COUNTER <= CLOCK_DEC;
when "00000110" => -- 6
    if(W='1') then
        DELAY          <= '0';
        ACLR           <= '1';
        CLOCK_CACHE     <= '0';
        CLOCK_MEM       <= '0';
        DATA_MEM       <= "ZZZZZZZZ";
    else
        DELAY          <= '1';
        ACLR           <= '0';
        ADDR_CACHE     <= ADDR_DEC;
        IN_CACHE        <= DATA_MEM;
        WRITE_CACHE     <= '1';
        CLOCK_CACHE     <= '0';
        ADDR_MEM        <= ADDR_DEC;
    end if;

```



```

        WRITE_MEM      <= '0';
        CLOCK_MEM      <= CLOCK_DEC;
    end if;
    CLOCK_COUNTER <= CLOCK_DEC;
when "00000111" => -- 7
    DELAY      <= '0';
    ACLR       <= '1';
    ADDR_CACHE <= ADDR_DEC;
    IN_CACHE   <= DATA_MEM;
    WRITE_CACHE <= '1';
    CLOCK_FLAG <= CLOCK_DEC;
    CLOCK_CACHE <= CLOCK_PUBLIC;
    CLOCK_COUNTER <= CLOCK_DEC;
when "00001000" => -- 8
    ADDR_CACHE <= ADDR_DEC;
    IN_CACHE   <= DATA_MEM;
    WRITE_CACHE <= '0';
    CLOCK_CACHE <= '0';
    CLOCK_MEM   <= '0';
    DELAY       <= '0';

    ACLR       <= '1';

    DATA_MEM   <= "ZZZZZZZZ";
    CLOCK_COUNTER <= CLOCK_DEC;
when others => null;
end case;
--else
--    CLOCK_COUNTER <= COUNTER_DEC_CHANGE;
end if;
end process;
end CACHE_ARCH;

```

Всего существует 6 операций, которые могут быть инициированы управляющим блоком КЭШа:

- 1) чтение и попадание
- 2) чтение и промах, флаг отложенной записи текущего way неактивен
- 3) чтение и промах, флаг отложенной записи текущего way активен
- 4) запись и попадание
- 5) запись и промах, флаг отложенной записи текущего way неактивен
- 6) запись и промах, флаг отложенной записи текущего way активен

Далее опишем детально каждую итерацию каждой из операций (так же содержание каждой итерации хорошо видно по коду VHDL).

1) Чтение, попадание (R && !MISS)

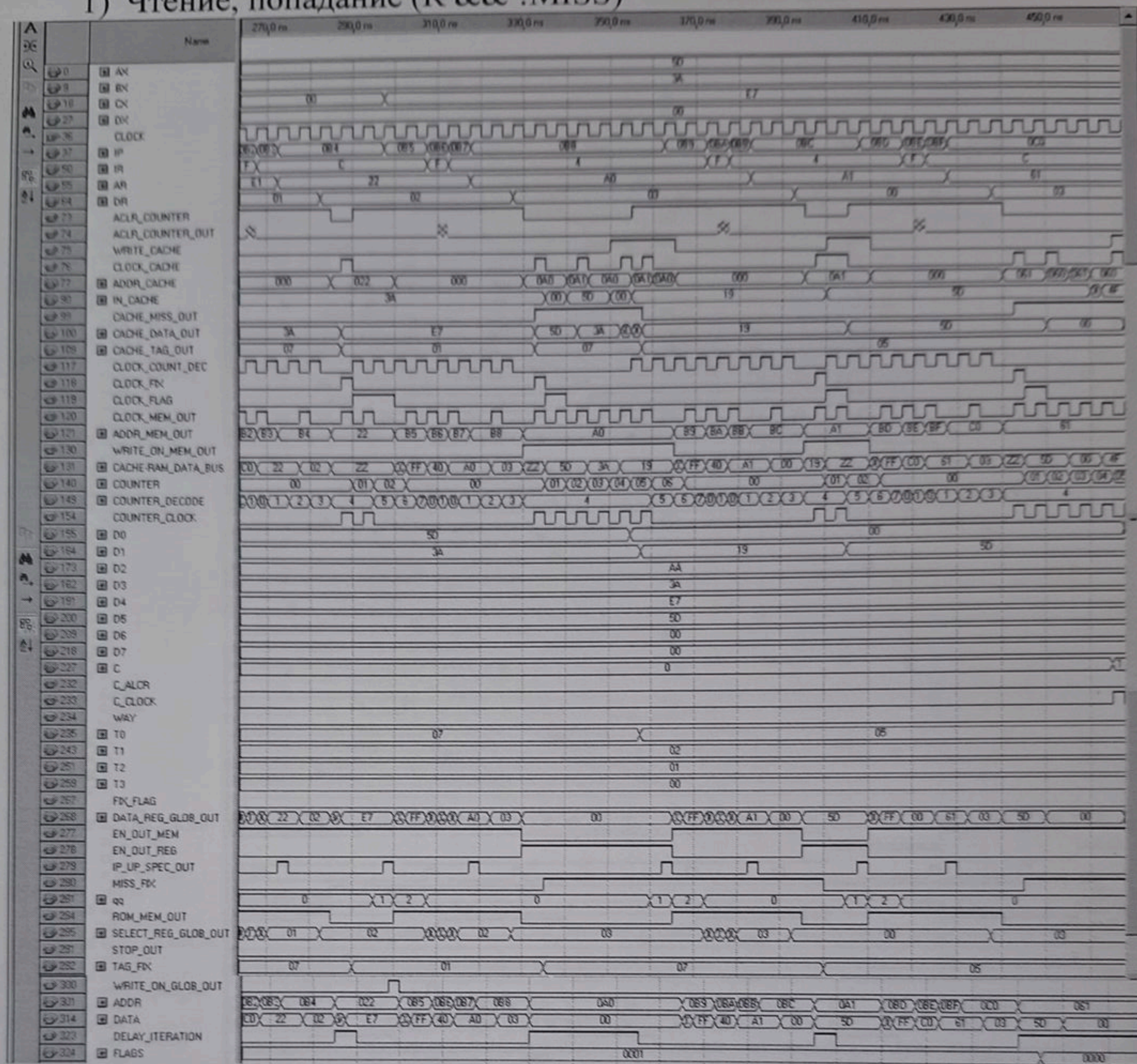


Рисунок 2.54

1. DELAY <= '1' – активный сигнал задержки подаётся на УУ, подача тактового сигнала на блок данных КЭШа, проверка, есть ли запрашиваемые данные в блоке данных КЭШа, фиксация выходных информационных сигналов блока данных КЭШа (CLOCK_FIX <= CLOCK_DEC), наращивание счётчика итераций КЭШа
2. сброс активного сигнала задержки DELAY, передача управления блоку УУ и соответствующему исполнительному блоку, сброс счётчика итераций КЭШа и выдача запрашиваемых данных на общую шину данных.

2) Чтение, промах, флаг неактивен (R && MISS && !F)

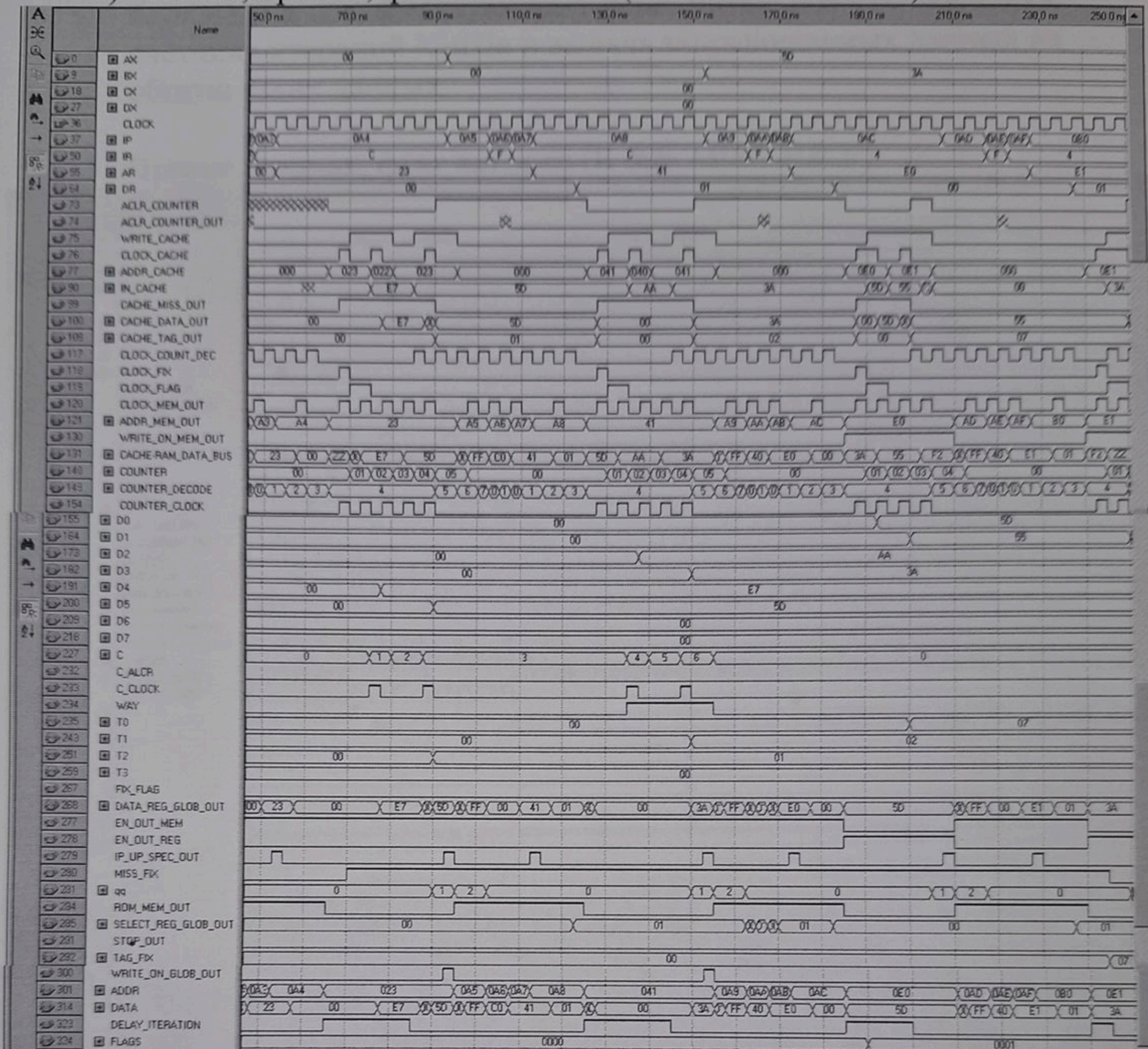
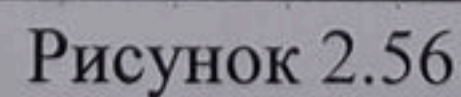


Рисунок 2.55

1. $DELAY \leq '1'$ – активный сигнал задержки подаётся на УУ, подача тактового сигнала на блок данных КЭШа, проверка, есть ли запрашиваемые данные в блоке данных КЭШа, фиксация выходных информационных сигналов блока данных КЭШа ($CLOCK_FIX \leq CLOCK_DEC$), наращивание счётчика итераций КЭШа
2. чтение из памяти запрашиваемых данных, но с инвертированным первым битом адреса – битом offset
3. запись данных с выходной шины данных памяти в соответствующую ячейку блока данных КЭШа
4. чтение из памяти запрашиваемых данных (offset неинвертирован, такой же, как на общей адресной шине)
5. запись данных с выходной шины данных памяти в соответствующую ячейку блока данных КЭШа

- 3) Чтение, промах, флаг активен (R && MISS && F)



1. DELAY <= '1' – активный сигнал задержки подаётся на УУ, подача тактового сигнала на блок данных КЭШа, проверка, есть ли запрашиваемые данные в блоке данных КЭШа, фиксация выходных информационных сигналов блока данных КЭШа (CLOCK_FIX <= CLOCK_DEC), наращивание счётчика итераций КЭШа
2. запись содержимого ячейки КЭШа с выхода блока данных КЭШа по адресу, старшими битами которого является тэг way вычитанного содержимого ячейки блока данных КЭШа
3. вычитывание соседней ячейки way с блока данных КЭШа (адрес тот же, только первый бит – offset инвертирован)
4. запись содержимого ячейки КЭШа с выхода блока данных КЭШа по адресу, старшими битами которого является тэг way вычитанного содержимого ячейки блока данных КЭШа, а первый бит всего адреса – offset инвертирован
5. вычитывание из RAM запрашиваемых данных
6. запись в блок данных КЭШа данных, которые были запрошены ранее с блока памяти (из RAM), вычитывание из RAM запрашиваемых данных по запрашиваемому адресу, но первый бит которого - offset инвертирован
7. запись в блок данных КЭШа данных, которые были запрошены ранее с блока памяти (из RAM) по запрашиваемому адресу, но первый бит которого - offset инвертирован
8. сброс активного сигнала задержки DELAY, передача управления блоку УУ и соответствующему исполнительному блоку, сброс счётчика итераций КЭШа и выдача запрашиваемых данных на общую шину данных.

4) Запись и попадание (!R && !MISS)

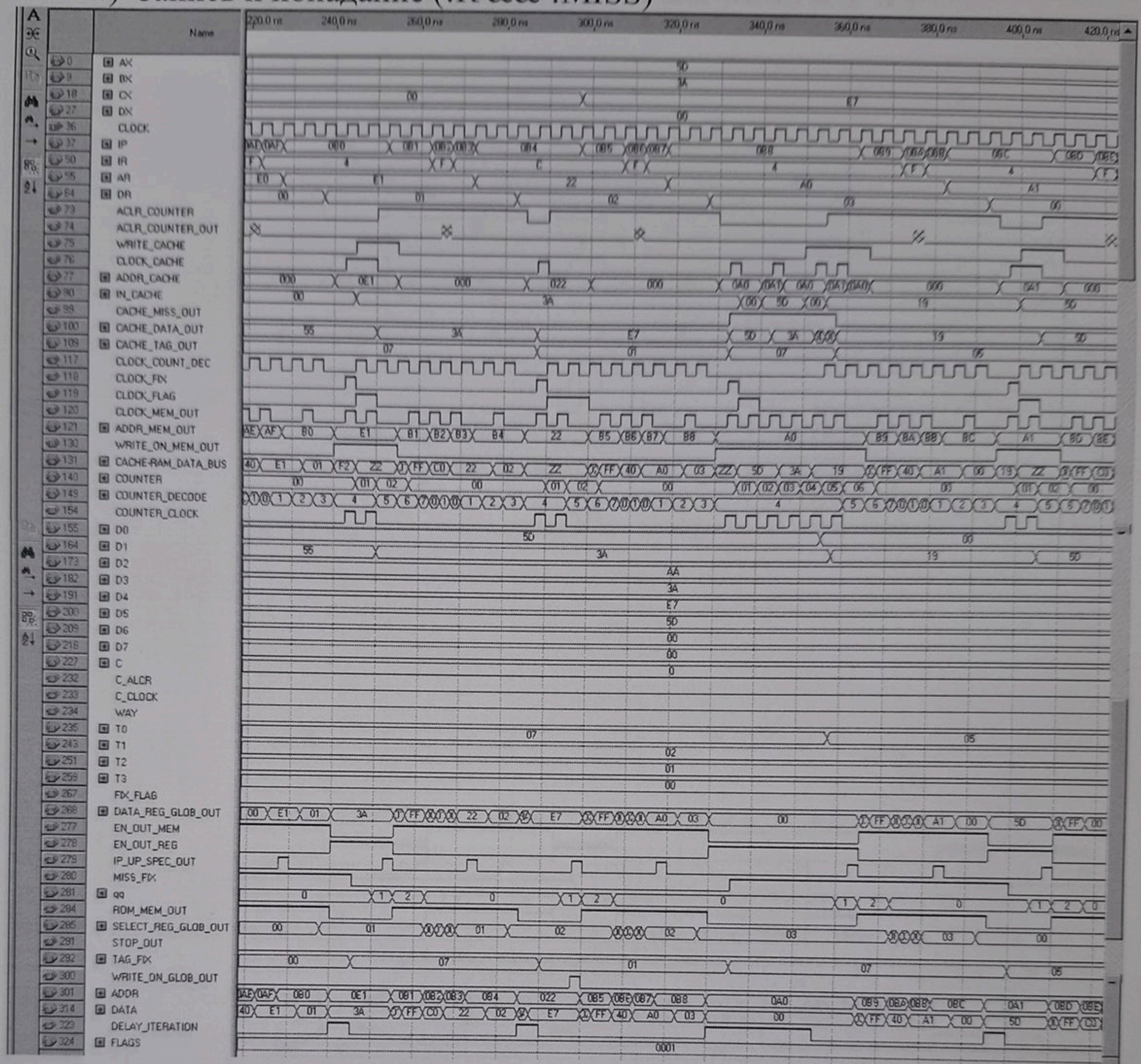


Рисунок 2.57

1. $DELAY \leq '1'$ – активный сигнал задержки подаётся на УУ, подача тактового сигнала на блок данных КЭШа, проверка, есть ли запрашиваемые данные в блоке данных КЭШа, фиксация выходных информационных сигналов блока данных КЭШа ($CLOCK_FIX \leq CLOCK_DEC$), наращивание счётчика итераций КЭШа
2. запись данных с общей шины данных в соответствующую ячейку блока данных КЭШа, обновление состояния флага соответствующего way
3. сброс активного сигнала задержки $DELAY$, передача управления блоку УУ и соответствующему исполнительному блоку, сброс счётчика итераций КЭШа и выдача запрашиваемых данных на общую шину данных

5) Запись, промах, флаг неактивен (!R && MISS && !F)

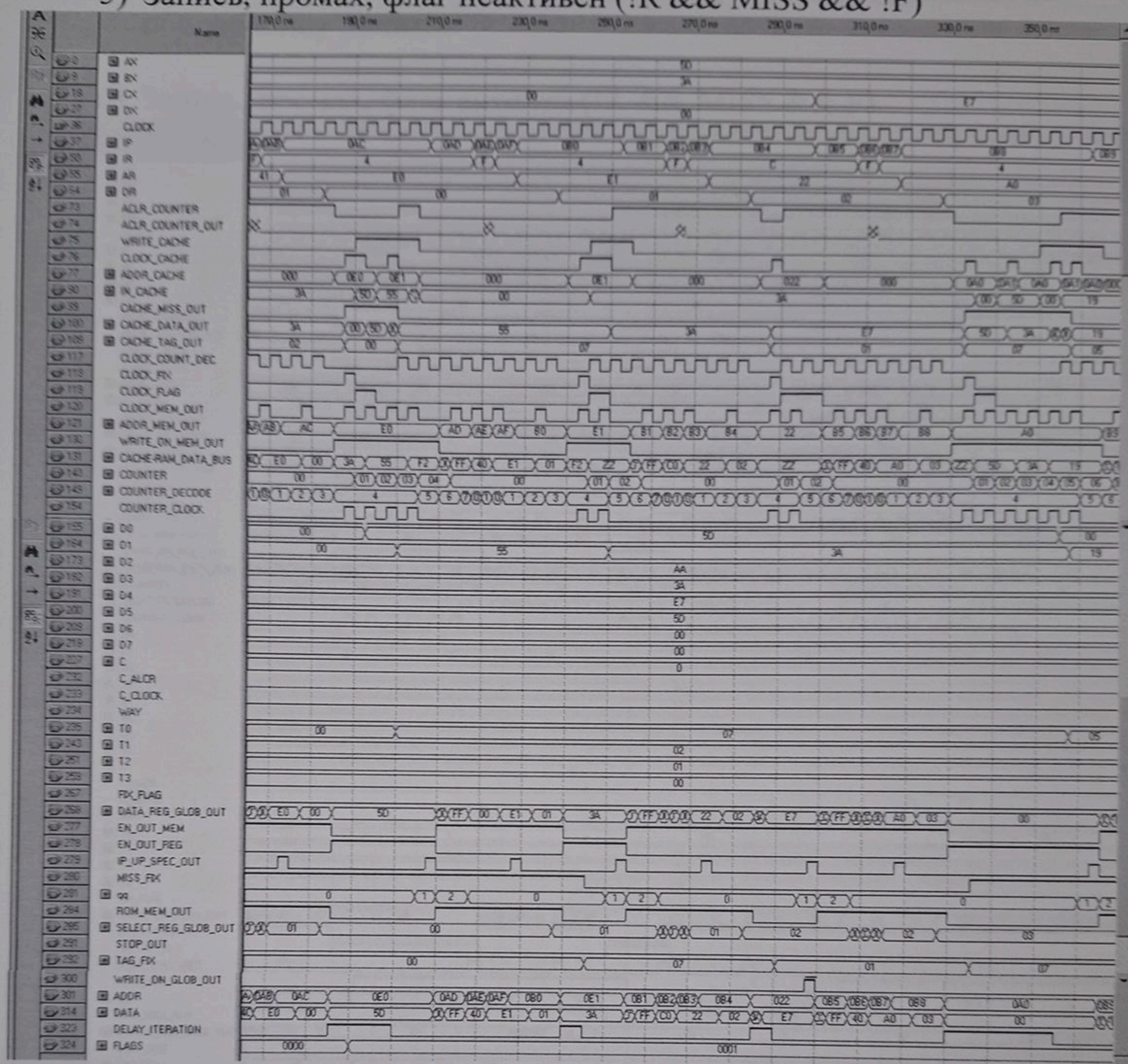
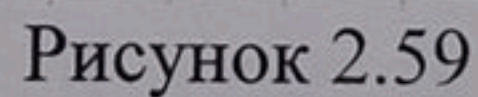


Рисунок 2.58

1. $DELAY \leq '1'$ – активный сигнал задержки подаётся на УУ, подача тактового сигнала на блок данных КЭШа, проверка, есть ли запрашиваемые данные в блоке данных КЭШа, фиксация выходных информационных сигналов блока данных КЭШа ($CLOCK_FIX \leq CLOCK_DEC$), наращивание счётчика итераций КЭШа
2. запись данных с общей шины данных в соответствующую ячейку блока данных КЭШа,
3. чтение данных из памяти RAM по запрашиваемому адресу, но с инвертированным первым битом – offset битом
4. запись данных с шины данных памяти (вычитанные данные с RAM) в соответствующую ячейку блока данных КЭШа (offset инвертирован), обновление состояния флага соответствующего way
5. сброс активного сигнала задержки $DELAY$, передача управления блоку УУ и соответствующему исполнительному блоку, сброс

6) Запись, промах, флаг активен (!R && MISS && F)



1. $DELAY \leq '1'$ – активный сигнал задержки подаётся на УУ, подача тактового сигнала на блок данных КЭШа, проверка, есть ли запрашиваемые данные в блоке данных КЭШа, фиксация выходных информационных сигналов блока данных КЭШа ($CLOCK_FIX \leq CLOCK_DEC$), наращивание счётчика итераций КЭШа
2. запись содержимого ячейки КЭШа с выхода блока данных КЭШа по адресу, старшими битами которого является тэг way вычитанного содержимого ячейки блока данных КЭШа
3. вычитывание соседней ячейки way с блока данных КЭШа (адрес тот же, только первый бит – offset инвертирован)
4. запись содержимого ячейки КЭШа с выхода блока данных КЭШа по адресу, старшими битами которого является тэг way вычитанного содержимого ячейки блока данных КЭШа, а первый бит всего адреса – offset инвертирован
5. запись данных с общей шины данных по запрашиваемому адресу в блок данных КЭШа, вычитывание данных из памяти по запрашиваемому адресу, но первый бит которого инвертирован – offset бит
6. запись в блок данных КЭШа данных, которые были запрошены ранее с блока памяти (из RAM) по запрашиваемому адресу, но первый бит которого - offset инвертирован
7. сброс активного сигнала задержки $DELAY$, передача управления блоку УУ и соответствующему исполнительному блоку, сброс счётчика итераций КЭШа и выдача запрашиваемых данных на общую шину данных.

Код, на котором проходило моделирование КЭШа:

Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+a	+b	+c	+d	+e	+f
00	FF	02	FF	E0	9E	A1	03	F8	01	F8	01	F8	01	F8	01	F8
10	01	F8	01	F8	01	F8	01	F8	01	F8	01	F8	01	F8	01	F8
20	01	F8	01	F8	01	F8	01	FC	AA	F7	02	F7	02	F7	02	F7
30	02	F7	02	F7	02	F7	02	F7	02	F7	02	F7	01	F7	03	00
40	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
70	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
80	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
90	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
a0	01	FF	C0	23	00	FF	C0	41	01	FF	40	E0	00	FF	40	E1
b0	01	FF	C0	22	02	FF	40	A0	03	FF	40	A1	00	FF	C0	61
c0	03	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
d0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
e0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
f0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Рисунок 2.60 - Содержимое ROM (ПЗУ)

Таблица 2.1 - Псевдокод

Адрес	Команда	Пояснения
[0x000]	NOP;	нет операции
[0x001]	NOP;	нет операции
[0x002]	NOP;	нет операции
[0x003]	JMP 0x09E;	переход на [IP+0x09E]
...	...	недостижимый код
[0x0A0]	NOP;	нет операции
[0x0A1]	NOP;	нет операции
[0x0A2]	MOV AX, [0x023]	R, M, !F - 2
[0x0A5]	NOP;	нет операции
[0x0A6]	MOV BX, [0x041]	R, M, !F - 2
[0x0A9]	NOP;	нет операции
	MOV [0x0E0], AX	!R, M, !F - 5
	NOP;	нет операции
	MOV [0x0E1], BX	!R, !M, F - 4
	NOP;	нет операции
	MOV CX, [0x022];	R, !M - 1
	NOP;	нет операции
	MOV [0x0A0], DX	!R, M, F - 6
	NOP;	нет операции
	MOV [0x0A1], AX;	!R, !M, F - 4
	NOP;	нет операции
	MOV DX, [0x061];	R, M, F - 3
...	...	код неважен для теста

Код программы представлен выше (см. таблицу 2.1).

Содержимое ОЗУ (RAM) до и после моделирования:

Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+a	+b	+c	+d	+e	+f
00	02	62	02	67	04	17	06	07	08	09	0A	0B	0C	0D	0E	0F
10	10	11	AA	13	14	15	16	B7	18	19	1A	09	1C	1D	1E	FF
20	D7	61	E7	5D	24	25	26	27	28	29	2A	2B	2C	2D	2E	59
30	30	31	32	6A	34	35	36	37	38	64	3A	3B	3C	3D	3E	3F
40	AA	3A	42	43	44	45	46	47	48	49	6A	4B	FA	4D	3A	A8
50	50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F
60	4F	69	55	10	64	65	66	67	68	69	7C	6B	6C	6D	6E	6F
70	70	37	72	73	74	75	76	8A	78	79	7A	7B	7C	7D	7E	7F
80	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
90	90	91	92	93	94	95	96	97	98	6A	9A	9B	9C	9D	9E	9F
a0	E0	19	6F	A3	A4	A5	A6	A7	A8	A9	E0	81	AC	AD	F3	0E
b0	B0	6A	B2	9F	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF
c0	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF
d0	D0	D1	D2	D3	D4	D5	D6	F1	D8	D9	DA	DB	DC	DD	27	DF
e0	F2	55	CC	5A	E4	E5	E6	E7	E8	E9	EA	EB	4D	ED	EA	EF
f0	A5	F1	F2	F3	F4	F5	F6	F7	4F	F9	EE	FB	6D	FD	08	FF

Рисунок 2.61 -Содержимое RAM (ОЗУ) до моделирования

MAIN_MODULE RAM_ROM_BIDIR:inst1 lpm_ram_io:inst altram:sram altsy																
Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+a	+b	+c	+d	+e	+f
000	02	62	02	67	04	17	06	07	08	09	0A	0B	0C	0D	0E	0F
010	10	11	AA	13	14	15	16	B7	18	19	1A	09	1C	1D	1E	FF
020	D7	61	E7	5D	24	25	26	27	28	29	2A	2B	2C	2D	2E	59
030	30	31	32	6A	34	35	36	37	38	64	3A	3B	3C	3D	3E	3F
040	AA	3A	42	43	44	45	46	47	48	49	6A	4B	FA	4D	3A	A8
050	50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F
060	4F	69	55	10	64	65	66	67	68	69	7C	6B	6C	6D	6E	6F
070	70	37	72	73	74	75	76	8A	78	79	7A	7B	7C	7D	7E	7F
080	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
090	90	91	92	93	94	95	96	97	98	6A	9A	9B	9C	9D	9E	9F
0a0	00	5D	6F	A3	A4	A5	A6	A7	A8	A9	E0	81	AC	AD	F3	0E
0b0	B0	6A	B2	9F	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF
0c0	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF
0d0	D0	D1	D2	D3	D4	D5	D6	F1	D8	D9	DA	DB	DC	DD	27	DF
0e0	5D	3A	CC	5A	E4	E5	E6	E7	E8	E9	EA	EB	4D	ED	EA	EF
0f0	A5	F1	F2	F3	F4	F5	F6	F7	4F	F9	EE	FB	6D	FD	08	FF

Рисунок 2.62 - Содержимое RAM (ОЗУ) после моделирования

2.8 Контроллер прямого доступа к памяти

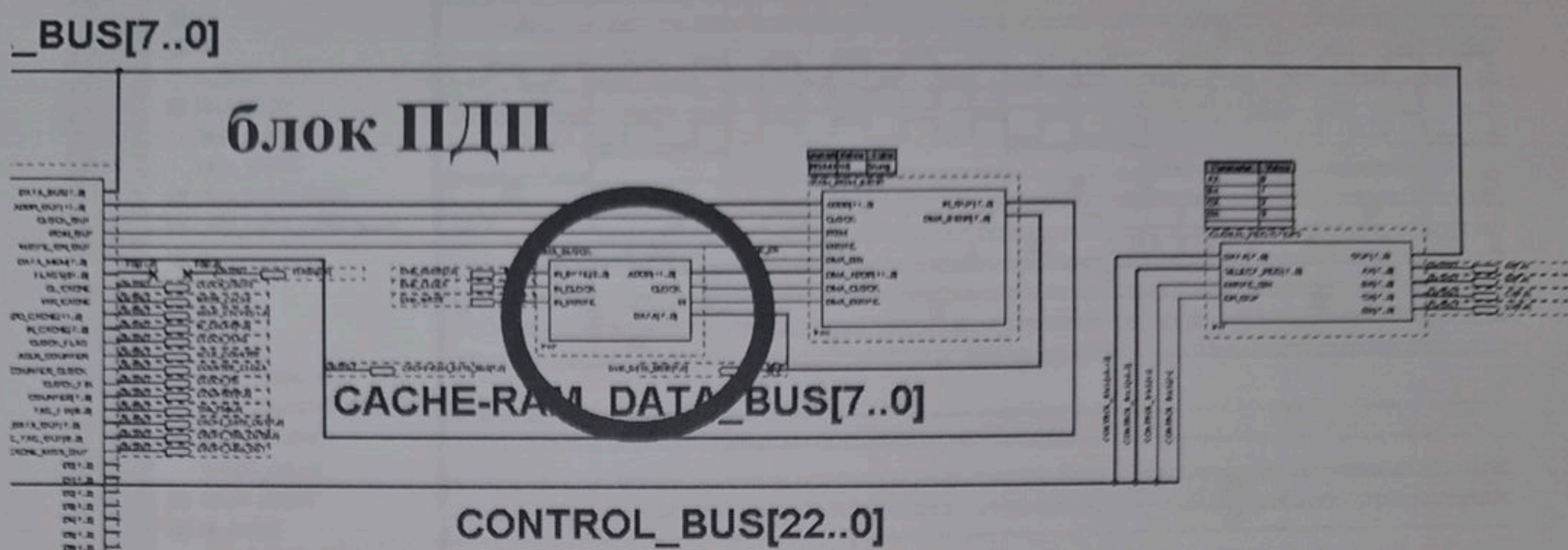


Рисунок 2.63 - Расположение блока PDIP в корне проекта

Блок PDIP предназначен для обеспечения прямого доступа к памяти ПУ.

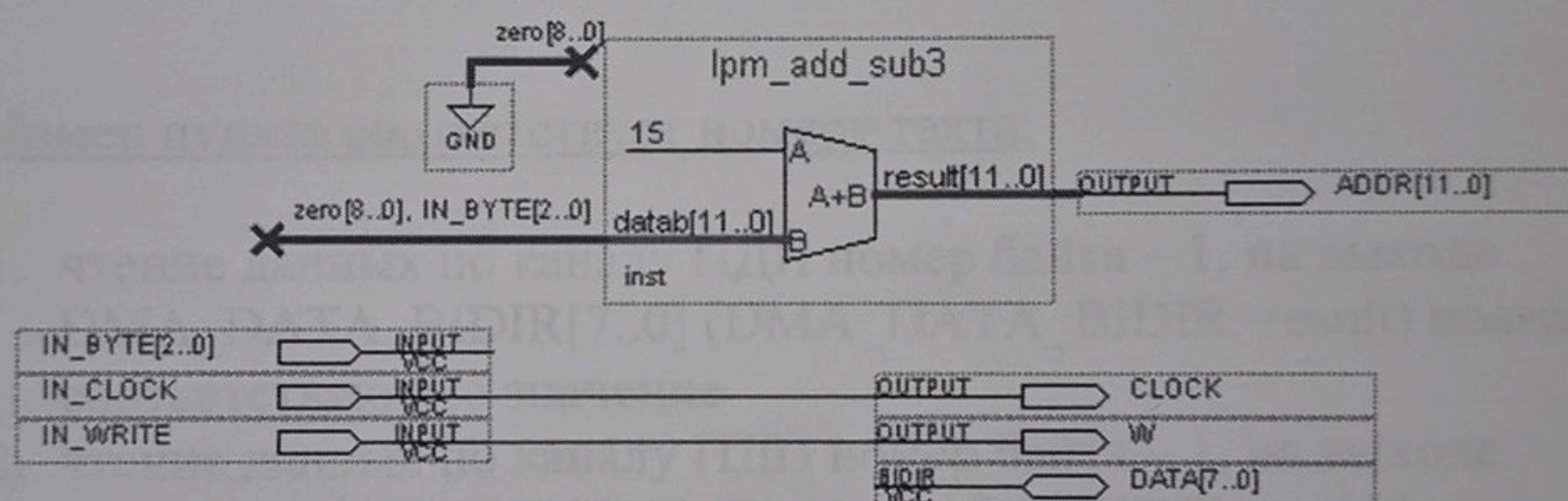


Рисунок 2.64 - Схема блока PDIP

lpm_add_sub3 – сумматор, обеспечивающий PDIP в заданном адресном диапазоне ([0x00F; 0x017])

Назначение пинов блока:

- **DATA[7..0]** – шина данных (BIDIR)
- **IN_BYTE[2..0]** – выбор номера байта для операции
- **ADDR[11..0]** – выходная адресная шина, идущая на блок памяти
- **IN_WRITE/W** – выбор режима обмена с памятью
- **IN_CLOCK/CLOCK** – тактовый сигнал

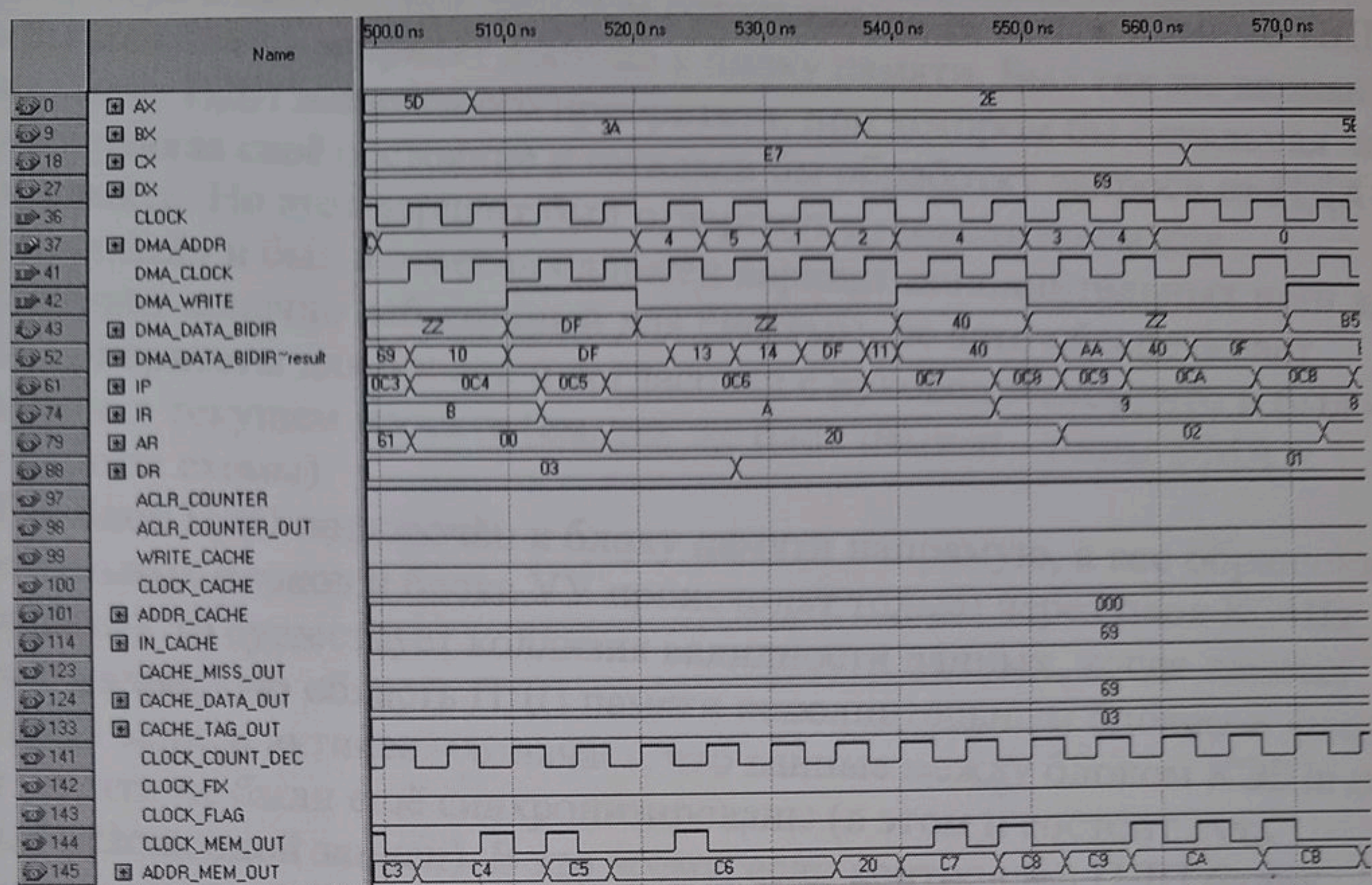


Рисунок 2.65 - Временная диаграмма функционального моделирования работы блока ПДП

Номер пункта соответствует номеру такта:

1. чтение данных по каналу ПДП номер байта – 1, на выходе DMA_DATA_BIDIR[7..0] (DMA_DATA_BIDIR~result) появилось соответствующее значение
2. чтение данных по каналу ПДП номер байта – 1, на выходе DMA_DATA_BIDIR[7..0] (DMA_DATA_BIDIR~result) появилось соответствующее значение
3. запись данных 0xDF по каналу ПДП номер байта – 1
4. запись данных 0xDF по каналу ПДП номер байта – 1
5. чтение данных по каналу ПДП номер байта – 4, на выходе DMA_DATA_BIDIR[7..0] (DMA_DATA_BIDIR~result) появилось соответствующее значение
6. чтение данных по каналу ПДП номер байта – 5, на выходе DMA_DATA_BIDIR[7..0] (DMA_DATA_BIDIR~result) появилось соответствующее значение
7. чтение данных по каналу ПДП номер байта – 1, на выходе DMA_DATA_BIDIR[7..0] (DMA_DATA_BIDIR~result) появилось соответствующее записанное ранее значение – 0xDF

Т.к. архитектура системы – принстонская – имеет совмещённые шины для RAM и ROM, то совмещённые шины всё время заняты – на них либо данные из ОЗУ, либо код команды-операнд-адрес или что-то ещё из ПЗУ – все команды оптимизированы так, что на каждом такте каждой команды эти общие шины заняты и нет никакой возможности обеспечить корректный

прямой доступ к памяти, при котором остальная часть схемы помимо ПДП имела бы меньший приоритет доступа к блоку памяти. Был так же вариант задания блоку ПДП наивысшего приоритета, при котором бы остальная часть схемы сохраняла своё состояние и ожидала бы обработку запроса от ПДА блоком памяти. Но этот вариант был отвергнут руководителем как «некорректный» и был в ответ предложен вариант дополнительных шин для ПДП. Вариант конечно избыточный для системы, но зато обеспечивает нужные приоритеты для блоков и согласован с руководителем, что и было реализовано в текущем проекте (см. так же блок памяти – пины DMA и логику работы схемы).

Т.к. блок ПДП подключён к блоку памяти напрямую, а все обращения исполнительных блоков и блока УУ происходят только через блок КЭШа (кэшируются), то существует коллизия валидности данных, когда данные записаны в адресную область ПДП памяти исполнительным блоком и флаг отложенной записи активен – означает, что данные между блоком КЭШа и блоком памяти не были ещё синхронизированы (в этом и состоит суть флаговой отложенной записи). В это время, если контроллер ПДП будет считывать данные из памяти, то он получит невалидные данные – старые данные, которые не были обновлены после операции записи – в этом случае валидные данные находятся только в блоке КЭШа. Есть вариант решения – не кэшировать область памяти ПДП, но ведь в том и суть, чтобы ПДП обеспечивал быстрый доступ к памяти без каких-либо значительных задержек. В любом случае не составит труда добавить в блок КЭШа два компаратора и в случае попадания адреса в диапазон адресов ПДП прокидывать сигналы непосредственно на блок памяти – т.е. не кэшировать попросту адресный диапазон ПДП. Тем не менее, в данном проекте был выбран метод реализации, подробно описанный в предыдущем абзаце.

2.9 Предсказатель переходов

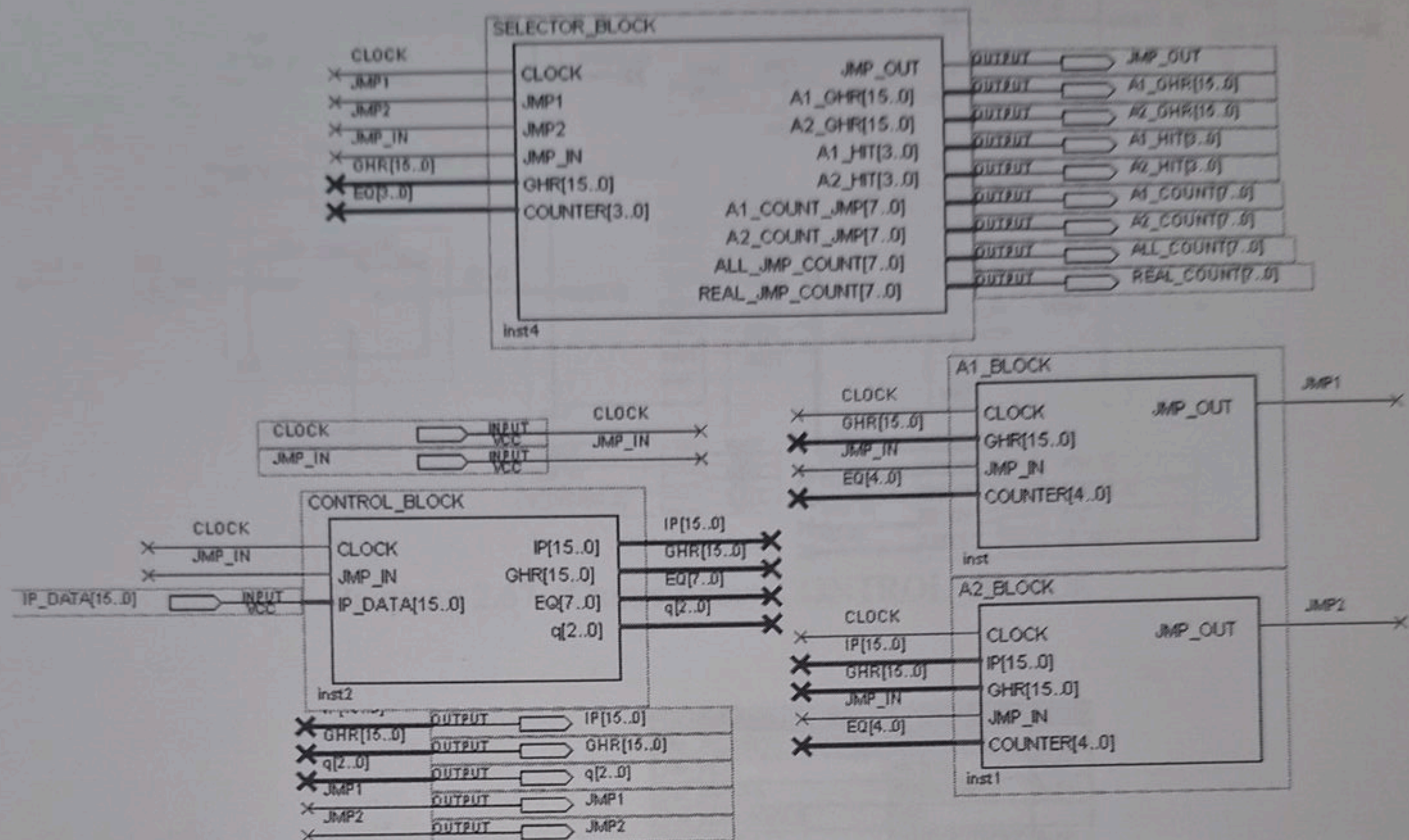


Рисунок 2.66 - Корневая схема блока предсказателя переходов

Перечислим основные блоки схемы:

- CONTROL_BLOCK – блок управления, содержит в себе регистры IP и GHR, а так же осуществляет контроль итераций
- A1_BLOCK – схема предсказаний, формирование шаблона по 4 битам GHR
- A2_BLOCK – схема предсказаний, формирование шаблона по 4 битам GHR xor IP
- SELECTOR_BLOCK – блок селектора, выбирает предсказание наиболее от того блока, который ошибался меньшее число раз

Блок предсказания переходов не входит в главный проект ввиду отсутствия в проекте условных переходов (бессмысленно его интегрировать со всей предыдущей схемой). Поэтому блок сильно детально рассмотрен не будет.

GHR[15..0] IP[15..0]

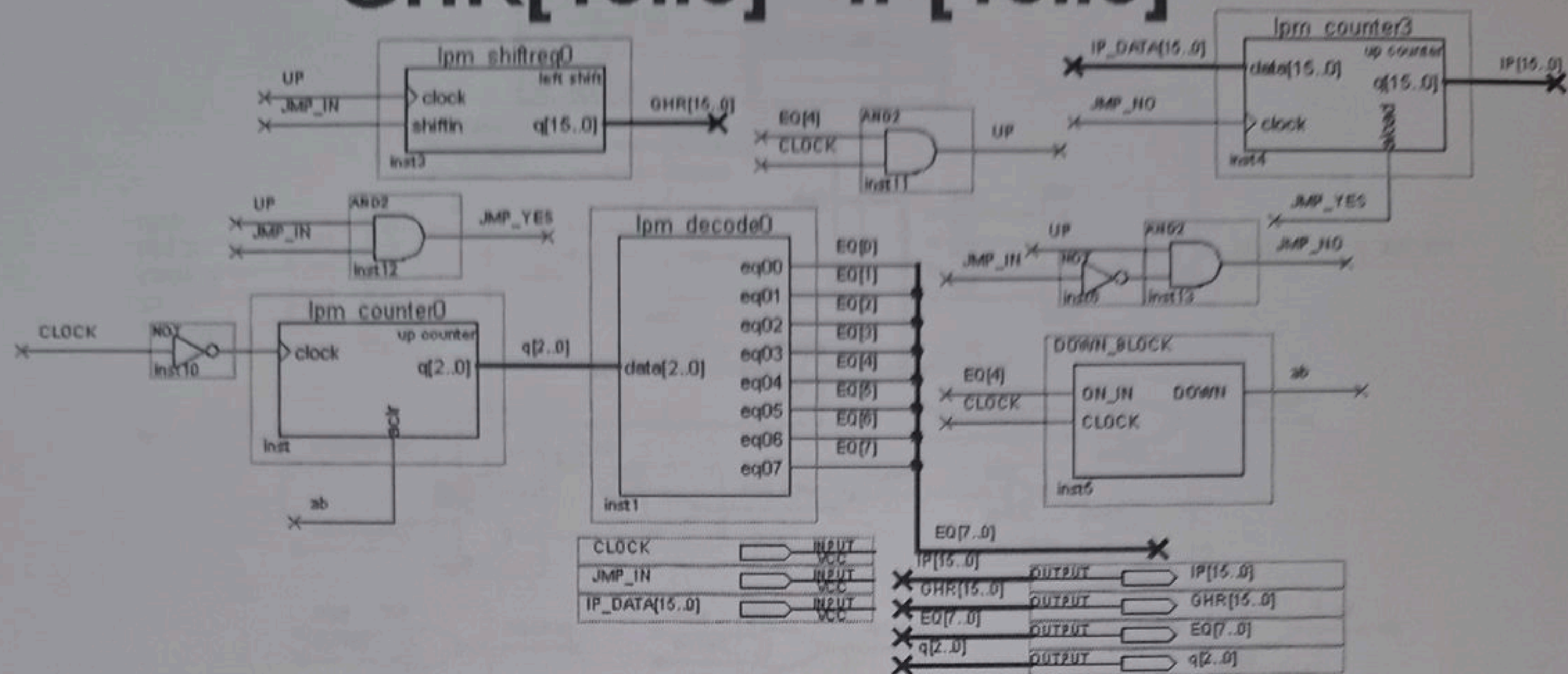


Рисунок 2.67 - Схема блока CONTROL_BLOCK

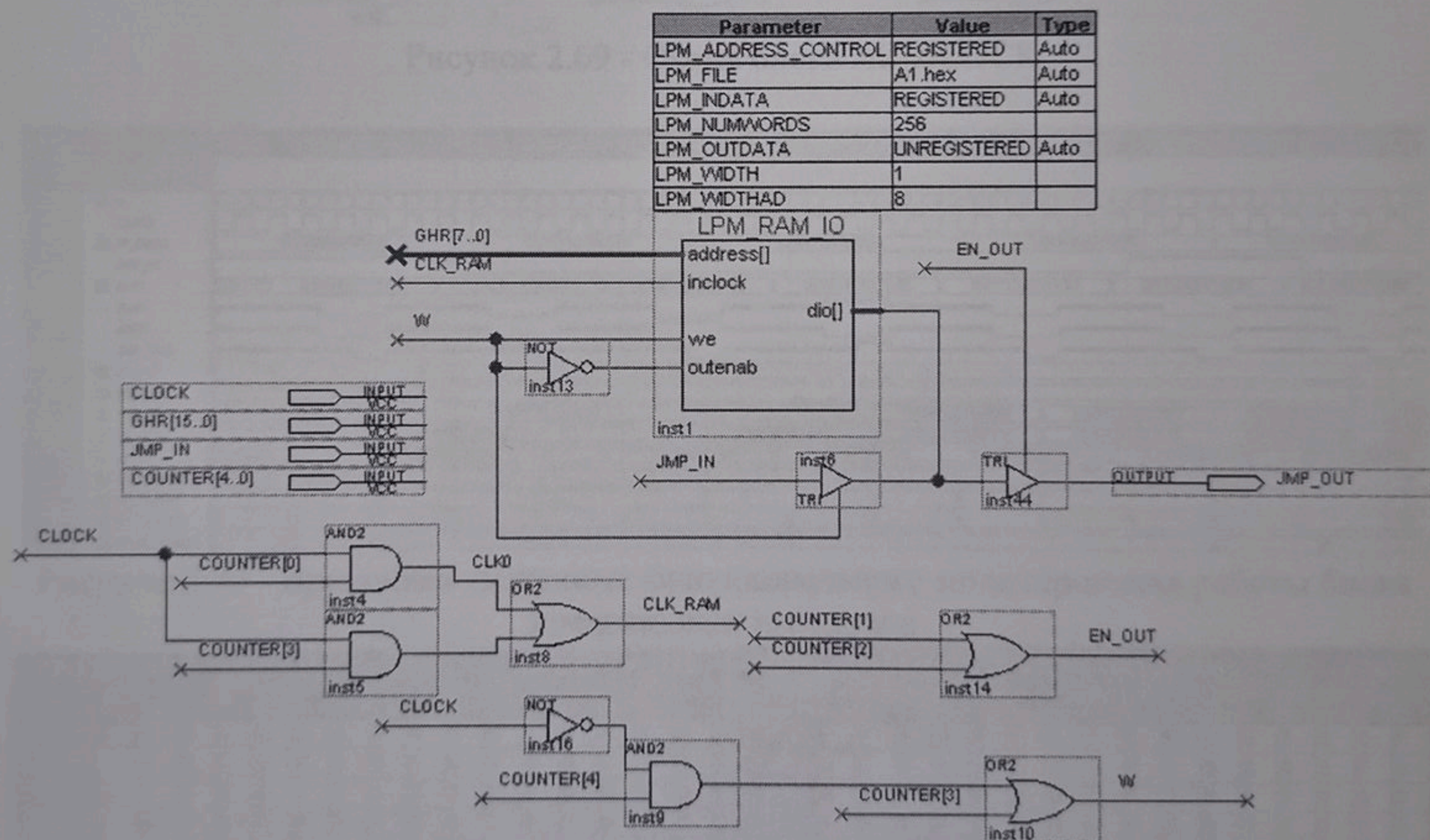


Рисунок 2.68 - Схема блока A1_BLOCK

Схема блока CONTROL_BLOCK представлена в приложении III. Первоначально перед моделированием все блоки памяти схем предсказателей переходов были проинициализированы нулями.

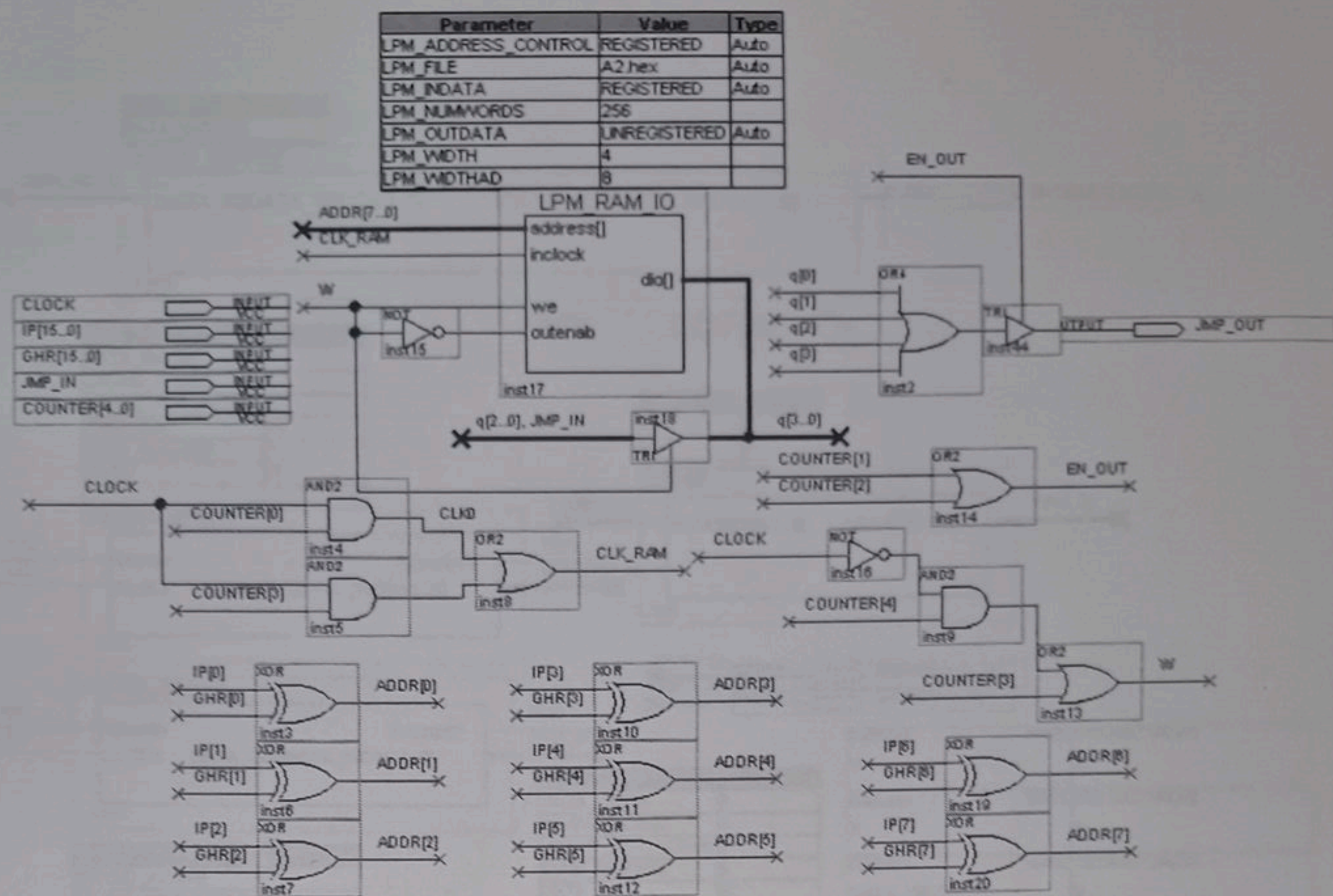


Рисунок 2.69 - Схема блока A2_BLOCK

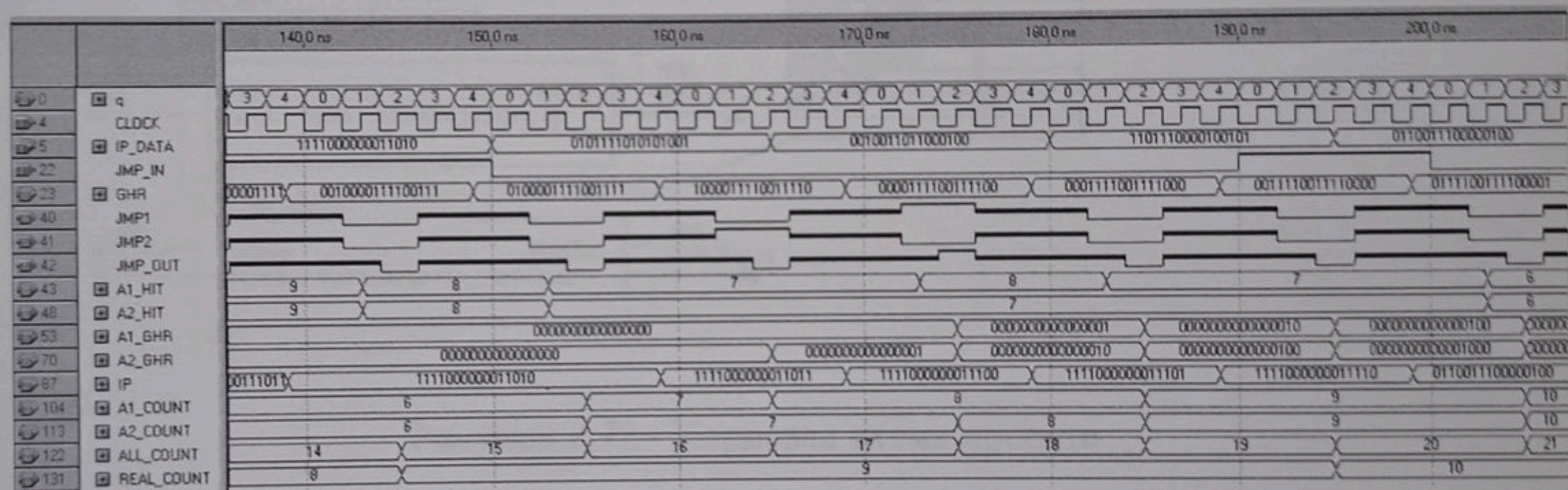


Рисунок 2.70 - Временная диаграмма функционального моделирования работы блока предсказания переходов

[MAIN_MODULE A1_BLOCK:inst1 lpm_ram_io:inst1 altram:st															
Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+a	+b	+c	+d	+e
00	1	1	0	1	0	0	0	0	0	0	0	0	0	1	0
10	1	0	0	1	0	0	0	0	1	0	0	0	0	1	0
20	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
30	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0
40	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0
50	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0
60	1	0	0	0	1	0	1	0	0	0	1	0	0	0	0
70	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0
80	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0
90	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0
a0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
b0	0	0	0	0	0	1	0	0	0	0	0	1	1	0	0
c0	0	0	0	0	1	0	0	0	1	0	0	0	0	1	0
d0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0
e0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
f0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0

[MAIN_MODULE A2_BLOCK:inst1 lpm_ram_io:inst17 altram															
Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+a	+b	+c	+d	+e
00	F	F	0	0	0	F	0	0	F	F	0	0	F	0	F
10	0	0	0	0	0	F	0	0	0	0	0	0	0	0	F
20	0	0	F	0	0	0	0	0	F	0	0	0	0	0	0
30	F	F	0	0	0	0	0	0	0	0	0	0	0	0	0
40	0	0	0	0	0	0	0	0	0	0	0	F	0	0	0
50	0	0	0	F	0	0	0	0	0	0	F	0	0	0	0
60	0	0	0	F	0	0	0	0	F	0	0	F	0	0	0
70	F	0	0	0	0	0	0	F	0	0	0	0	F	0	0
80	0	0	0	0	0	0	0	0	F	F	0	0	0	0	0
90	0	0	0	0	0	F	0	0	0	0	0	0	0	0	0
a0	0	0	0	0	0	0	0	0	0	0	0	F	0	0	0
b0	0	0	F	0	0	0	0	0	0	0	0	0	0	0	0
c0	0	0	0	0	0	0	F	0	0	0	0	F	0	0	F
d0	0	0	F	0	0	0	0	0	0	0	0	0	0	0	F
e0	0	F	0	0	F	0	0	0	F	0	F	F	0	F	0
f0	0	0	0	0	0	0	0	F	0	0	0	0	0	F	0

Рисунок 2.71 - Дампы памяти блоков A1 (слева) и A2 (справа) после моделирования

2.10 Организация арбитража

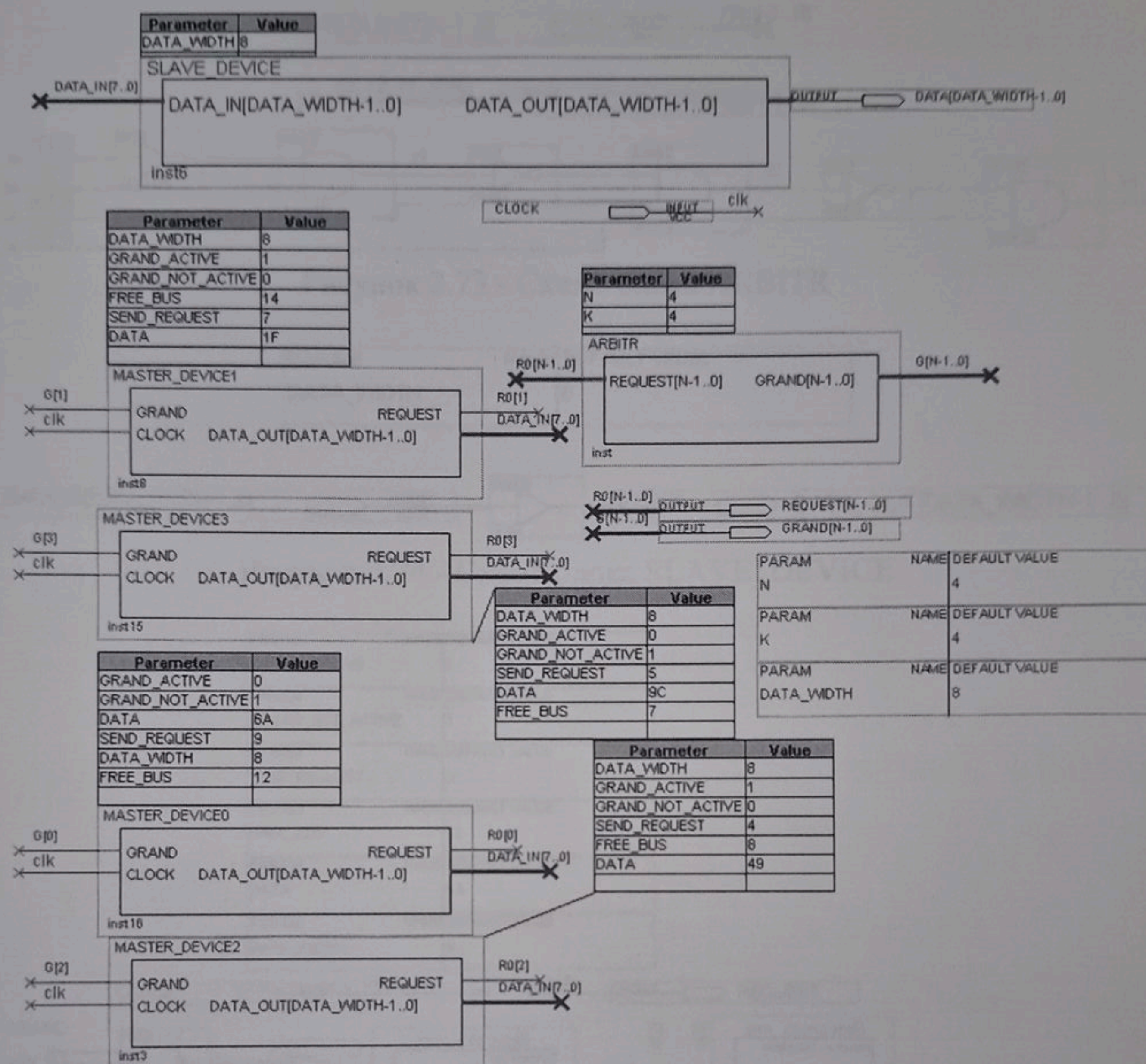


Рисунок 2.72 - Корневая схема проекта

Назначение основных элементов схемы блока арбитража:

- MASTER_DEVICE_x – ведущее устройство, способное захватывать шину, x- номер устройства
- ARBITR – арбитр, устанавливает приоритеты доступа к шине
- SLAVE_DEVICE – ведомое устройство, просто прокидывает данные на шину далее, имеется для наглядности

Т.к. арбитраж не включён в схему главного проекта, детальное рассмотрение работы не требуется, ввиду своей тривиальности.

Все блоки ведущих устройств имеют одну и ту же схему, отличия заключаются только в том, на каком такте тот или иной блок шлёт запрос на предоставление шины на блок арбитра, сколько тактов он удерживает шину и какие данные он на неё выставляет.

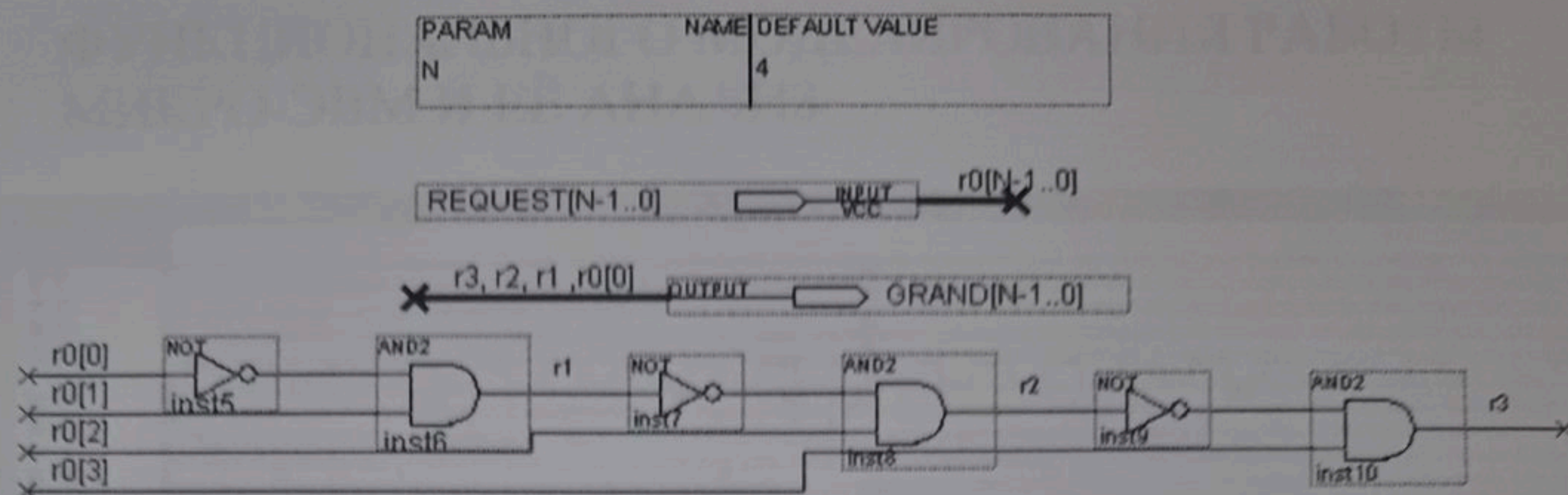


Рисунок 2.73 - Схема блока ARBITR

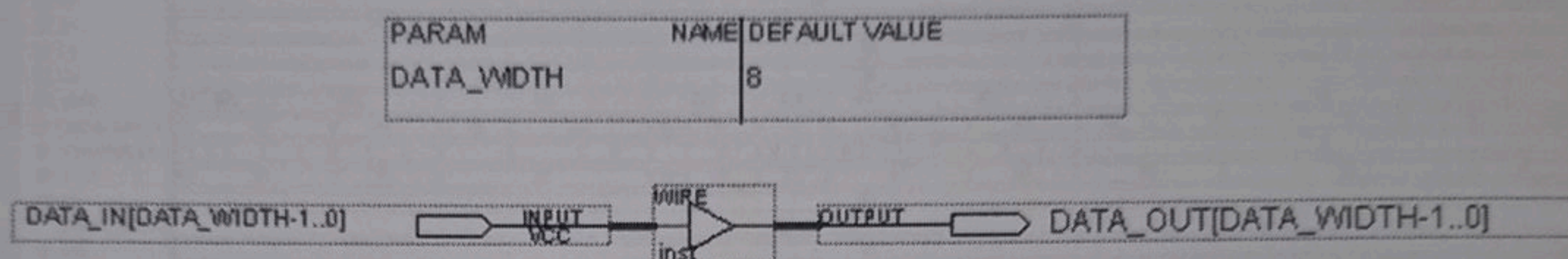


Рисунок 2.74 - Схема блока SLAVE_DEVICE

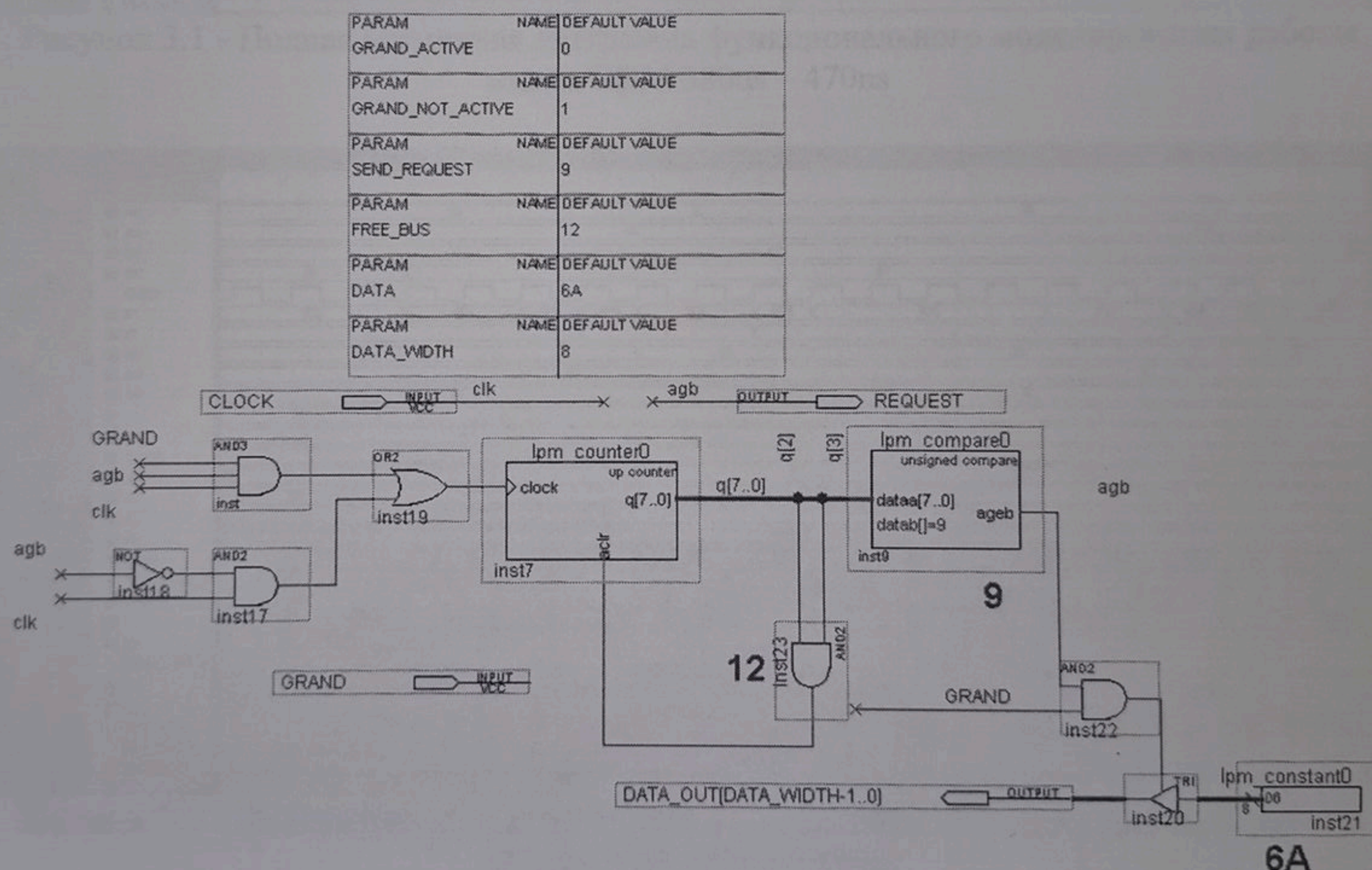


Рисунок 2.75 - Схема блока MASTER_DEVICE

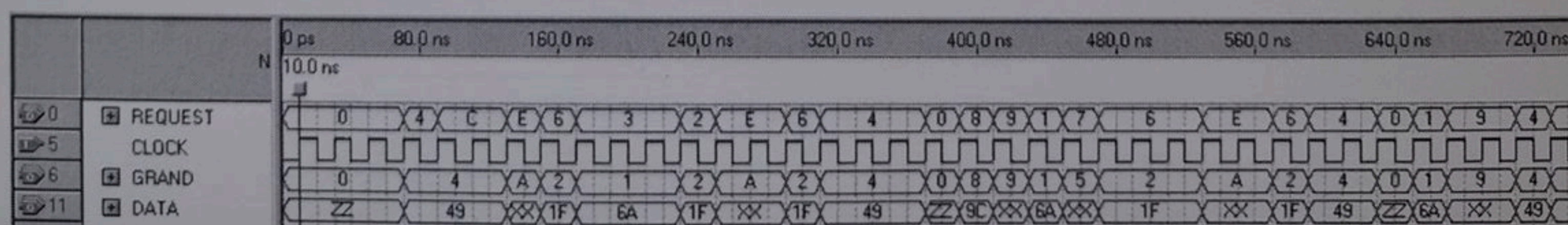


Рисунок 2.76 - Временная диаграмма функционального моделирования работы блока арбитража

3 ПОЛНАЯ ВРЕМЕННАЯ ДИАГРАММА

ФУНКЦИОНАЛЬНОГО МОДЕЛИРОВАНИЯ РАБОТЫ МИКРО-ЭВМ И ЕЁ АНАЛИЗ

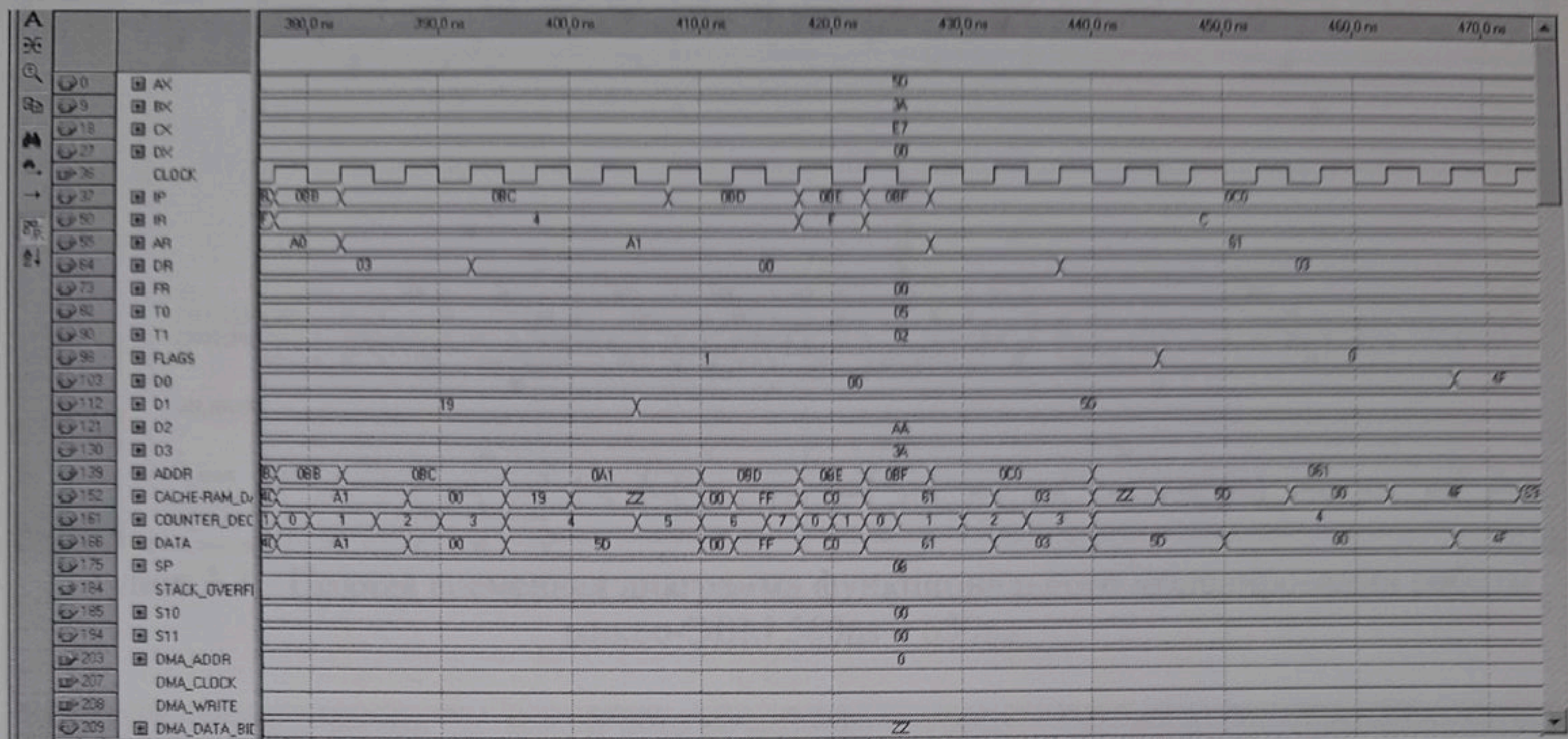


Рисунок 3.1 - Полная временная диаграмма функционального моделирования работы микро-ЭВМ 380ns – 470ns

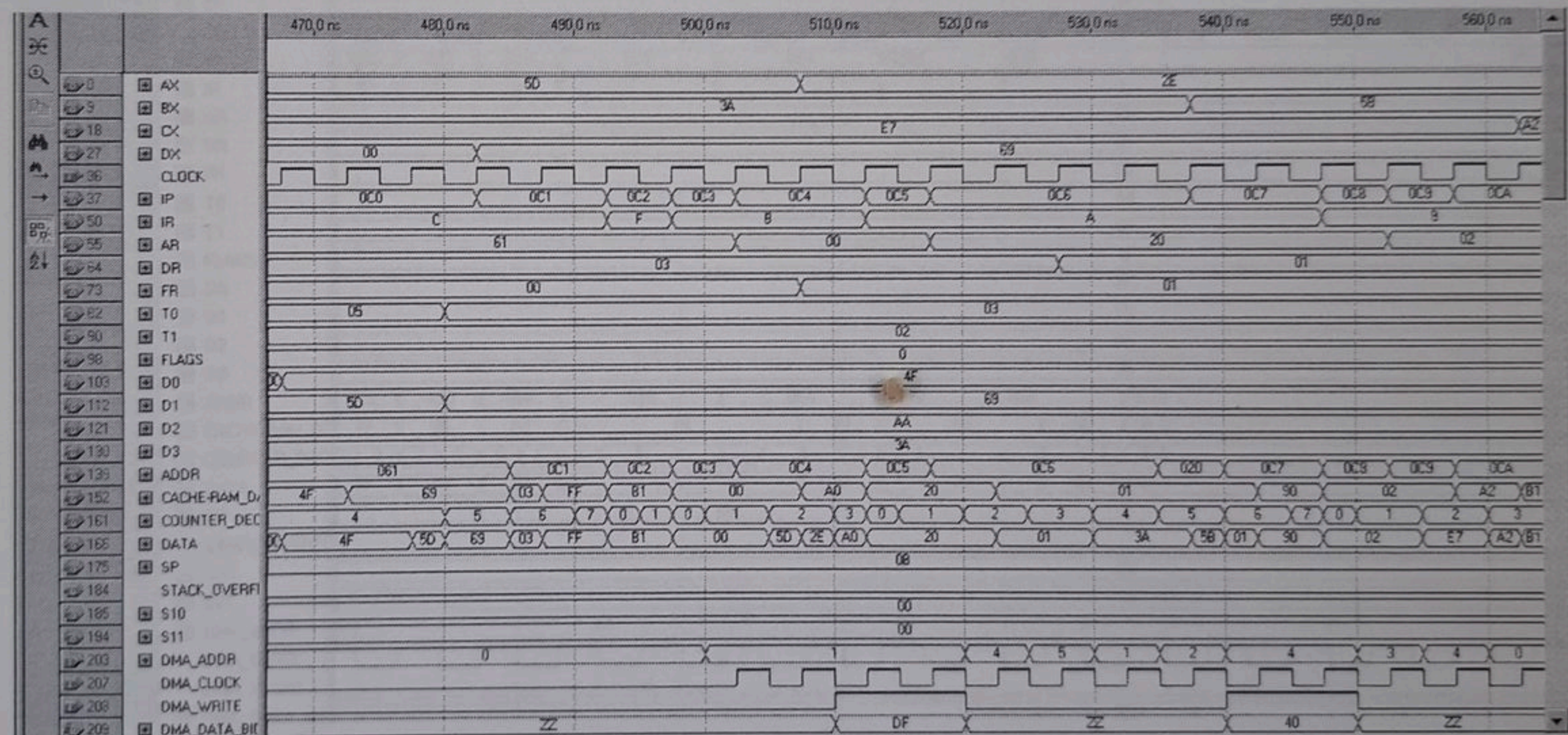
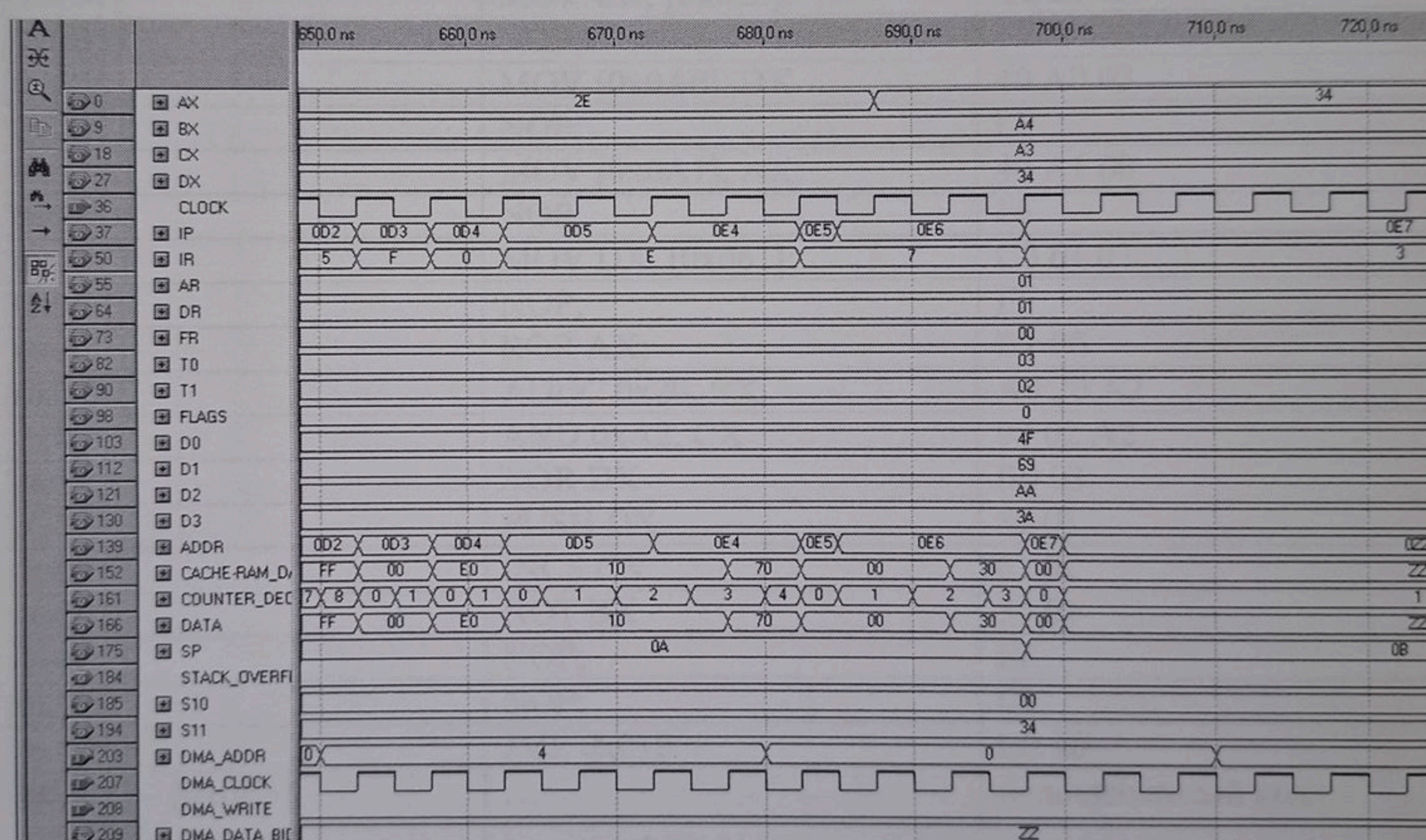
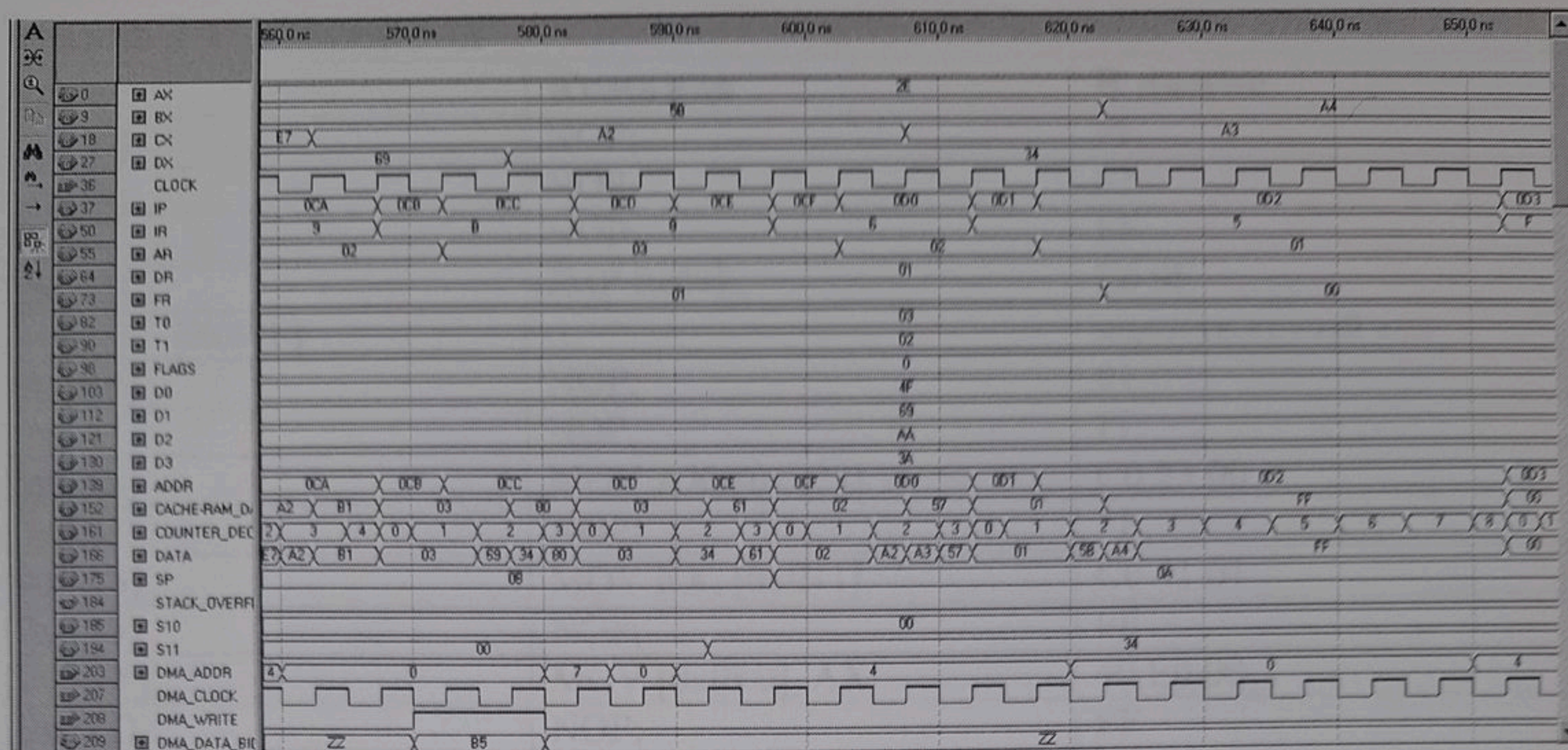


Рисунок 3.2 - Полная временная диаграмма функционального моделирования работы микро-ЭВМ 470ns – 560ns



Корневая структурная схема проекта представлена в приложении Щ.

Таблица 3.1 - Псевдокод

Адрес	Команда	В дампе
[0x000]	NOP;	FF
[0x001]	NOP;	02
[0x002]	NOP;	FF
[0x003]	JMP 0x09E;	E0 9E
[0x004] – [0x09F]	...	недостижимый код
[0x0A0]	NOP;	01
[0x0A1]	NOP;	FF
[0x0A2]	MOV AX, [0x023]	C0 23 00
[0x0A5]	NOP;	FF
[0x0A6]	MOV BX, [0x041]	C0 41 01
[0x0A9]	NOP;	FF
[0x0AA]	MOV [0x0E0], AX	40 E0 00
[0x0AD]	NOP;	FF
[0x0AE]	MOV [0x0E1], BX	40 E1 01
[0x0B1]	NOP;	FF
[0x0B2]	MOV CX, [0x022];	C0 22 02
[0x0B5]	NOP;	FF
[0x0B6]	MOV [0x0A0], DX	40 A0 03
[0x0B9]	NOP;	FF
[0x0BA]	MOV [0x0A1], AX;	40 A1 00
[0x0BD]	NOP;	FF
[0x0BE]	MOV DX, [0x061];	C0 61 03
[0x0C1]	NOP;	FF
[0x0C2]	ROR AX;	B1 00
[0x0C4]	ADDC 0x20, BX	A0 20 A2
[0x0C7]	AND 0xA2, CX	90 02 A2
[0x0CA]	ROR DX	B0 03
[0x0CC]	PUSH DX	80 03
[0x0CE]	INCS CX	61 02
[0x0D0]	NOT BX	57 01
[0x0D2]	NOP;	FF
[0x0D3]	NOP;	00
[0x0D4]	JMP 0x010	E0 10
[0x0D5] – [0x0E3]	...	недостижимый код
[0x0E4]	POP AX	76 00
[0x0E6]	HALT	32
[0x0E7] – [0xFFFF]	...	недостижимый код

Каждая операция представленная в таблице 3.1 и её итерация рассмотрены подробно в разделе 2, за детальным рассмотрением итераций операции см. соответствующий подраздел раздела 2.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно разработана специализированная микро-ЭВМ с заданными характеристиками и полным заданным набором команд.

Т.к. микро-ЭВМ имеет совмещенные общие шины для RAM и ROM, а синхронизация с блока данных КЭШа с блоком памяти задана флаговая отложенная по варианту, то прямой доступ к памяти затруднён. Затруднён, поскольку все операции ЭВМ агрессивно оптимизированы – так, что в каждый момент времени (на каждом такте) неизбежно задействована общая шина данных. При этом выходом для прямого доступа из этой ситуации является задание блоку ПДП наивысшего приоритета доступа к общим шинам, что нарушает сам принцип работы КПДП – он должен иметь доступ к шинам только тогда, когда из не использует процессор (остальная часть схемы – устройство управления и некоторый исполнительный блок). Либо другой вариант, предложенный руководителем, - сделать блок памяти с несколькими шинами (дополнительные шины управления для ПДП – адресная, данных и управления), что и было сделано в текущем проекте.

Тип синхронизации блока данных КЭШа и блока памяти имеет некоторую особенность функционирования – при отложенной флаговой записи данные в памяти не всегда валидны – случай, когда в блоке флагов имеется хотя бы один активный флаг – сигнал о том, что уже как минимум одна ячейка памяти является невалидной (валидные данные находятся в блоке данных КЭШа). В таком случае, если адрес этой ячейки (номер строки, тэг, смещение) попадают в адресный диапазон ПДП, то КПДП прочитает невалидные данные и будет думать, что они являются верными, когда это не так. В другом же случае, если КПДП осуществит запись, при таком раскладе, данные будут испорчены блоком синхронизации КЭШа, т.к. после считывания таких данных непосредственно после записи их КПДП получит совсем другие данные, которые были выгружены блоком синхронизации КЭШа. Все эти коллизии можно устранить либо изменив корне архитектуру микро-ЭВМ на архитектуру с отдельными шинами RAM и ROM, либо путём исключения адресного диапазон ПДП из кэшируемого адресного диапазона.

СПИСОК ЛИТЕРАТУРЫ

1. D. A. Patterson, J. L. Hennesy – Computer Organization and Design, Morgan Kaufmann, 2009.
2. Самаль Д.И - Структурная и функциональная организация ЭВМ. - БГУиР, кафедра ЭВМ, 2013.
3. Зотов В.Ю. –Проектирование Встраиваемых Микропроцессорных Систем На Основе ПЛИС Фирмы XILINX, 2006.
4. Сергиенко А.М. – VHDL Для Проектирования Вычислительных.Устройств, 2003
5. Поляков А.К. – Языки VHDL и VERILOG в Проектировании Цифровой Аппаратуры, Москва СОЛОМОН-Пресс, 2003
6. И.И. Глецевич, В.А. Прытков, А.В. Отвагин - Методические указания по дипломному проектированию для студентов специальности 40 02 01 «Вычислительные машины, системы и сети» всех форм обучения, Минск, 2013.
7. Documentation: Quartus II Development Software – режим доступа: <http://www.altera.com/literature/lit-qts.jsp>.

ПРИЛОЖЕНИЕ А
(обязательное)
Система команд микро-ЭВМ